

VILNIAUS UNIVERSITETAS

MATEMATIKOS IR INFORMATIKOS FAKULTETAS

DUOMENŲ MOKSLO STUDIJŲ PROGRAMA

Duomenų mokslo projektas

Kalbos modelių taikymas Lietuvos Respublikos Seimo 2016–2020 m. ir 2020–2024 m. kadencijų posėdžių stenogramų analizei

Application of Language Models for the Analysis of Transcripts of the Seimas of the Republic of Lithuania for the 2016–2020 and 2020–2024 Terms

Arnas Kazanavičius

Naglis Bražėnas

Justina Seiliūnaitė

Darbo vadovas : Asist., Dr. Andrius Buteikis

Vilnius
2026

Santrauka

Šio projekto pagrindinis tikslas yra pritaikyti ir palyginti kalbos modelius, leidžiančius Lietuvos Respublikos Seimo posėdžių stenogramas klasifikuoti į pagrindines posėdžių temas, išskirti svarbiausias frazes, atskleidžiančias šių temų turinį, bei nustatyti jų emocinį toną, siekiant palyginti 2016–2020 m. ir 2020–2024 m. Seimo kadencijas.

Šiame darbe analizuojami lietuviški politiniai tekstai, kurių analizei taikyti tiek tradiciniai metodai, tiek neuroniniais tinklais pagrįsti kalbos modeliai. Klasifikavimui pagal temas buvo naudotas LitLatBERT modelis, vėliau svarbiausių frazių išskyrimui taikyti TF–IDF, YAKE ir KeyBERT metodai. Sentimentų analizei naudoti VADER ir BERT modeliai.

Analizės metu nustatyta, kad papildomai apmokytas LitLatBERT modelis tiksliausiai klasifikavo stenogramas į diskusijų temas, TF–IDF ir YAKE metodai pateikė lengviau interpretuojamas svarbiausias frazes, o VADER metodas geriausiai atspindėjo neutralų politinės kalbos pobūdį sentimentų analizėje. Lyginant kadencijas pastebėta, kad 2016–2020 m. ir 2020–2024 m. kadencijose daugiausiai dėmesio skirta teisės, vidaus politikos bei socialinės politikos klausimams, tačiau ekonomikos bei energetikos temose išryškėjo skirtumai tarp laikotarpių. Abiejose analizuotose kadencijose, temų svarbiausios frazės daugiausiai aptartos pozityvaus emocinio tono kontekstuose, o skirtumai tarp kadencijų minimalūs.

Raktiniai žodžiai: kalbos modeliai, Lietuvos Respublikos Seimas, posėdžių stenogramos, teksto klasifikavimas, sentimentų analizė, svarbiausių frazių išskyrimas.

Summary

The main objective of this project is to adapt and compare language models that enable the classification of transcripts from sessions of the Seimas of the Republic of Lithuania into main session topics, the identification of key phrases that reveal the content of these topics, and the determination of the emotional tone of the sessions, with the aim of comparing the 2016–2020 and 2020–2024 terms of the Seimas.

This project analyzes Lithuanian political texts, employing both traditional methods and neural network-based language models for analysis. The LitLatBERT model was used to classify texts into thematic categories, followed by the application of TF–IDF, YAKE, and KeyBERT methods to identify the most important phrases. VADER and BERT models were used for sentiment analysis.

The analysis revealed that the fine-tuned LitLatBERT model was the most accurate at classifying transcripts into discussion topics, TF–IDF and YAKE methods provided the most important phrases that were easier to interpret, and VADER method best reflected the neutral nature of political discourse in sentiment analysis. A comparison of the terms revealed that during the 2016–2020 and 2020–2024 terms, the greatest attention was paid to issues of law, domestic policy, and social policy; however, differences between the periods became apparent in the topics of economics and energy. In both terms analyzed, the most important phrases related to these topics were discussed primarily in contexts with a positive emotional tone, and the differences between the terms were minimal.

Keywords: language models, Seimas of the Republic of Lithuania, parliamentary session transcripts, text classification, sentiment analysis, key phrase extraction.

Iliustracijų sąrašas

1 2016–2020 m. kadencijos žodžių debesis prieš teksto valymą 18
2 2020–2024 m. kadencijos žodžių debesis prieš teksto valymą 19
3 2016–2020 m. kadencijos žodžių debesis po teksto valymo 21
4 2020–2024 m. kadencijos žodžių debesis po teksto valymo 21
5 Posėdžių temų pasiskirstymas pagal kadencijas 25
6 Silueto metodo įvertis pagal klasterių skaičių 26
7 Mišrių klasterių prognozuotų klasių pasiskirstymas 28
8 Prognozuotų LR Seimo posėdžių temų pasiskirstymas 29
9 Diskusijų temų svarbiausių frazių VADER sentimentų pasiskirstymas (2016–2020) (1) 39
10 Diskusijų temų svarbiausių frazių VADER sentimentų pasiskirstymas (2020–2024) (1) 40
11 Diskusijų temų svarbiausių frazių BERT sentimentų pasiskirstymas (2016–2020) (1) . 43
12 Diskusijų temų svarbiausių frazių BERT sentimentų pasiskirstymas (2020–2024) (1) . 44
13 Diskusijų temų svarbiausių frazių VADER sentimentų pasiskirstymas (2016–2020) (2) 59
14 Diskusijų temų svarbiausių frazių VADER sentimentų pasiskirstymas (2020–2024) (2) 60
15 Diskusijų temų svarbiausių frazių BERT sentimentų pasiskirstymas (2016–2020) (2) . 60
16 Diskusijų temų svarbiausių frazių BERT sentimentų pasiskirstymas (2020–2024) (2) . 61

Lentelių sąrašas

1 Stenogramų aprašomoji statistika pagal kadencijas 17
2 Sakinių aprašomoji statistika pagal kadencijas 17
3 Aktyviausi kalbėtojai pagal kadencijas 19
4 Teminių kategorijų grupių apibūdinimai 24
5 Mišrių temų klasteriai su dažniausiai pasitaikančiais žodžiais 27
6 LitLatBERT modelio mokymo ir validacijos rezultatai 27
7 Temų svarbiausios TF–IDF frazės pagal kadencijas 32
8 Temų svarbiausios YAKE frazės pagal kadencijas 33
9 Pradinės vidaus politikos ir institucijų temos svarbiausios KeyBERT frazės 35
10 Temų svarbiausios KeyBERT frazės pagal kadencijas 36
11 2016–2020 metų kadencijos svarbiausių frazių VADER sentimentai 41
12 2020–2024 metų kadencijos svarbiausių frazių VADER sentimentai 42
13 2016–2020 m. kadencijos klasteriai, jų raktažodžiai ir žymos 52
14 2020–2024 m. kadencijos klasteriai, jų raktažodžiai ir žymos 55

Turinys

Santrauka **2**

Summary **3**

Iliustracijų sąrašas **4**

Lentelių sąrašas **5**

Įvadas **8**

1. Literatūros analizė **10**

2. Metodologija **12**
2.1. Teksto klasifikavimas 12
2.2. TF–IDF vektorizavimas ir K–vidurkių klasterizavimas 12
2.3. Kosinusinis panašumas 13
2.4. YAKE svarbiausių frazių išskyrimo metodas 14
2.5. KeyBERT svarbiausių frazių išskyrimo metodas 14
2.6. Mašininis vertimas 15
2.7. VADER sentimentų analizės metodas 15
2.8. BERT sentimentų analizės metodas 15

3. Duomenys **17**
3.1. Pradinė duomenų analizė 17
3.1.1. Aprašomoji statistika 17
3.1.2. Dažniausiai pasikartojančių žodžių analizė 18
3.1.3. Aktyviausių kalbėtojų analizė 19
3.2. Duomenų apdorojimas 20

4. Teksto klasifikavimas **23**
4.1. Neapmokyto modelio tekstų klasifikavimas 23
4.2. Apmokyto modelio tekstų klasifikavimas 25
4.3. Metodų palyginimas 29

5. Svarbiausios frazės **31**
5.1. TF–IDF metodo analizė 31
5.2. YAKE metodo analizė 33
5.3. KeyBERT metodo analizė 34
5.4. Metodų palyginimas 36

6. Sentimentų analizė **38**
6.1. Konteksto išgavimas 38
6.2. Kontekstų vertimas į anglų kalbą 38
6.3. VADER sentimentų analizė 38
6.4. BERT sentimentų analizė 43
6.5. Sentimentų analizės metodų palyginimas 44

Rezultatai ir išvados **46**

A priedas: Dirbtinio intelekto naudojimas **51**

B priedas: Klasterizuotų tekstų lentelės **52**

C priedas: Sentimentų analizės grafikai **59**

D priedas: Python kodas **61**

Įvadas

Lietuvos Respublikos Seimo posėdžiai yra viena iš svarbiausių politinio proceso dalių, kurių metu yra svarstomi ir priimami įstatymai bei kiti valstybei ir jos piliečiams reikšmingi sprendimai. Šių posėdžių eiga yra detaliai dokumentuojama stenogramose, kurios atspindi sprendimų priėmimo procesą, seimo narių argumentus bei tarpusavio diskusijas. Rankinė stenogramų analizė reikalauja daug laiko, ypač, kai norima apimti visą kadenciją ar atlikti palyginimus tarp dviejų ar daugiau kadencijų. Kalbos modelių taikymas leidžia automatizuoti šį procesą ir tokiu būdu paversti posėdžių duomenis į struktūruotus rodiklius, kuriuos galima analizuoti politologijoje, komunikacijos srityje bei duomenų moksle. Stenogramų analizė taip pat skatintų institucijų skaidrumą, leidžiant sistemingai stebėti, kokios temos dominuoja, kaip kinta prioritetai bei kaip argumentuojami sprendimai. Taip pat kadencijų lyginimas yra itin aktualus, nes jis leidžia stebėti politinių diskusijų tendencijas tokias, kaip dominuojančios temos ir emocinio tono kitimas tam tikrose politinėse srityse.

Šio darbo tikslas yra pritaikyti ir palyginti kalbos modelius, leidžiančius Lietuvos Respublikos Seimo posėdžių stenogramas klasifikuoti į pagrindines posėdžių temas, identifikuoti svarbiausias frazes atskleidžiančias posėdžių temų turinį ir palyginti jų emocinį toną tarp 2016–2020 m. ir 2020–2024 m. kadencijų. Tikslui pasiekti iškelti uždaviniai:

1. Atlikti pradinę Lietuvos Respublikos Seimo posėdžių stenogramų analizę.
2. Atlikti pradinį Lietuvos Respublikos Seimo posėdžių stenogramų apdorojimą bei paruošti tekstus kalbos modelių taikymui;
3. Atlikti Lietuvos Respublikos Seimo stenogramų klasifikavimą į posėdžių temas naudojant skirtingus klasifikavimo metodus bei nustatyti, kuris teksto klasifikavimo metodas yra tiksliausias Lietuvos Respublikos Seimo stenogramoms;
4. Išskirti svarbiausias frazes kiekvienai posėdžių temų klasei taikant skirtingus modelius bei nustatyti, kuris modelis geriausiai padeda išskirti svarbiausias Lietuvos Respublikos Seimo stenogramų frazes;
5. Išversti Lietuvos Respublikos Seimo stenogramų svarbiausias frazes į anglų kalbą.
6. Taikant skirtingus modelius svarbiausioms posėdžių temų frazėms atlikti sentimentų analizę bei nustatyti, kuris modelis yra tiksliausias Lietuvos Respublikos Seimo posėdžių stenogramoms analizuoti;
7. Palyginti stenogramų klasifikavimo, svarbiausių frazių identifikavimo ir sentimentų analizės rezultatus tarp 2016–2020 m. ir 2020–2024 m. kadencijų.

Darbą sudaro šešios pagrindinės dalys. Pirmoje darbo dalyje pateikiama literatūros analizė, kurioje apžvelgiami natūralios kalbos apdorojimo metodai, teksto klasifikavimo, svarbiausių frazių išskyrimo bei sentimentų analizės modeliai ir ankstesni tyrimai, susiję su parlamentinių stenogramų

analize. Toliau pristatoma darbo metodologija ir aptariami taikyti metodai bei modeliai. Trečioje darbo dalyje aprašomi naudoti Lietuvos Respublikos Seimo posėdžių stenogramų duomenys, atliekama pradinė jų analizė bei duomenų paruošimas tolimesniam modelių taikymui. Vėlesnėse darbo dalyse atliekamas stenogramų klasifikavimas į pagrindines temas, išskiriamos svarbiausios frazės bei atliekama sentimentų analizė, lyginant skirtingų metodų ir modelių rezultatus. Darbo pabaigoje pateikiami rezultatų palyginimai tarp 2016–2020 m. ir 2020–2024 m. Lietuvos Respublikos Seimo kadencijų, apibendrinamos išvados.

1. Literatūros analizė

Politinių stenogramų analizė leidžia perteikti politikų ir politinių partijų požiūrį į svarbias visuomenei temas bei suprasti demokratinius procesus, vykstančius įstatymų leidžiamosiose institucijose, tačiau dėl didelio tekstų kiekio ir specifinės kalbos jų rankinė analizė yra sudėtinga, todėl politikos ir socialinių mokslų tyrėjai vis dažniau taiko kompiuterinius metodus politinių pozicijų tyrimui [AB20] [AB18]. Automatizuota teksto analizė suteikia galimybę reikšmingai sumažinti didelių tekstų rinkinių analizės kaštus, todėl tai tampa svarbiu įrankiu politologijos tyrimuose, kur politika ir jos konfliktai dažnai pasireiškia rašytiniu ar žodiniu pavidalu [GS13]. C. Kertcher ir M. Zwilling (žr. [KZ25]) teigia, kad sentimentai politinėse kalbose gali atskleisti požiūrį į problemas, užsienio politikos poziciją ir naudos siekiančias strategijas. Taip pat teigia, kad natūralios kalbos apdorojimo įrankių naudojimas padeda identifikuoti bei prognozuoti politinių pozicijų pokyčius. Tuo tarpu S. Bhatia ir D. P (žr. [BD18]) pabrėžia, kad politines pažiūras galima analizuoti ne tik vertinant sentimentus, bet ir atsižvelgiant, kaip jie išreiškiami skirtingų temų kontekste, nes toks požiūris leidžia tiksliau interpretuoti politines ideologijas.

Kalbant apie politinių tekstų klasifikavimą, Z. Terechshenko ir kt. (žr. [TLP$^+$20]) tyrimai parodė, kad geriausiai politinių tekstų klasifikavimui veikia Transformerių (angl. Transformer) modeliai tokie, kaip RoBERTa ir XLNet, ypač, kai neturima daug sužymėtų duomenų, nors tyrėjai vėliau pamini, kad jie negali visiškai panaikinti sužymėtų duomenų svarbos politologiniuose tyrimuose. Tačiau Yu Wang (žr. [Wan23]) teigia, kad iš anksto apmokyti kalbos modeliai gali efektyviai spręsti sužymėtų duomenų trūkumo problemą, o papildomai apmokyti naudojant vos 10 % sužymėtų duomenų gali pranokti kryžminės temų klasifikacijos metodus politiniuose tekstuose. Taip pat panašumu pagrįsti metodai yra efektyvus sprendimas neprižiūrimame teksto klasifikavime, nes leidžia susieti tekstus su klasių aprašymais remiantis semantiniu panašumu, nenaudojant anotuotų mokymo duomenų. Tyrimai rodo, kad tokie metodai gali pasiekti aukštą tikslumą ir dažnai pranokti alternatyvius mokymosi be pavyzdžių (angl. *zero–shot*) metodus [SBM22].

Tuo tarpu lietuvių kalbos tekstų klasifikavimo kontekste tyrimai rodo, kad specifiniai lietuvių kalbai pritaikyti modeliai veikia geriau nei daugiakalbiai. Pavyzdžiui, E. Kankevičiūtė ir kt. (žr. [KSM23]) nagrinėjo lietuviškų komentarų neapykantos kalbos aptikimą ir parodė, kad LitLatBERT modelis pasiekia geresnius rezultatus nei daugiakalbis mBERT, kur LitLatBERT pasiekia 72 % tikslumą, o tuo tarpu mBERT 63 %.

Svarbiausių frazių išskyrimas leidžia automatiškai identifikuoti pagrindines temas ir argumentus didelės apimties politiniuose tekstuose, tokiuose kaip Seimo stenogramos.

Literatūroje raktinių frazių išskyrimo metodai skirstomi į statistinius, grafinius ir vektorine reprezentacija (angl. *embedding*) pagrįstus metodus. Statistiniai metodai, kaip TF–IDF dažnai naudojami kaip atskaitos taškas, nes jie yra paprasti ir remiasi tik žodžių ar frazių dažniu, todėl neatsižvelgia į semantiką ir politiniuose tekstuose dažniau išskiria pasikartojančias formuluotes, o ne argumentus. Grafų metodai geriau atskleidžia teksto struktūrą, tačiau jų gebėjimas perteikti semantinę reikšmę yra ribotas [PT19].

Dėl šių ribojimų naujesniuose tyrimuose vis dažniau taikomi vektorine reprezentacija pagrįs-

ti metodai, kurie frazes vertina pagal jų semantinį panašumą visam dokumentui. Tai leidžia tiksliau identifikuoti ne tik temas, bet ir argumentus [BMH+18]. Be to, tyrimai rodo, kad BERT pagrindu sukurti modeliai dar labiau pagerina raktinių frazių išskyrimo tikslumą, nes geba modeliuoti kontekstą ir atpažinti semantiškai svarbias, tačiau ne visada dažnas frazes [DVM+21].

Kita vertus, pažangesni metodai vis dar reikalauja išsamesnių tyrimų, ypač mažiau išteklių turinčioms kalboms, tokioms kaip lietuvių kalba [RRU20]. Be to, ne anglų kalbų tekstų analizė gali kelti papildomų sunkumų dėl sudėtingos kalbos struktūros [BS25].

Sentimentų analizė, taikoma izoliuotoms svarbiausioms frazėms, neretai gali būti mažiau patikima, nes jų sentimentinė reikšmė priklauso nuo konteksto. Tyrimai rodo, kad tas pats žodis ar frazė skirtinguose kontekstuose gali įgyti skirtingą, net priešingą sentimentinį atspalvį, todėl vien tik pavienių žodžių ar frazių analizė gali būti klaidinanti. Dėl šios priežasties sentimentui nustatyti būtina atsižvelgti į kontekstinę informaciją, kuri leidžia tiksliau įvertinti tikrąją žodžio ar frazės poliarizaciją [AMB+15]. Atsižvelgiant į tai, sentimentų analizės tyrimuose dažnai lyginami skirtingo sudėtingumo metodai. Nuo paprastesnių, leksikonu paremtų modelių iki pažangesnių, kontekstą suvokiančių neuroninių tinklų.

Vienas iš dažniausiai naudojamų leksikonu pagrįstų metodų yra VADER. Šis modelis remiasi iš anksto sudarytu žodynu ir heuristinėmis taisyklėmis, leidžiančiomis įvertinti net ir trumpų tekstų sentimentą bei tam tikrus kontekstinius niuansus, todėl yra lengvai pritaikomas ir nereikalauja papildomo modelio apmokymo [You24]. Vis dėlto, nors VADER yra efektyvus paprastesnėse užduotyse, jam gali būti sudėtinga atpažinti subtilius niuansus, netiesioginę formuluotę ar nuo platesnio konteksto priklausančias reikšmes [KZ25].

Dėl to naujesniuose tyrimuose vis dažniau taikomi kontekstui jautrūs modeliai. Vienas reikšmingiausių pavyzdžių yra BERT, kuris leidžia modeliuoti dvikryptį kontekstą ir tiksliau interpretuoti žodžių ar frazių reikšmes priklausomai nuo jų vartojimo aplinkos [DCL+19]. Skirtingai nei VADER, BERT geba įvertinti platesnį kontekstą, tačiau net ir jis gali netiksliai interpretuoti kultūriškai ar istoriškai specifinius tekstus, o subtilesnė politinė kalba gali lemti skirtumus tarp šių modelių pateikiamų rezultatų [KZ25].

Nepaisant modelių sudėtingumo, sentimentų analizė mažų resursų kalbose gali būti riboto tikslumo, todėl dažnai tekstas pirmiausia išverčiamas į anglų kalbą ir tik tada atliekama analizė. Tyrimai rodo, kad toks metodas gali būti tikslesnis nei tiesioginė analizė originalo kalba, o mašininio vertimo rezultatai stipriai koreliuoja su žmogaus vertimu [HSF22] [RB25].

2. Metodologija

2.1. Teksto klasifikavimas

Teksto klasifikavimui bei svarbiausių frazių išskyrimui bus naudojamas LitLatBERT modelis, kuris yra apmokytas trimis kalbomis: lietuvių, latvių ir anglų. Šiame darbe modelis taikomas dviem skirtingais būdais. Kadangi neturime sužymėtų duomenų, jis naudojamas tekstų semantinių reprezentacijų (angl. *embeddings*) generavimui, o vėliau, turint dalinai sužymėtus duomenis, bus pritaikomas kaip klasifikatorius, atliekant papildomą modelio apmokymą (angl. *fine–tuning*). Šis modelis, veikia geriau nei daugiakalbis BERT modelis, nes apima tik tris kalbas. Taip pat jis naudoja XLM–RoBERTa–base architektūrą [EMB].

XLM–RoBERTa–base yra iš anksto apmokytas modelis, naudojantis maskuotos kalbos modeliavimo (MLM) metodą. Šio metodo metu atsitiktinai paslepiama apie 15 % sakinio žodžių, o modelis mokomas juos atspėti. Toks mokymo principas leidžia modeliui efektyviai išmokti dvipuses sakinio reprezentacijas [AI].

Modelio optimizavimo tikslas yra maksimizuoti neigiamą logaritminį tikėtinumą teisingai atspėti paslėptus žodžius:

$$\mathcal{L}_{MLM} = -\sum_{i \in M} \log P(x_i \mid \tilde{X}; \theta) \tag{1}$$

čia θ – modelio parametrai, o $P(x_i \mid \tilde{X}; \theta)$ – modelio prognozuota tikimybė, kad paslėptas žodis x_i yra teisingas, atsižvelgiant į visą paslėptą seką $\tilde{X}$ [JM23].

2.2. TF–IDF vektorizavimas ir K–vidurkių klasterizavimas

Kadangi neturime sužymėtų duomenų ir norime įsitikinti, ar tekstų klasifikavimas yra tikslesnis apmokius su 10 % duomenų, duomenų žymėjimui pasitelksime nestruktūrizuotų duomenų grupavimo metodą, naudojant TF–IDF vektorizaciją ir K–vidurkių klasterizavimą.

Teksto reprezentacijai naudojamas TF–IDF (angl. *Term Frequency–Inverse Document Frequency*) metodas, kuris kiekvienam terminui t dokumente d (mūsų atveju dokumentas yra laikomas posėdžio aptariamo klausimo stenogramos dalis) priskiria svorį pagal formulę [JM23]:

$$\text{tf-idf}(t,d) = \text{tf}(t,d) \cdot \log \frac{N}{df(t)} \tag{2}$$

čia $\text{tf}(t,d)$ yra termino dažnis dokumente, $df(t)$ – dokumentų skaičius, kuriuose pasitaiko terminas t, o N – bendras dokumentų skaičius. TF–IDF metodas leidžia sumažinti dažnų, tačiau mažai informatyvių žodžių įtaką.

Po vektorizavimo taikomas K–vidurkių algoritmas, kuris yra vienas iš paprasčiausių ir populiariausių neprižiūrimų klasterizavimo algoritmų. Šis algoritmas nenaudoja sužymėtų duomenų ir suskirsto duomenis į k klasterių naudodamas centrinį tašką kiekvienam klasteriui. Iš pradžių centrai

yra išdėstyti atsitiktinai, o vėliau jie juda link artimiausių duomenų taškų centro. Tai kartojama, kol centrai nebesikeičia arba jų pokyčiai tampa labai maži [Sá].

Šio algoritmo tikslas yra minimizuoti atstumus tarp duomenų taškų ir jų klasterių centrų, kas formaliai apibrėžiama taip:

$$J = \sum_{i=1}^{k} \sum_{x \in C_i} \|x - \mu_i\|^2 \tag{3}$$

čia k – klasterių skaičius, C_i – i–tasis klasteris, o μ_i – jo centroidas. Algoritmas iteratyviai priskiria duomenų taškus artimiausiam centroidui ir atnaujina centroidų reikšmes.

Optimalus klasterių skaičius k parinktas naudojant silueto (angl. *Silhouette*) metodą, kuris leidžia įvertinti, kiek duomenų taškai yra panašūs į jam priklausantį klasterį lyginant su kitais klasteriais. Šis koeficientas apskaičiuojamas taip:

$$s = \frac{b - a}{\mathsf{max}(a,b)} \tag{4}$$

čia a – vidutinis atstumas tarp nagrinėjamo taško ir kitų to paties klasterio taškų, o b – mažiausias vidutinis atstumas iki kito klasterio taškų.

Silueto koeficiento reikšmės yra tarp -1 ir 1. Kuo reikšmė artimesnė 1, tuo geriau duomenų taškas atitinka savo klasterį ir tuo aiškesnis klasterių atskyrimas, o neigiamos reikšmės rodo, kad taškas gali būti priskirtas netinkamam klasteriui. Optimalia laikoma ta k reikšmė, kuriai vidutinis silueto koeficientas yra didžiausias [Muk18].

2.3. Kosinusinis panašumas

Kosinusinio panašumo (angl. *cosine similarity*) matas, leidžiantis įvertinti semantinį panašumą tarp dviejų tekstinių vienetų. Klasifikavimo uždavinyje bus skaičiuojamas panašumas tarp teksto ištraukos ir klasės apibūdinimo, o svarbiausių frazių paieškoje bus skaičiuojamas panašumas tarp teksto ištraukos ir kandidatės frazės.

Kiekviena kandidatinė frazė ir visas dokumentas pirmiausia paverčiami vektorinėmis reprezentacijomis, naudojant iš anksto apmokytą kalbos modelį. Tuomet apskaičiuojamas kosinusinis panašumas tarp frazės ir dokumento vektorių, kuris apibrėžiamas kaip šių vektorių skaliarinės sandaugos ir jų ilgių sandaugos santykis.

$$\text{Kosinusinis panašumas} = \frac{A \cdot B}{\|A\|\|B\|} \tag{5}$$

čia A žymi frazės vektorių ir B žymi dokumento vektorių. Kosinusinio panašumo reikšmė pateikiama intervale nuo -1 iki 1, kur reikšmės artimos -1 rodo priešingą semantinę kryptį, reikšmės apie 0 – mažą arba neegzistuojantį panašumą, o reikšmės artimos 1 – didelį semantinį panašumą tarp lyginamų tekstinių vienetų [JM23].

2.4. YAKE svarbiausių frazių išskyrimo metodas

Svarbiausių frazių išskyrimui bus naudojamas neprižiūrimas raktinių žodžių ir frazių išskyrimo algoritmas YAKE (angl. *Yet Another Keyword Extractor*) metodas. YAKE metodas kiekvieną dokumentą analizuoja atskirai, todėl svarbiausių frazių reikšmingumas nustatomas remiantis ne viso tekstyno, o konkretaus dokumento turiniu. Pirmiausia tekstas suskaidomas į žodžius ir kandidatines frazes, pašalinami mažai informatyvūs žodžiai, o vėliau kiekvienam terminui apskaičiuojami keli statistiniai požymiai, tokie kaip žodžio dažnis dokumente, jo pozicija tekste, pasiskirstymas tarp sakinių, didžiųjų ir mažųjų raidžių ypatybes bei ryšiai su kitais žodžiais [CMP+20].

Kiekvienam žodžiui priskiriamas reikšmingumo svoris, kuris apskaičiuojamas sujungiant lokalius požymius. Žodžio w svoris gali būti aprašomas taip:

$$S(w) = \frac{T_{pos}(w) \cdot T_{rel}(w)}{T_{case}(w) + T_{freq}(w) + T_{sent}(w)} \tag{6}$$

čia $T_{pos}(w)$ žymi žodžio pozicijos tekste požymį, $T_{rel}(w)$ – žodžio ryšį su kitais terminais, $T_{case}(w)$ – didžiųjų ir mažųjų raidžių naudojimo ypatybes, $T_{freq}(w)$ – žodžio dažnį dokumente, o $T_{sent}(w)$ – žodžio pasiskirstymą skirtinguose sakiniuose. Mažesnė $S(w)$ reikšmė rodo didesnį žodžio svarbumą dokumente.

Kadangi analizuojant politines stenogramas dažnai svarbūs ne pavieniai žodžiai, o kelių žodžių junginiai, naudojamas YAKE kandidačių frazių išskyrimo įrankis. Kandidatinės frazės svoris apskaičiuojamas remiantis ją sudarančių žodžių svoriais:

$$S(kw) = \frac{\prod_{w \in kw} S(w)}{TF(kw) \cdot \left(1 + \sum_{w \in kw} S(w)\right)} \tag{7}$$

čia kw žymi kandidatinę frazę, $S(w)$ – atskiro frazę sudarančio žodžio svorį, o $TF(kw)$ – kandidatinės frazės dažnį dokumente. Kuo mažesnė $S(kw)$ reikšmė, tuo frazė laikoma svarbesne nagrinėjamame tekste.

2.5. KeyBERT svarbiausių frazių išskyrimo metodas

Svarbiausių frazių išskyrimui taip pat bus naudojamas semantiniu panašumu paremtas raktinių žodžių ir frazių išskyrimo metodas KeyBERT.

Pirmiausia analizuojamas tekstas suskaidomas į kandidatines frazes, sudarytas iš kelių iš eilės einančių žodžių junginių, taikant n–gramų metodą. Toliau visas dokumentas ir kiekviena kandidatinė frazė paverčiami vektorinėmis reprezentacijomis, naudojant iš anksto apmokytą kalbos modelį. Šiame darbe tam bus naudojamas LitLatBERT modelis, kadangi jis yra pritaikytas lietuvių kalbai. Kiekvienos kandidatinės frazės svarbumas nustatomas apskaičiuojant kosinusinį panašumą tarp dokumento ir kandidatės frazės vektorių. Gavus visų kandidačių frazių panašumo įverčius, frazės surikiuojamos mažėjančia tvarka pagal jų panašumą su dokumentu [Gro20]. Tolimesnei analizei atrenkamas iš anksto nustatytas skaičius geriausiai įvertintų frazių. Taikant KeyBERT metodą šiame darbe, svarbiausios frazės bus išskiriamos iš kiekvienos temos dokumentų atskirai.

2.6. Mašininis vertimas

Lietuvos Respublikos Seimo stenogramų teminių kategorijų svarbiausių frazių bei jų aplinkinių sakinių vertimui iš lietuvių kalbos į anglų kalbą buvo pasirinktas neuroninio mašininio vertimo modelis Helsinki–NLP/opus–mt–tc–big–lt–en. Jis yra paremtas OPUS–MT projektu ir naudoja Marian NMT transformerių architektūrą. Šis modelis apmokytas naudojant OPUS daugiakalbius tekstynus [NLP20].

2.7. VADER sentimentų analizės metodas

Sentimentų analizei naudosime VADER (angl. *Valence Aware Dictionary and Sentiment Reasoner*) modelį, kuris yra leksikonu ir taisyklėmis paremtas sentimentų analizės metodas, skirtas nustatyti teksto emocinę reikšmę. Šis metodas ypač tinkamas trumpų tekstų analizei, nes geba efektyviai įvertinti neformalią kalbą, įskaitant santrumpas, šnekamosios kalbos elementus ar kitus ypatumus.

VADER veikimo principas grindžiamas tuo, kad kiekvienam žodžiui priskiriama iš anksto nustatyta sentimentinė reikšmė, atspindinti jo emocinį krūvį. Šios reikšmės vėliau agreguojamos, taikant papildomas taisykles, kurios leidžia atsižvelgti į tam tikrus kontekstinius aspektus, pavyzdžiui, neiginius ar stiprinančius žodžius. Galutiniam teksto įvertinimui naudojamas bendrasis sentimentų rodiklis (angl. *compound score*). Jis apskaičiuojamas normalizuojant visų žodžių sentimentinių reikšmių sumą [Gee25]:

$$\textit{sentimentų rodiklis} = \frac{\sum_i s_i}{\sqrt{(\sum_i s_i)^2 + \alpha}}, \tag{8}$$

kur s_i žymi atskiro žodžio sentimentinę reikšmę, o α yra normalizavimo konstanta. Ši formulė užtikrina, kad galutinė reikšmė būtų apribota intervale nuo -1 iki 1.

Remiantis šia reikšme, tekstas klasifikuojamas į tris kategorijas [mys18]:

- teigiamas, kai *sentimentų rodiklis* $> 0{,}05$;
- neigiamas, kai *sentimentų rodiklis* $< -0{,}05$;
- neutralus, kai *sentimentų rodiklis* $\in [-0{,}05; 0{,}05]$.

2.8. BERT sentimentų analizės metodas

Sentimentų analizės metu VADER modelis bus lyginamas su BERT. BERT (angl. *Bidirectional Encoder Representations from Transformers*) yra transformerių architektūra paremtas giliojo mokymosi modelis, skirtas natūralios kalbos apdorojimo uždaviniams spręsti. Skirtingai nei tradiciniai metodai, BERT analizuoja tekstą dvikrypčiu būdu, atsižvelgdamas į visą kontekstą, todėl gali tiksliau interpretuoti žodžių ir frazių reikšmes. [DCL+19]

Sentimentų analizėje BERT modelio taikymas gali būti aprašomas kaip kelių etapų procesas. Pirmiausia tekstas suskaidomas į teksto vienetus (angl. *tokens*) ir paverčiamas skaitinėmis reprezentacijomis – įterpiniais (angl. *embeddings*). Šie įterpiniai perduodami į BERT modelį, kuriame per savidėmesio (angl. *self–attention*) mechanizmą apskaičiuojami ryšiai tarp visų sakinio žodžių. Dėmesio funkcija apibrėžiama taip:

$$\text{Dėmesys}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V, \tag{9}$$

kur Q, K ir V yra užklausos, rakto ir reikšmės vektoriai, o d_k žymi rakto vektoriaus dimensiją [VSP+17]. Softmax funkcija savidėmesio mechanizme normalizuoja panašumo įverčius tarp vieno žodžio ir visų kitų sakinio žodžių, paversdama juos tikimybiniais svoriais, kurių suma lygi 1.

Po šio etapo modelis suformuoja kontekstinę viso teksto reprezentaciją, kuri atspindi semantinius ryšius tarp žodžių. Ši reprezentacija naudojama klasifikavimo užduočiai, kurios metu tekstas priskiriamas vienai iš sentimentų kategorijų (pvz., teigiamas, neigiamas arba neutralus). [DCL+19]

3. Duomenys

Šiame tyrime naudojami Lietuvos Respublikos Seimo stenogramų duomenys, apimantys dvi kadencijas: 2016–2020 m. ir 2020–2024 m. Stenogramų duomenys buvo surinkti iš oficialios Lietuvos Respublikos Seimo interneto svetainės [Lie].

Duomenys svetainėje struktūrizuoti hierarchiškai: pirmiausia pagal kadenciją, tuomet pagal sesiją (nurodant sesijos numerį ir tipą, t. y. eilinę ar neeilinę sesiją), o galiausiai – pagal konkretų posėdį, identifikuojamą pagal datą ir posėdžio laiką (rytinis arba vakarinis).

Iš viso buvo surinkti 883 posėdžių stenogramų failai, iš kurių 455 priklauso 2016–2020 m. kadencijai, o 428 – 2020–2024 m. kadencijai.

Tolimesnei analizei duomenys buvo saugomi „Parquet“ formatu, kuris leidžia efektyviai dirbti su didelės apimties tekstiniais duomenimis ir užtikrina spartesnį duomenų apdorojimą.

3.1. Pradinė duomenų analizė

Šiame poskyryje, prieš pradedant kalbos modelių taikymą Seimo stenogramų analizei, darbe atlikta pradinė duomenų analizė siekiant nustatyti turimų tekstų apimtis, geriau suprasti jų turinį ir ypatybes.

3.1.1. Aprašomoji statistika

Pirmiausiai buvo atlikta stenogramų aprašomosios statistikos analizė pagal kadencijas, siekiant palyginti jų turinio apimtis tarpusavyje:

1 lentelė. *Stenogramų aprašomoji statistika pagal kadencijas*

Kadencija	Stenogramų sk.	Žodžių sk.	Vid. žodžių stenogramoje	Med. žodžių stenogramoje
2016–2020	455	8 686 491	19 091,19	18 750
2020–2024	428	8 193 157	19 142,89	18 976
Bendrai	883	16 879 648	19 116,25	18 907

2 lentelė. *Sakinių aprašomoji statistika pagal kadencijas*

Kadencija	Sakinių sk.	Vid. sakinio ilgis	Med. sakinio ilgis
2016–2020	1 095 504	7,93	4,00
2020–2024	979 107	8,37	5,00
Bendrai	2 074 611	8,14	4,00

Iš lentelės (žr. 1 lentelėje) matyti, kad 2016–2020 m. kadencijoje stenogramų šiek tiek daugiau (455) nei 2020–2024 m. kadencijoje (428), taip pat ir bendras žodžių skaičius yra didesnis. Tai rodo, kad pirmoji kadencija buvo kiek aktyvesnė pagal bendrą kalbėjimo apimtį. Vis dėlto tiek vidutinis,

tiek medianinis žodžių skaičius vienoje stenogramoje abiejose kadencijose yra labai panašūs (apie 19 tūkst.), todėl galima teigti, kad tipinis stenogramos ilgis iš esmės nesiskiria.

Iš lentelės (žr. 2 lentelėje) matyti, kad nors 2016–2020 m. kadencijoje bendras sakinių skaičius yra didesnis, vidutinis sakinio ilgis yra trumpesnis (7,93 žodžio), palyginti su 2020–2024 m. kadencija (8,37 žodžio). Taip pat pastebima, kad sakinio ilgio mediana 2016–2020 m. kadencijoje yra mažesnė, kas rodo, kad dauguma sakinių yra trumpi, o vidurkį didina ilgesni sakiniai.

3.1.2. Dažniausiai pasikartojančių žodžių analizė

Toliau abiejų kadencijų stenogramoms buvo sugeneruoti žodžių debesies grafikai, siekiant nustatyti ir palyginti prieš duomenų valymą dažniausiai pasikartojusius žodžius:


Žodžių debesis: 2016-2020
kad
ir
tai
dėl
ar
už
Seimo
įstatymo
komitetas
taip
bet
Ačiū
nėra
kolegos
apie
jeigu
projekta
PIRMININKĖ
komiteto
būtų
tikrai
galime
kurie
dar
aš
su
Lietuvos
Gerbiama
mes
prieš
tikrųjų
Dėkoju
kaip
jūs
pakeitimo
klausimas
pritarti
LVŽSF
XIIIP
pat
ne
jau
buvo
PIRMININKAS
valstybės
mūsų
tik
sutarimu
ta
galima
nutarimo
darbotvarkė
sveikatos
Prašau
straipsnio
projekto
Europos
gali
tą
jie
po
ji
Todėl
arba
nors
kai
iki
Balsavo
pasakyti
straipsnisi
pirmininkė
Vyriausybė
bendru
nariai
bu
pritarta
Teisė
dėlto
ką
darbo
o
labai
dabar
Gerbiami
LKDF
metų
reikia
nuo
šio
Balsai
biudžeto
tačiau
galbūt
manau
viena
šiuo
daugiau
tolliau
Respublikos
tiesiog
Motyvai
kiek
Balsuojame
Prašom
priimti
tam
vi
visi
pagal
turėtų
Kviečiu
tiek
tos
kur
narių
žmonių
priėmimas
Lietuvos
Seimo
Ačiū
Gerbiamieji
Komisijos
LSDPF
posėdžio
kartą
departamento
pasiūlymų
reikalų
kuri
kalbėti
kitų
man
pritarti
gerai
turime
metu
jūsų
PIRMININKAS
buvo
salėje
pritarė
tie
Klausia
kadangi
LVŽSF
straipsnių
Klausimą
klausimu
pasiūlymas
straipsnisi
svarstymo
priimta
kita
balsuoti
eurų
gal
tada
nuomonė
savo
tokia
kolega
pasakyti
Dėkoju
jūs
kaip
tik
ta
galima
nutarimo
darbotvarkė
sveikatos
Prašau
S
P
klausimą
Kviečiu
tiek
tos
kur
narių
žmonių
priėmimas
tikrųjų
Respublikos
tiesiog
Motyvai
kiek
Balsuojame
Prašom
priimti
tam
vi
visi
pagal
turėtų
Europos
gerai
turime
metu
jūsų
projekto
straipsnio
pritarti
kalbėti
kitų
man
reikalų
kuri
pasiūlymų
Klausia
kadangi
LVŽSF
straipsnių
tačiau
galbūt
manau
viena
šiuo
daugiau
tolliau
Respublikos
tiesiog
Motyvai
kiek
Balsuojame
Prašom
priimti
tam
vi
visi
pagal
turėtų
Kviečiu
tiek
tos
kur
narių
žmonių
priėmimas
tikrųjų


Žodžių debesis: 2020-2024
kad
ir
tai
yra
ar
kaip
aš
buvo
Seimo
įstatymo
projektas
dėl
PIRMININKAS
Ačiū
Lietuvos
prieš
LKDF
taip
jeigu
nėra
gerbiama
labai
mes
ne
Nr
iš
bet
pakeitimo
pritarti
klausimas
sutarimu
reikia
kita
Dėkoju
Gerbiami
Balsuojame
kolegos
komiteto
pirmininke
XIVP
Europos
Todėl
galėtų
Dėkui
Teisė
salėje
viena
savo
pagal
galime
bendru
dabar
jūs
tam
turi
kalbėti
apsaugo
LVŽSF
kurie
kuri
iki
tų
kai
gali
tuo
kiek
gal
Klausia
Taigi
priėmimas
arba
valstybė
straipsnių
šiuo
šio
atveju
viso
eurų
jų
kartą
net
daug
klausimą
žmonės
gerai
po
tik
noriu
PIRMININKĖ
tačiau
daugiau
tada
kartu
biudžeto
Balsavo
jieM
žmonių
atrodo
straipsnio
projekto
darbotvarkė
toliau
būtent
nutarimo
Prašau
metu
turime
irgi
Respublikos
pritarta
tiesiog
kolega
balsuoti
dėlto
jūsų
būti
toks
turbūt
darbo
visų
Lietuvoje
Kviečiu
dar
jums
posėdžio
straipsnių
valstybė
pakeitimo
pritarti
ji
nebuvo
kodėl
priėmimas
tikrųjų
šiuo
šio
atveju
reikia
kita
už
sutarimu
tos
arba
ar
aš
klausimas
Balsai
motyvų
Dėkui
Teisė
nors
salėje
ten
viena
savo
kurio
per
kai
galbūt
norėčiau
šį
kur
nuo
nariai
pateikimo
prie
gali
tuo
kiek
gal
Klausia
Taigi
bus
manau
jūs
narių
ką
Vyriausybė
komiteto
tam
apsaugo
turi
LVŽSF
dabar
tie
galime
bendru
kalbėti
visi
J
tiek
val
šį
XIVP
kitų
metai
klausimususilaikė
sveikatos
Todėl
kuri
iki
tų
kurie
balsavimo
galėtų
Europos
O
Dėkui
Teisė
nors
salėje
ten
viena
savo
kurio
per
kai
galbūt
norėčiau
šį
kur
nuo
nariai
pateikimo
prie
gali
tuo
kiek
gal
Klausia
Taigi
pirmininke
PIRMININKAS
reikia
kita
už
sutarimu
tos
arba
ar
aš
klausimas
Balsai
motyvų
Dėkui
Teisė
nors
salėje
ten
viena
savo
kurio
per
kai
galbūt
norėčiau
šį
kur
nuo
nariai
pateikimo
prie
gali
tuo
kiek
gal
Klausia
Taigi
kolegos
žmonės
gerai
po
tik
noriu
PIRMININKĖ
tačiau
daugiau
tada
kartu
biudžeto
Balsavo
jieM
žmonių
atrodo
straipsnio
projekto
darbotvarkė
toliau
būtent
nutarimo
Prašau
metu
turime
irgi
Respublikos
pritarta
tiesiog
kolega
balsuoti
dėlto
jūsų
būti
toks
turbūt
darbo
visų
Lietuvoje
Kviečiu
dar
jums
posėdžio
straipsnių
valstybė
pakeitimo
pritarti
ji
nebuvo
kodėl
priėmimas
tikrųjų
šiuo
šio
atveju
reikia
kita
už
sutarimu
tos
arba
ar
aš
klausimas
Balsai
motyvų
Dėkui
Teisė
nors
salėje
ten
viena
savo
kurio
per
kai
galbūt
norėčiau
šį
kur
nuo
nariai
pateikimo
prie
gali
tuo
kiek
gal
Klausia
Taigi
kolegos
žmonės
gerai
po
tik
noriu
PIRMININKĖ
tačiau
daugiau
tada
kartu
biudžeto
Balsavo
jieM
žmonių
atrodo
straipsnio
projekto
darbotvarkė
toliau
būtent
nutarimo
Prašau
metu
turime
irgi
Respublikos
pritarta
tiesiog
kolega
balsuoti
dėlto
jūsų
būti
toks
turbūt
darbo
visų
Lietuvoje
Kviečiu
dar
jums
posėdžio
straipsnių
valstybė
pakeitimo
pritarti
ji
nebuvo
kodėl
priėmimas
tikrųjų
šiuo
šio
atveju
reikia
kita
už
sutarimu
tos
arba
ar
aš
klausimas
Balsai
motyvų
Dėkui
Teisė
nors
salėje
ten
viena
savo
kurio
per
kai
galbūt
norėčiau
šį
kur
nuo
nariai
pateikimo
prie
gali
tuo
kiek
gal
Klausia
Taigi
pirmininke
PIRMININKAS
reikia
kita
už
sutarimu
tos
arba
ar
aš
klausimas
Balsai
motyvų
Dėkui
Teisė
nors
salėje
ten
viena
savo
kurio
per
kai
galbūt
norėčiau
šį
kur
nuo
nariai
pateikimo
prie
gali
tuo
kiek
gal
Klausia
Taigi
kolegos
žmonės
gerai
po
tik
noriu
PIRMININKĖ
tačiau
daugiau
tada
kartu
biudžeto
Balsavo
jieM
žmonių
atrodo
straipsnio
projekto
darbotvarkė
toliau
būtent
nutarimo
Prašau
metu
turime
irgi
Respublikos
pritarta
tiesiog
kolega
balsuoti
dėlto
jūsų
būti
toks
turbūt
darbo
visų
Lietuvoje
Kviečiu
dar
jums
posėdžio
straipsnių
valstybė
pakeitimo
pritarti
ji
nebuvo
kodėl
priėmimas
tikrųjų
šiuo
šio
atveju

Iš lentelės (žr. 3 lentelėje) matyti, kad abiejose kadencijose kalbėtojų skaičius yra beveik vienodas (457 ir 459), todėl bendras dalyvaujančių asmenų kiekis iš esmės nesikeitė.

Didžiausi skirtumai matomi tarp aktyviausių kalbėtojų. 2016–2020 m. kadencijoje lyderiai (M. Majauskas, A. Sysas) pasisakė daugiau kartų nei 2020–2024 m. lyderis J. Razma, todėl galima teigti, kad pirmoje kadencijoje aktyvumas labiau koncentruotas.

Tuo tarpu 2020–2024 m. aktyviausių kalbėtojų pasisakymai pasiskirsto tolygiau, atsiranda ir naujų pavardžių, nors kai kurie (J. Razma, E. Pupinis) tarp dažniausiai kalbančių išlieka abiejose kadencijose.

Toks dažnas kalbėtojų vardų pasirodymas tekstuose gali lemti jų įtraukimą tarp temų ir svarbiausių frazių, todėl prieš pradedant kalbos modelių taikymą turimus tekstus būtina išvalyti.

3.2. Duomenų apdorojimas

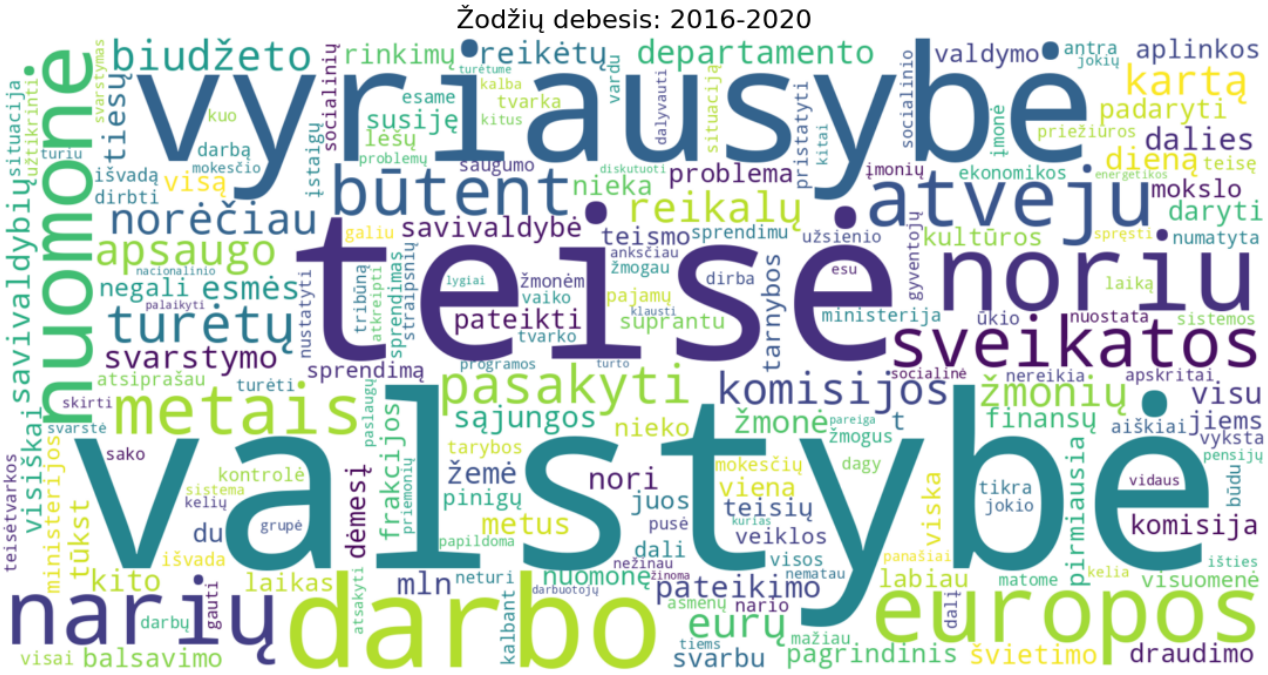
Atliekant Lietuvos Respublikos seimo stenogramų paruošimą kalbos modelių naudojimui, buvo atliekamas teksto valymo procesas, skirtas pašalinti nereikšmingą ir analizę iškraipančią informaciją.

Pirmiausia iš teksto buvo šalinami kalbėtojų žymėjimai, kuriuos modelis gali interpretuoti kaip reikšmingus požymius dėl jų dažnumo. Taip pat buvo šalinamas tekstas skliaustuose, kuris nurodo garsinius efektus (pvz., „šurmulys salėje"), partijų trumpinius (pvz., „LVŽSF"), papildomą informaciją. Pašalintos ir stenogramoms būdingos frazės ir žymėjimai (pvz., nuorodos į įstatymus, straipsnius), kurie nėra semantiškai reikšmingi teksto turinio analizei.

Tam tikri žodžiai buvo pašalinti nepaisant to, kad iš pirmo žvilgsnio jie gali atrodyti semantiškai svarbūs. Analizės metu pastebėta, kad tam tikri žodžiai dažnai nėra tiesiogiai susiję su nagrinėjama tematika. Pavyzdžiui, žodis „kaukė" dažniausiai pasirodė ne sveikatos kontekste, o frazėse, kuriomis raginimą laikytis saugumo reikalavimų COVID–19 laikotarpiu. Dėl dažno pasikartojimo tokiame kontekste, šie žodžiai galėjo paveikti modelio rezultatus, todėl buvo pašalinti.

Tekstas buvo konvertuojamas į mažąsias raides, šalinami skaičiai, skyrybos ženklai bei perteklinai tarpai. Taip pat buvo pašalinti ir nereikšmingi žodžiai (angl. *stopwords*).

Atlikus teksto valymą, kadencijoms buvo sugeneruoti žodžių debesies grafikai, siekiant patikrinti, kaip pasikeitė dažniausiai tekste pasirodantys žodžiai išvalius nereikalingą ir perteklinę informaciją



3 pav. *2016–2020 m. kadencijos žodžių debesis po teksto valymo*



4 pav. *2020–2024 m. kadencijos žodžių debesis po teksto valymo*

Iš grafikų (žr. 3 pav. ir 4 pav.) matyti, kad po teksto valymo žodžių debesys tapo informatyvesni. Nebedominuoja bendriniai, analizei nereikšmingi žodžiai, o ryškiau išsiskiria teminiai terminai. Tai rodo, kad teksto valymas buvo efektyvus ir leidžia geriau identifikuoti nagrinėjamas temas.

2016–2020 m. debesyje (žr. 3 pav.) aiškiai išryškėja su viešuoju valdymu ir politika susiję žodžiai, tokie kaip „valstybė“, „nariai“, „darbo“, „sveikatos“, „komisijos“, „biudžeto“, „europos“. Taip pat matyti ekonominiai ir administraciniai terminai („eurų“, „finansų“, „mokesčių“).

2020–2024 m. debesyje (žr. 4 pav.) išlieka panašios pagrindinės temos, tačiau labiau išsiskiria su sprendimų priėmimu ir veiksmų vykdymu susiję žodžiai, tokie kaip „darbo“, „turėtų“, „apsaugos“, „balsavimo“, „biudžeto“, „europos“. Taip pat ryškesni socialiniai aspektai („žmonių“, „sveikatos“), kas gali rodyti didesnį dėmesį visuomenės gerovei.

Kadangi vienas Seimo stenogramos dokumentas atitinka vieną posėdį, jame įprastai aptariami skirtingų temų klausimai. Dėl šios priežasties viso dokumento klasifikavimas pagal vieną temą būtų netikslus ir nepagrįstas. Siekiant padidinti klasifikavimo pagal temų tikslumą, stenogramų tekstai buvo suskaidyti į mažesnius vienetus pagal konkrečius posėdyje aptariamus klausimus arba skelbiamas informacines žinutes. Šis skaidymas atliktas remiantis tekste esančiomis laiko žymomis, kurios pateikiamos prieš kiekvieną naują svarstomą klausimą ar informacinį pranešimą. Po skaidymo duomenų rinkinį sudarė 22 277 teksto segmentai.

4. Teksto klasifikavimas

Šioje dalyje Lietuvos Respublikos Seimo stenogramos buvo klasifikuojamos į temines posėdžių klases, siekiant nustatyti pagrindines tekstų temas. Klasifikavimui buvo naudojamas iš anksto apmokytas kalbos modelis LitLatBERT. Tyrimo metu buvo atliekami du eksperimentai: klasifikavimas naudojant modelį be papildomo apmokymo ir klasifikavimas naudojant modelį, papildomai apmokytą su 10 % sužymėtų duomenų.

4.1. Neapmokyto modelio tekstų klasifikavimas

Pirmiausia visi iš anksto paruošti tekstai buvo paversti į skaitinę vektorinę formą, naudojant LitLatBERT modelį. Šio proceso metu tekstai buvo suskaidyti į mažesnius teksto vienetus (angl. *tokens*), kurie buvo transformuoti į vektorines reprezentacijas, atspindinčias jų semantinę reikšmę (angl. *embeddings*).

Kad visi tekstai būtų vienodo ilgio ir būtų galima juos apdoroti vienodomis sąlygomis, kiekvienas tekstas buvo papildytas užpildymo reikšmėmis (angl. *padding*) iki nustatyto maksimalaus ilgio. Tai būtina dėl to, kad transformerių architektūros modeliai, tokie kaip LitLatBERT, apdoroja įvestis fiksuoto dydžio daugiamačiais skaitiniais vektoriais, todėl skirtingo ilgio tekstai turi būti suvienodinti. Siekiant užtikrinti, kad šios užpildymo reikšmės nedarytų įtakos skaičiavimams, buvo naudojama papildoma maskavimo informacija (angl. *attention mask*), kuri nurodo, kurios teksto dalys yra realus turinys, o kurios užpildymas.

Gauti teksto vienetų vektoriai buvo apjungti į vieną bendrą teksto vektorių apskaičiuojant jų vidurkį. Tokiu būdu kiekvienas Lietuvos Respublikos Seimo stenogramos tekstas buvo paverstas į vieną semantinį vektorių, kuris reprezentuoja viso teksto prasmę.

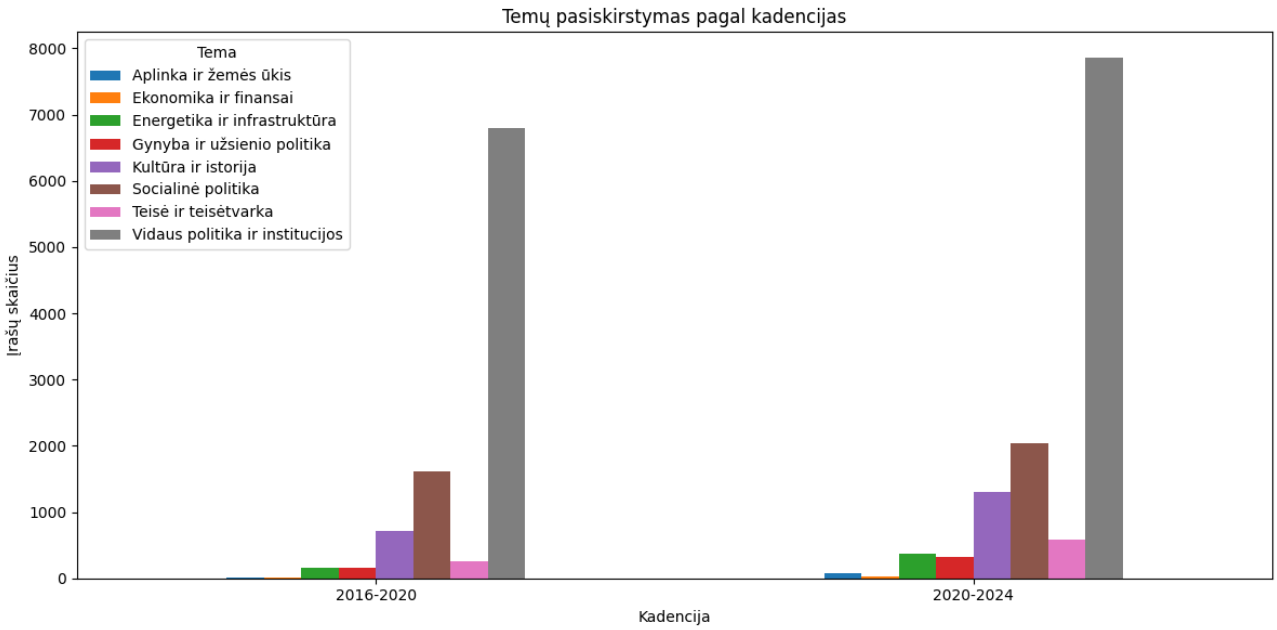
Klasifikavimui buvo taikomas semantinio panašumo metodas. Kiekviena Seimo posėdžių tema buvo apibrėžta naudojant žodžių grupes, kurios apibūdina konkrečią sritį (žr. 4 lentelėje). Klasių apibrėžimai taip pat buvo paversti į vektorines reprezentacijas naudojant LitLatBERT modelį.

Toliau buvo apskaičiuotas kiekvieno teksto semantinis panašumas su visomis temomis. Tam buvo naudojamas kosinusinio panašumo metodas, leidžiantis įvertinti vektorių tarpusavio atstumą. Kiekvienam tekstui buvo gautos panašumo reikšmės su visomis temomis.

Galutinė klasė kiekvienam tekstui buvo priskirta pagal didžiausią kosinusinio panašumo įvertį. Tokiu būdu kiekvienas Seimo stenogramos įrašas buvo automatiškai priskirtas vienai iš teminių kategorijų.

4 lentelė. *Teminių kategorijų grupių apibūdinimai*

Tema	**Temos apibūdinimas**
Gynyba ir užsienio politika	krašto apsauga, kariuomenė, NATO, nacionalinis saugumas, gynyba, karas, Ukraina, Rusija, sienų apsauga, užsienio politika, diplomatija, sankcijos, tarptautiniai santykiai, geopolitika, misijos
Ekonomika ir finansai	ekonomika, biudžetas, finansai, mokesčiai, PVM, pajamos, išlaidos, deficitas, infliacija, investicijos, ekonomikos augimas, fiskalinė politika, fondai
Socialinė politika	socialinė apsauga, sveikata, švietimas, ligoninės, gydytojai, mokyklos, pensijos, pašalpos, šeimos politika, vaiko teisės, gerovė, neįgalieji
Teisė ir teisėtvarka	teisė, teisėtvarka, teismai, policija, prokuratūra, konstitucinis teismas, baudžiamoji atsakomybė, administraciniai nusižengimai, įstatymai
Vidaus politika ir institucijos	vidaus politika, seimas, vyriausybė, institucijos, rinkimai, partijos, VRK, komisijos, įstatymų leidyba, valdymas, politinė sistema
Aplinka ir žemės ūkis	aplinkos apsauga, klimato kaita, ekologija, miškai, vanduo, atliekos, žemės ūkis, ūkininkai, kaimas, žuvininkystė, gamtos ištekliai
Energetika ir infrastruktūra	energetika, elektros energija, dujos, šildymas, kainos, transportas, infrastruktūra, viešieji pirkimai, atsinaujinanti energija
Kultūra ir istorija	kultūra, menas, paveldas, muziejai, istorija, istorinė atmintis, genocidas, rezistencija, švietimo kultūra, mokslas, universitetai



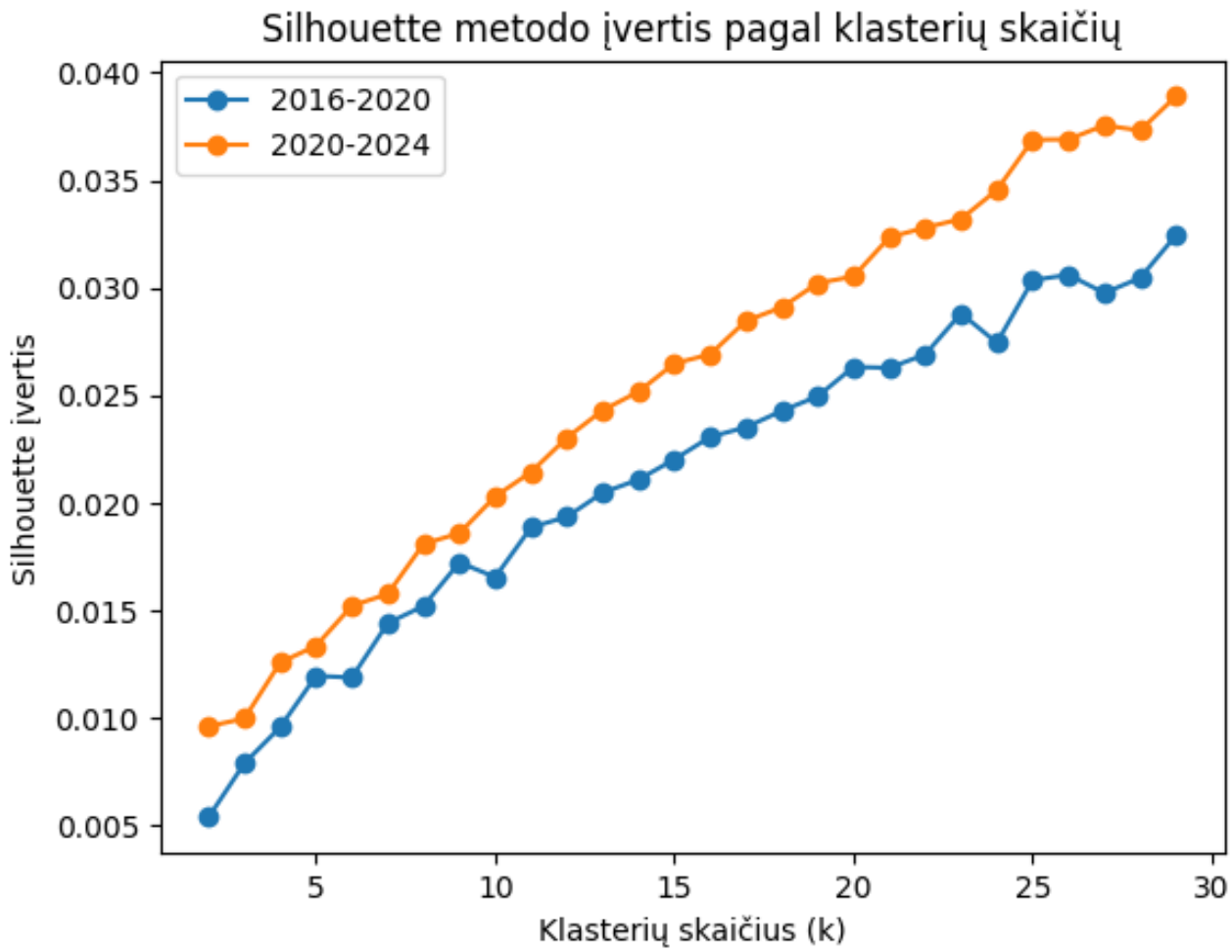
5 pav. *Posėdžių temų pasiskirstymas pagal kadencijas*

Rezultatai rodo (žr. 5 pav.), kad tekstai buvo paskirstyti per visas klases ir nė viena tema neliko be priskirtų įrašų, todėl taikytas semantinio panašumo metodas geba priskirti tekstus visoms apibrėžtoms temoms. Lyginant kadencijas, matyti, kad abiem laikotarpiais mažiausiai tekstų priskirta ekonomikos bei aplinkos ir žemės ūkio temoms, o daugiausiai – vidaus politikai ir institucijoms. Vis dėlto ši kategorija abiejose kadencijose aiškiai dominuoja ir gerokai lenkia kitas temas. Nors tai iš dalies gali būti paaiškinama parlamentinių diskusijų pobūdžiu, toks skirtumas kelia tam tikrų abejonių dėl priskyrimo tikslumo. Kadangi klasifikavimas atliktas naudojant kosinusinį panašumą, tikėtina, kad vidaus politikos ir institucijų klasė yra semantiškai artimiausia didžiajai daliai tekstų, todėl dalis kitoms temoms priskirtino turinio gali būti nukreipiama į šią kategoriją.

4.2. Apmokyto modelio tekstų klasifikavimas

Siekiant pritaikyti modelį klasifikavimui reikalingi sužymėti duomenys, taigi buvo atliktas duomenų žymėjimas. Pirmiausia tekstai, naudojant TF–IDF metodą, buvo paversti vektorinėmis reprezentacijomis į žodyną įtraukiant tik tuos terminus, kurie pasitaiko ne daugiau kaip 80 % dokumentų ir ne mažiau kaip penkiuose dokumentuose, siekiant sumažinti tiek labai retų, tiek pernelyg bendrinių žodžių įtaką. Analizei naudotos unigramos ir bigramos, siekiant išlaikyti tiek pavienius terminus, tiek reikšmines frazes, svarbias politinių tekstų kontekste.

Toliau, siekiant nustatyti tinkamą klasterių skaičių, buvo taikytas silueto (angl. *silhouette*) metodas. Klasterių skaičius buvo vertinamas intervale nuo 2 iki 29, siekiant kuo išsamiau atskleisti Seimo diskusijų temų įvairovę. Silueto koeficientas buvo apskaičiuojamas naudojant kosinusinį atstumą, o optimalus klasterių skaičius buvo parenkamas pagal didžiausią gautą silueto reikšmę. Tiek 2016–2020 m. kadencijai, tiek 2020–2024 m. kadencijai buvo nustatytas optimalus 29 klasterių skaičius (žr. 6 pav.).



6 pav. *Silueto metodo įvertis pagal klasterių skaičių*

Naudojant K–vidurkių (angl. *K–means*) algoritmą, kiekvienas tekstas buvo priskiriamas artimiausiam klasteriui ir kiekvienam klasteriui buvo išskirti jį geriausiai apibūdinantys terminai. Pagal šiuos terminus rankiniu būdu klasteriams buvo priskiriamos teminės žymos (žr. 13 ir 14 lentelėse). Pastebėta, kad dalis klasterių buvo sudaryti ne tik iš persidengiančių temų, bet ir iš žodžių, kurie nebuvo vienareikšmiškai susieti su viena temine žyma (žr. 5 lentelę). Dėl šios priežasties šie klasteriai nebuvo naudojami rankiniam anotavimui siekiant išvengti netikslaus žymėjimo išlaikant tikslią papildomo apmokymo (angl. *fine–tuning*) duomenų imtį.

5 lentelė. *Mišrių temų klasteriai su dažniausiai pasitaikančiais žodžiais*

Kadencija	Klasteris	Dažniausiai pasitaikantys žodžiai
2016–2020 m.	8	svarstymo, narių, teisės, niekas, europos, ratifikavimo, vyriausybės, stadijoje, reikalų, svarstymas, valstybės, pateikimo, svarstė, du, ekonomikos
2016–2020 m.	25	valstybės, žemės, vyriausybės, europos, metais, darbo, pasakyti, atveju, žmonių, būtent, teisės, žmonės, turėtų, švietimo, kartą
2020–2024 m.	22	svarstymo, lydimasis, apsaugos, narių, niekas, balsavimas, stadija, balsavimo, frakcijos, numatytu, nematau, kelių, aplinkos, teisių, darbo
2020–2024 m.	25	darbo, valstybės, europos, žmonių, metais, būtent, vyriausybės, žmonės, eurų, atveju, pasakyti, turėtų, mln, teisės, metus

Literatūros analizė parodė, kad transformerių modeliai gali pasiekti gerus rezultatus papildomai apmokant net ir su maždaug 10 % visų duomenų. Atsižvelgiant į tai, tekstai buvo rikiuojami pagal kosinusinį atstumą iki klasterio centro ir iš kiekvieno klasterio atrinkti arčiausiai centroido esantys tekstai. Kiekvienai teminei kategorijai buvo siekiama suformuoti apytiksliai 280 tekstų imtį, bendrai sudarančią apie 10 % visų tekstų. Kadangi vienai temai galėjo būti priskirti keli klasteriai, iš kiekvieno klasterio atrenkamų tekstų skaičius buvo apskaičiuojamas proporcingai pagal tai, kiek klasterių buvo priskirta konkrečiai temai. Kultūros ir istorijos temai buvo rasti tik du klasteriai, kurie bendrai sudarė 256 tekstus, todėl visiško klasių balanso išlaikyti nepavyko. Galutinę duomenų imtį sudarė 2216 sužymėtų įrašų, naudotų modelio mokymui ir validavimui.

Tuomet duomenų rinkinys, naudojant sluoksninę imtį, buvo padalintas į 90 % mokymo ir 10 % validavimo aibes.

LitLatBERT modelio papildomo mokymo metu buvo taikomas AdamW optimizavimo algoritmas su 2×10^{-5} mokymosi greičiu bei L2 svorių reguliavimu. Mokymosi procesas vykdytas 3 epochas. Siekiant užtikrinti stabilų mokymąsi, buvo taikomas dinaminis mokymosi greičio reguliavimas (angl. *learning rate scheduler*) su 10 % pradine apšilimo faze (angl. *warm–up*) bei gradientų normos ribojimas (angl. *gradient clipping*).

Modelio veikimas buvo vertinamas naudojant validavimo aibę. Kiekvienos epochos pabaigoje buvo apskaičiuojama nuostolio funkcijos reikšmė (angl. loss), tikslumas (angl. accuracy) bei svertinis F1 įvertis (angl. *weighted F1–score*).

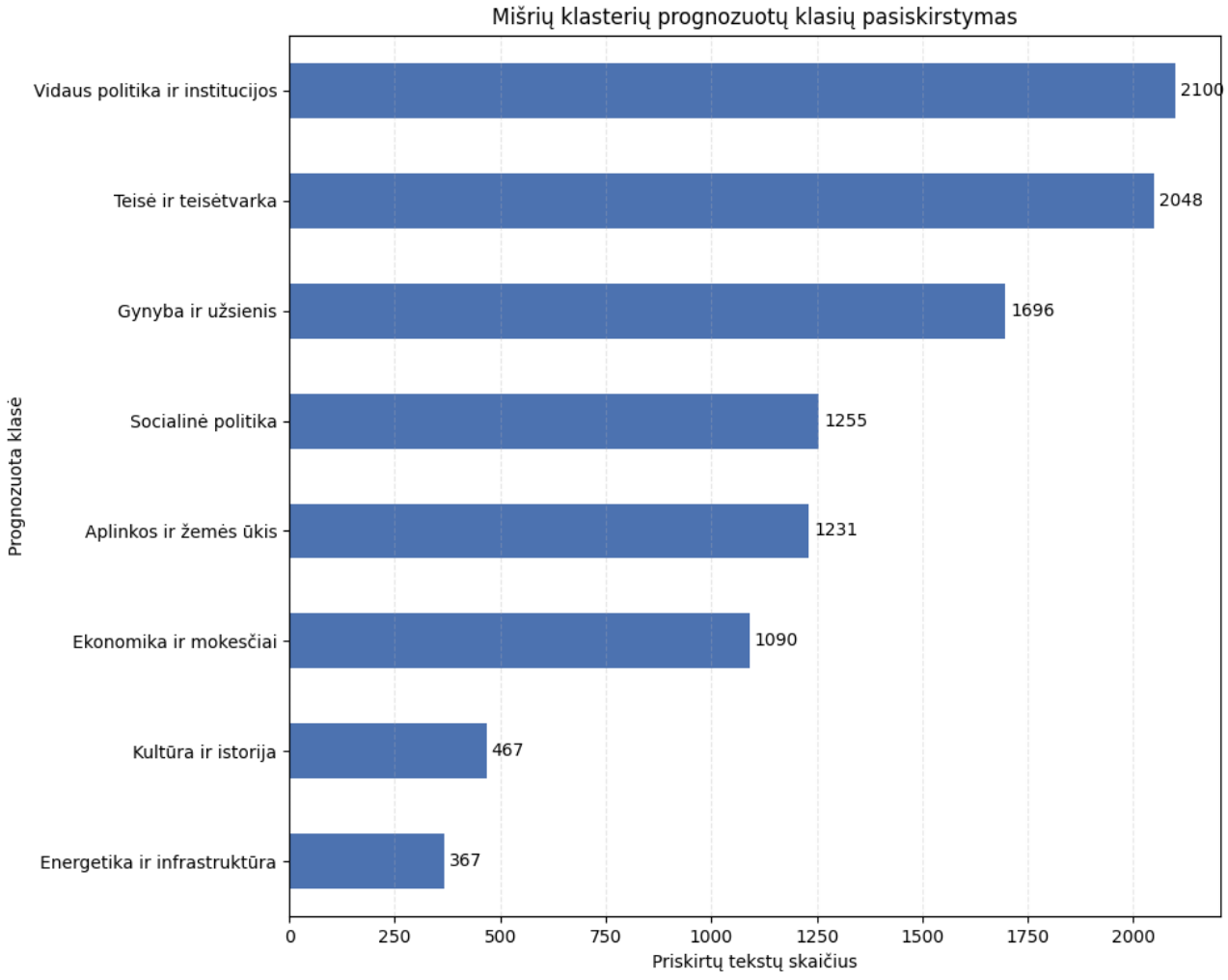
6 lentelė. *LitLatBERT modelio mokymo ir validacijos rezultatai*

Epocha	Mokymo paklaida	Validacijos paklaida	Tikslumas	F1 įvertis
1	1,7419	0,9338	0,9414	0,9382
2	0,5955	0,3023	0,9595	0,9593
3	0,2166	0,1724	0,9730	0,9729

Rezultatai (žr. 6 lentelėje) rodo nuoseklų modelio mokymosi progresą per visas tris epochas. Mokymosi paklaida reikšmingai mažėja nuo 1,7419 iki 0,2166, o validacijos paklaida taip pat nuosekliai krenta nuo 0,9338 iki 0,1724. Tai rodo, kad modelis ne tik gerai prisitaiko prie mokymo duomenų, bet ir pakankamai gerai veikia su naujais duomenimis.

Taip pat matyti, kad tikslumas ir svertinis F1 rodiklis didėja nuo 0,94 iki 0,97. Tai rodo nuosekliai gerėjančią klasifikavimo kokybę viso mokymo proceso metu. Be to, galima pastebėti, kad šių metrikų reikšmės išlieka artimos, kas leidžia manyti, jog klasių prognozavimas yra gana vienodas. Galutinės epochos rezultatai rodo, kad modelis pasiekia aukštą klasifikavimo kokybę, todėl galima tikėtis patikimų rezultatų tolimesnėse klasifikavimo užduotyse.

Iš klasterių, kurie nebuvo panaudoti mokymo duomenų imčiai sudaryti, buvo formuojama testavimo duomenų aibė. Ji naudota siekiant įvertinti, kaip apmokytas modelis klasifikuoja tekstus, kurių neprižiūrimas K–vidurkių klasterizavimas neatskyrė į aiškius klasterius. Toks papildomas vertinimas buvo svarbus dėl to, kad nepanaudoti klasteriai bendrai sudarė apie 10 000 tekstų.

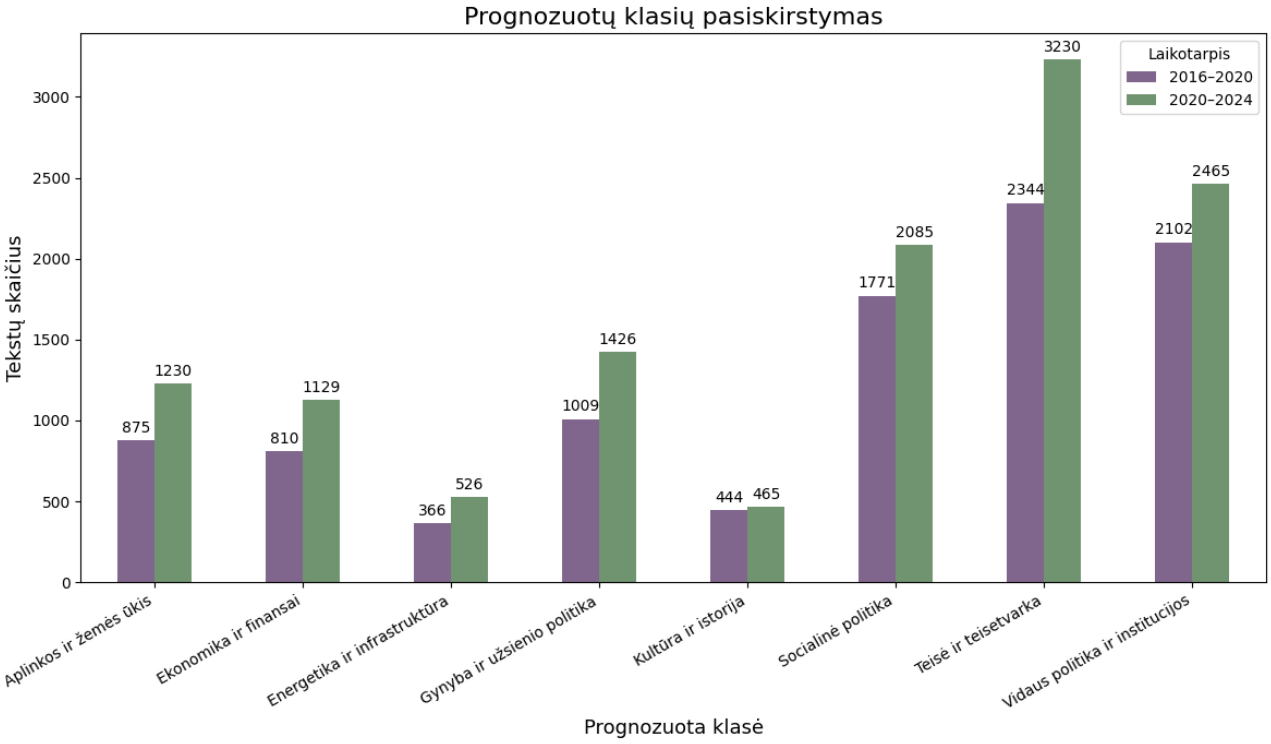


7 pav. *Mišrių klasterių prognozuotų klasių pasiskirstymas*

Grafike galima matyti (žr. 7 pav.), kad papildomai apmokytas modelis mišrių klasterių tekstų nepriskyrė vienai dominuojančiai klasei, o paskirstė juos tarp kelių teminių kategorijų. Toks pasiskirstymas rodo, kad šiuose klasteriuose buvo skirtingų temų tekstai, todėl dėl teminio persidengimo

jiems buvo sudėtinga priskirti vieną aiškią žymą rankinio anotavimo metu. Tai leidžia daryti prielaidą, kad šių klasterių įtraukimas į mokymo duomenų aibę nebuvo būtinas. Papildomai, atsitiktiniu būdu, peržiūrėjus dalį tekstų nustatyta, kad modelis daugeliu atvejų mišriuose klasteriuose esančius tekstus priskyrė logiškoms ir kontekstą atitinkančioms klasėms.

Galiausiai buvo atliekamas visų tekstų klasifikavimas į Seimo diskusijų temas tolimesnei Lietuvos Respublikos Seimo stenogramų analizei.



8 pav. *Prognozuotų LR Seimo posėdžių temų pasiskirstymas*

Iš stulpelinės diagramos (žr. 8 pav.) matyti, kad modelis gana gerai priskiria tekstus temoms ir nė viena klasė neliko be priskirtų įrašų ar ryškiai išsiskiriančios klasės. Tiek 2016–2020 m., tiek 2020–2024 m. kadencijose dominuoja teisės ir teisėtvarkos, vidaus politikos ir institucijų bei socialinės politikos temos, kurios sudaro didžiausią prognozuotų tekstų dalį. Mažiausiai tekstų abiem laikotarpiais priskirta kultūros ir istorijos bei energetikos ir infrastruktūros temoms, tai gali rodyti, kad šios temos Lietuvos Respublikos Seimo posėdžiuose buvo aptarinėjamos mažiausiai, tačiau negalima atmesti galimybės, kad modelis blogiausiai atpažįsta šių teminių kategorijų tekstus.

4.3. Metodų palyginimas

Tiek semantinio panašumo metodas, tiek papildomai apmokytas LitLatBERT modelis, sudaro galimybę priskirti tekstus visoms teminėms kategorijoms, tačiau jų rezultatai skiriasi.

Tekstų klasifikavimas naudojant semantinio panašumo metodą be papildomo LitLatBERT modelio apmokymo veikia pakankamai gerai, nes tekstai buvo priskirti visoms klasėms. Vis dėlto šis metodas stipriai priklauso nuo temų aprašymų kokybės, todėl rezultatuose išryškėja viena dominuojanti klasė – vidaus politika ir institucijos.

Tuo tarpu LitLatBERT modelis, papildomai apmokytas su anotuotais tekstais, parodo labiau subalansuotą tekstų priskyrimą. Taip pat galima pastebėti, kad šis modelis geriau identifikuoja ir atskiria tekstus, kurie klasterizavimo metu nebuvo aiškiai atskiriami.

Atsižvelgiant į gautus rezultatus ir vizualinį palyginimą, tolimesnei Lietuvos Respublikos Seimo stenogramų analizei buvo pasirinktas papildomai apmokyto LitLatBERT modelio klasifikuotas tekstas, kadangi jis geriau atspindi teminį pasiskirstymą ir mažiau priklauso nuo rankiniu būdu apibrėžtų temų aprašymų.

5. Svarbiausios frazės

Šioje dalyje Lietuvos Respublikos Seimo stenogramose buvo išskiriamos svarbiausios frazės, siekiant nustatyti pagrindines posėdžiuose aptariamas sąvokas, klausimus ir temas. Frazių išskyrimui buvo taikomi keli skirtingi metodai: TF–IDF, YAKE ir KeyBERT. Tyrimo metu buvo siekiama palyginti, kaip skirtingi metodai atskleidžia svarbiausius stenogramų tekstuose pasikartojančius terminus bei frazes.

5.1. TF–IDF metodo analizė

Svarbiausių frazių išskyrimui pirmiausia buvo taikomas TF–IDF metodas. Frazės buvo išskiriamos atskirai kiekvienos temos tekstams. Siekiant sumažinti procedūrinių frazių įtaką rezultatams, analizei naudotas iš anksto apdorotas ir išvalytas tekstas, kaip aprašyta 3.2. sk.. Be to, frazių kandidatams sudaryti buvo pasirinktas 2–3 žodžių n–gramų intervalas, leidžiantis atrinkti informatyvius, tačiau pernelyg ilgų ar neįprastų žodžių junginių neformuojančius terminus.

7 lentelė. *Temų svarbiausios TF–IDF frazės pagal kadencijas*

Tema	2016–2020 m. kadencija	2020–2024 m. kadencija
Aplinka ir žemės ūkis	aplinkos apsaugos žemės ūkio teisės departamento	aplinkos apsaugos žemės ūkio atliekų tvarkymo
Ekonomika ir finansai	biudžeto finansų pelno mokesčio gyventojų pajamų	viešųjų pirkimų vertės mokesčio pridėtinės vertės mokesčio
Energetika ir infrastruktūra	elektros energetikos geležinkelių transporto gamtinių dujų	elektros energetikos atsinaujinančių išteklių atsinaujinančių išteklių energetikos
Gynyba ir užsienio politika	nacionalinio saugumo europos sąjungos krašto apsaugos	europos sąjungos nacionalinio saugumo užsienio reikalų
Kultūra ir istorija	kultūros paveldo etninės kultūros istorinės atminties	visuomenės informavimo kultūros paveldo etninės kultūros
Socialinė politika	socialinio draudimo sveikatos priežiūros sveikatos draudimo	socialinio draudimo mokslo studijų sveikatos draudimo
Teisė ir teisėtvarka	teisės departamento administracinių nusižengimų teisės teisėtvarkos	administracinių nusižengimų teisės departamento administracinių nusižengimų administracinių
Vidaus politika ir institucijos	darbų programos vietos savivaldos darbų programą	vietos savivaldos netekusiu galios pripažinimo netekusiu galios

Išskirtos TF–IDF frazės (žr. 7 lentelėje) rodo, kad daugumos temų rezultatai atitinka jų turinį ir leidžia identifikuoti pagrindines posėdžiuose aptartas sritis. Lyginant kadencijas matyti, kad kai kuriose teminėse kategorijose išlieka panašus žodynas, pavyzdžiui, aplinkos ir žemės ūkio temoje abiejose kadencijose vyrauja su aplinkos apsaugos ir žemės ūkio terminai. Kita vertus, kai kuriose teminėse kategorijose išryškėja skirtumai tarp kadencijų; pavyzdžiui, 2016–2020 m. kadencijos energetikos ir infrastruktūros temoje tarp svarbiausių frazių patenka „geležinkelių transportas“ ir „gamtinės dujos“, o 2020–2024 m. stenogramose pagal TF–IDF įvertį svarbesnės su atsinaujinančiais ištekliais susijusios frazės.

Nors buvo atliktas griežtas teksto valymas, kai kuriose temose išskirtos frazės atspindi ne tik teminį turinį, bet ir Seimo darbo procedūrų įtaką rezultatams. Pavyzdžiui, aplinkos ir žemės ūkio temoje tarp svarbiausių frazių patenka „teisės departamento“, o vidaus politikos ir institucijų temoje – „netekusiu galios“. Be to, pastebima, kad TF–IDF metodas išskyrė nedaug trijų žodžių frazių ir jos visos dubliuojasi su trumpesnėmis dviejų žodžių frazėmis. Taip pat pastebėta, kad pateikta „administracinių nusižengimų administracinių“ frazė yra semantiškai netiksli ir neturėtų egzistuoti stenogramoje.

Tai rodo, kad agresyvus teksto valymas gali neigiamai paveikti TF–IDF svarbiausių frazių išrinkimo rezultatus.

5.2. YAKE metodo analizė

Svarbiausios frazės taip pat buvo išskiriamos taikant YAKE algoritmą. Prieš taikant šį metodą, 3.2. sk. teksto valymo funkcija buvo modifikuota taip, kad skyrybos ženklai nebūtų pašalinami. Siekiant sumažinti pavienių, tik atskiruose trumpuose tekstuose reikšmingomis tampančių frazių įtaką rezultatams, YAKE metodas buvo taikomas ne kiekvienam tekstui atskirai, o sujungtam vienos kadencijos ir vienos temos tekstų rinkiniui. Nustatyta maksimali trijų žodžių frazės ilgio riba. Be to, buvo taikomas YAKE bibliotekoje integruotas dedupLim parametras, skirtas panašių kandidatinių frazių dubliavimuisi sumažinti. Šiame darbe pasirinkta vidutinė dedupLim reikšmė. Galiausiai buvo pritaikytas papildomas filtras, pašalinantis frazes, kuriose kartojasi tas pats žodis.

8 lentelė. *Temų svarbiausios YAKE frazės pagal kadencijas*

Tema	**2016–2020 m. kadencija**	**2020–2024 m. kadencija**
Aplinka ir žemės ūkis	pagrindinis aplinkos apsaugos žemės ūkio paskirties žemės ūkio ministerija	žemės ūkio maisto pagrindinis aplinkos apsaugos žemės ūkio paskirties
Ekonomika ir finansai	gyventojų pajamų mokesčio valstybės biudžeto savivaldybių pagrindinis biudžeto finansų	gyventojų pajamų mokesčio pridėtinės vertės mokesčio mln tūkst eurų
Energetika ir infrastruktūra	atsinaujinančių išteklių energetikos energetikos kontrolės komisijos kainų energetikos kontrolės	atsinaujinančių išteklių energetikos energetikos atsinaujinančių išteklių išteklių energetikos atsinaujinančių
Gynyba ir užsienio politika	nacionalinio saugumo gynybos europos sąjungos valstybių valstybės saugumo departamento	nacionalinio saugumo gynybos europos sąjungos valstybių krašto apsaugos sistemos
Kultūra ir istorija	kultūros paveldo komisijos valstybinės kultūros paveldo kultūros paveldo apsaugos	kultūros paveldo komisijos valstybinės kultūros paveldo valstybinės kalbos komisijos
Socialinė politika	socialinių reikalų darbo sveikatos draudimo fondo privalomojo sveikatos draudimo	socialinių reikalų darbo sveikatos draudimo fondo sveikatos priežiūros įstaigų
Teisė ir teisėtvarka	teisės departamento nuomonė kanceliarijos teisės departamento teisės departamento dalies	kanceliarijos teisės departamento viena teisės departamento teisės departamento pristatyti
Vidaus politika ir institucijos	valstybės valdymo savivaldybių vyriausiosios rinkimų komisijos savivaldybių tarybų rinkimų	valstybės valdymo savivaldybių valstybės biudžeto savivaldybių vyriausiosios rinkimų komisijos

YAKE rezultatai (žr. 8 lentelėje) rodo, kad išskirtos frazės daugeliu atvejų atitinka teminių kategorijų turinį. Lyginant kadencijas matyti, kad dalyje temų išlieka panašus žodynas, tačiau kai kuriose

kategorijose pastebimi ir skirtumai tarp laikotarpių. Pavyzdžiui, energetikos ir infrastruktūros temoje 2020–2024 m. kadencijoje aiškiau išryškėja atsinaujinančių išteklių tematika, o 2016–2020 m. kadencijoje frazės labiau siejamos su energetikos kontrole ir kainų reguliavimu. Taip pat teminiai skirtumai matomi ir ekonomikos ir mokesčių temoje.

Visos YAKE metodu išskirtos svarbiausios frazės yra trijų žodžių ilgio. Be to, kai kurios tos pačios kadencijos ir temos frazės yra tarpusavyje panašios. Pavyzdžiui, 2016–2020 m. aplinkos ir žemės ūkio temoje išskirtos frazės „žemės ūkio paskirties“ ir „žemės ūkio ministerija“, o 2020–2024 m. energetikos ir infrastruktūros temoje – „atsinaujinančių išteklių energetikos“, „energetikos atsinaujinančių išteklių“ ir „išteklių energetikos atsinaujinančių“. Tai rodo, kad panašių kandidatinių frazių dubliavimąsi ribojantis dedupLim parametras ne visais atvejais buvo pakankamai efektyvus.

5.3. KeyBERT metodo analizė

Svarbiausios frazės taip pat buvo išskiriamos naudojant LitLatBERT modelį KeyBERT metodo struktūroje. Kadangi BERT tipo modelių taikymas yra skaičiavimo požiūriu imlus procesas, pirmiausia buvo atliktas testavimas su 50 vidaus politikos ir institucijų temai priskirtų 2016–2020 m. kadencijos tekstų. Prieš taikant šį metodą, 3.2. sk. teksto valymo funkcija buvo modifikuota taip, kad skyrybos ženklai nebūtų pašalinami.

Vėliau visi tekstai buvo paversti į skaitinę vektorinę formą, naudojant LitLatBERT modelį. Kadangi transformerių architektūros modeliai turi ribotą maksimalų įvesties ilgį, prieš taikant KeyBERT metodą tekstai buvo suskaidyti į trumpesnius fragmentus. Pirmiausia tekstas buvo segmentuojamas į sakinius, o vėliau sakiniai jungiami į didesnius fragmentus taip, kad kiekvieno fragmento ilgis neviršytų 480 teksto vienetų. Teksto vienetų skaičius buvo apskaičiuojamas naudojant taikomo sakinių įterpinių modelio tokenizatorių. Tokiu būdu buvo išsaugomas sakinių vientisumas ir sumažinta rizika, kad tekstas bus nutrauktas sakinio viduryje.

KeyBERT metodo taikymo metu buvo naudoti keli pagrindiniai parametrai. Kandidatinės frazės buvo sudaromos taikant 2–3 žodžių n–gramas, todėl į analizę buvo įtraukiami tik kelių žodžių terminų junginiai. Siekiant sumažinti tarpusavyje labai panašių frazių pasikartojimą, buvo taikomas maksimalaus marginalinio aktualumo metodas, naudojant vidutinę įvairovės reikšmę. Šis nustatymas leido sumažinti semantiškai panašių ar pasikartojančių frazių patekimą į galutinį rezultatų sąrašą. Galutinis frazės tinkamumas temai buvo vertinamas skaičiuojant semantinį panašumą tarp frazės vektoriaus ir temos centro vektoriaus. Papildomai galutiniame etape buvo taikomas lematizacija pagrįstas filtras, užtikrinantis, kad skirtingomis gramatinėmis formomis pateiktos, tačiau tą pačią žodžių struktūrą turinčios frazės į galutinį rezultatų sąrašą nebūtų įtraukiamos kelis kartus.

9 lentelė. *Pradinės vidaus politikos ir institucijų temos svarbiausios KeyBERT frazės*

Tema	**2016–2020 m. kadencija**
Vidaus politika ir institucijos	konstruktyviai bendradarbiauti valdančiąja proaktyviai tautiečių susigrąžinimą biurokratizuota gremėzdiška nelanksti politika netaptų biurokratizmo nebalsuoti ekspromtu neišgąsdinti koaliciją atsiprašau nesupratome teko konstruktyviai bendradarbiauti trupučiu nesumažinome padidinti nieko nenubyrėjo pralaimėjote atsiprašau nesupratome paskelbti

Pagal pradinius KeyBERT rezultatus matyti, kad metodas sugeneravo semantiškai su vidaus politikos ir institucijų tema susijusias frazes, tačiau jų kokybė nebuvo pakankama galutinei analizei. Dalis frazių buvo gramatiškai netaisyklingos arba labiau priminė sakinio fragmentus nei savarankiškus terminų junginius, pavyzdžiui, „konstruktyviai bendradarbiauti valdančiąja“ ir „nebalsuoti ekspromtu neišgąsdinti“.

Atsižvelgiant į šiuos rezultatus, taip pat buvo atliktas papildomas teksto valymas. Į valymo funkciją buvo įtraukti tiek bendri procedūrinės kalbos elementai, tiek pradiniuose KeyBERT rezultatuose pastebėti triukšmingi žodžiai. Pavyzdžiui, papildomai filtruoti tokie žodžiai kaip „atsiprašau“, „konstruktyviai“, „bendradarbiauti“ ir „gremėzdiška“. Papildomai buvo pašalinti tekstai, kurių ilgis po valymo buvo ne didesnis nei 25 žodžiai. Tokie trumpi tekstai dažniausiai atitinka procedūrines Seimo posėdžių žinutes, kurios nesuteikia reikšmingos informacijos apie tiriamas temas.

Be to, tarp rezultatų matyti nemažai veiksmažodinių terminų ar šnekamųjų stenogramos fragmentų, pavyzdžiui, „teko konstruktyviai bendradarbiauti“ ir „nieko nenubyrėjo pralaimėjote“. Dėl šios priežasties buvo nuspręsta papildomai taikyti kalbos dalių filtravimą, paliekant tik tas frazes, kuriose yra bent vienas daiktavardis ir nėra veiksmažodžių.

10 lentelė. *Temų svarbiausios KeyBERT frazės pagal kadencijas*

Tema	2016–2020 m. kadencija	2020–2024 m. kadencija
Aplinka ir žemės ūkis	nesumažės vyriausybę iniciatorius padiskutuosime problemų nesumažės nesveikas įsitikinimu nekonstitucinis	valdytojui kelių įregistravimą nepasitikėjimą visą sujudimą neveiksnumą lengviau biurokratijos
Ekonomika ir finansai	nepasitikėjimu piliečiais gabrieliaus lietuvoje nesiplanuoja periodui valiutomis mokėjimams europos	paskirstymas neartėjame problemos kadastrinių matavimų įregistravimo nuostolingai rūpintumėmės bankų
Energetika ir infrastruktūra	veto racionalus prieštaraučiau sujudimas nepaprastai didelis gyventojams užkrautų biurokratinę	sumažintų nepasitikėjimą pandemijos plėtros pasitikslinau nejaukiai paskirstymas bepavadintumei teisingiau
Gynyba ir užsienio politika	stimulas įsitikinimu europos pareiškimai bendradarbiavimo priešiškomis mažumoms georgijaus juostą	bekompromisės valstybingumo įtvirtinimą nepasitikėjimo iniciatorių atstovui reikalausime deputinizacijos visus
Kultūra ir istorija	iniciatyva apgailestauju kadenciją skausmo neproduktyvaus nereikalingo sostinės praradimu suredagavimas	adekvačios pavadinčiau savigarbos jėzuitų auklėtinį sarbievijų pažeidžiamumą valstybės nepasitikėjimą
Socialinė politika	konkurencijos išaiškintų nesąžiningus problemos giliu įsitikinimu bendradarbiausime tomis aiškiai	daugiau taiklumo įsitikinimu nueisime didesnį nepasitikėjimą kalbės nepasitikėjimą valstybės
Teisė ir teisėtvarka	sėjamas nepasitikėjimas dabartine spėriai dirbame susiklausymo rekomendacijomis vadovautumėmės skandinavijos	įvesime nepasitikėjimą vyriausybe artimuosiuose įsivyrautų taika romuvos diskriminaciniais mažesnėms
Vidaus politika ir institucijos	rezoliucija nepasitikėjimo lietuvos viešinimo nepasigendu drįsčiau įsitikinimu prezidentūros skambučiai	neplanuotai pratrūkus pandemijai savivaldoje neužtikrinimu savivaldybės parašus nepasitikėjimo pirmiausia

KeyBERT rezultatai rodo, kad išskirtos frazės dažnai nėra pakankamai aiškios ar savarankiškai interpretuojamos kaip svarbiausi terminų junginiai. Dalis jų primena sakinių fragmentus arba retorinės kalbos likučius, pavyzdžiui, „nesumažės vyriausybę iniciatorius“, „veto racionalus prieštaraučiau“ ar „spėriai dirbame susiklausymo“.

Taip pat matyti, kad kai kurios frazės tik ribotai atitinka konkrečių temų turinį. Pavyzdžiui, aplinkos ir žemės ūkio, ekonomikos ar socialinės politikos kategorijose išskirti junginiai ne visada leidžia aiškiai nustatyti, kokie konkretūs klausimai buvo svarbiausi. Net atlikus ypatingai griežtą teksto valymą, dalis rezultatų labiau atspindi bendrą politinės kalbos toną, pavyzdžiui, „nepasitikėjimo“, „įsitikinimu“ ar „problemų“ formuluotes.

5.4. Metodų palyginimas

Tiriant visus metodus, teksto valymas išliko sudėtingas uždavinys. Per silpnas valymas palieka daug procedūrinių Seimo stenogramoms būdingų frazių, tokių kaip teisėkūros proceso ar posėdžio

eigos elementai. Kita vertus, per griežtas valymas gali išardyti sakinių struktūrą ir dėl to sudaryti sąlygas nelogiškų ar gramatiškai netaisyklingų frazių atsiradimui.

Lyginant TF–IDF ir YAKE metodų rezultatus matyti, kad abu metodai daugeliu atvejų gebėjo išskirti temas atitinkančias frazes, tačiau jų rezultatų pobūdis skyrėsi. TF–IDF metodas dažniau išskyrė trumpesnius ir abstraktesnius terminų junginius, kurie aiškiai nurodo bendrą temos kryptį, todėl metodas yra tinkamas kaip bazinis atskaitos taškas kitų metodų vertinimui. Tuo tarpu YAKE metodas dažnai pateikė konkretesnes frazes, sudarytas iš trijų žodžių junginių, todėl kai kuriais atvejais jo rezultatai buvo informatyvesni nei TF–IDF rezultatai. Kita vertus, YAKE rezultatuose pastebėtas didesnis tarpusavyje panašių frazių pasikartojimas.

KeyBERT metodas, nepaisant teorinio pranašumo vertinti frazes pagal semantinį panašumą į temos tekstų centrą, šiame tyrime pateikė prasčiau interpretuojamus rezultatus. Išskirtos frazės dažnai priminė sakinių fragmentus arba retorinės kalbos likučius. Tam įtakos galėjo turėti tai, kad lietuvių kalbai trūksta pakankamai efektyvių kalbos dalių nustatymo įrankių ir aiškiai apibrėžtų kalbos dalių žymų struktūrų, kurias būtų galima patikimai taikyti kandidatinių frazių filtravimui. Dėl šios priežasties net ir papildomai taikant kalbos dalių filtrą į galutinius rezultatus pateko gramatiškai ar semantiškai nepakankamai tikslių junginių.

Tolesniuose tyrimo etapuose bus naudojami YAKE metodo rezultatai, nes šis metodas pateikė sąlyginai konkrečias ir teminėms kategorijoms informatyvias frazes, nepaisant tam tikro panašių junginių pasikartojimo.

6. Sentimentų analizė

Šiame skyriuje atliekamas svarbiausių frazių konteksto išgavimas ir vertimas į anglų kalbą. Išverstoms frazėms ir jų kontekstams atliekama sentimentų analizė taikant VADER ir BERT modelius.

6.1. Konteksto išgavimas

Pirmiausia iš kiekvienos temos buvo atrenkamos trys aukščiausią reitingą turinčios frazės, kurios vėliau naudojamos paieškai stenogramose.

Toliau atliekamas dalinis teksto valymas. Iš stenogramų pašalinami vardai, pavardės, partijų pavadinimai, skaičiai, pertekliniai skyrybos ženklai ir kiti nereikalingi elementai. Taip pat sutvarkomi trumpiniai bei tarpai tarp žodžių. Išvalytas tekstas suskaidomas į sakinius.

Po teksto apdorojimo atliekama svarbiausių frazių paieška. Frazės ieškomos ne tik atskiruose sakiniuose, bet ir tarp dviejų gretimų sakinių bei visame dokumente. Papildomai naudojama lanksti paieška originaliuose stenogramų tekstuose, leidžianti aptikti frazes net tada, kai tarp jų žodžių yra skaičių ar papildomų simbolių.

Radus frazę, formuojami du kontekstai:

- `VADER kontekstas` – vieno sakinio kontekstas, naudojamas VADER sentimentų analizei;
- `BERT kontekstas` – trijų sakinių kontekstas, sudarytas iš ankstesnio, esamo ir sekančio sakinio.

Platesnis kontekstas BERT modeliui naudojamas todėl, kad transformerių architektūros modeliai geriau interpretuoja sakinio reikšmę platesniame teksto kontekste.

6.2. Kontekstų vertimas į anglų kalbą

Po kontekstų ištraukimo buvo atliekamas jų vertimas iš lietuvių kalbos į anglų kalbą. Vertimui naudotas neuroninis mašininio vertimo modelis `Helsinki-NLP/opus-mt-tc-big-lt-en`.

Žemiau pateikiamas išgauto konteksto vertimo pavyzdys:

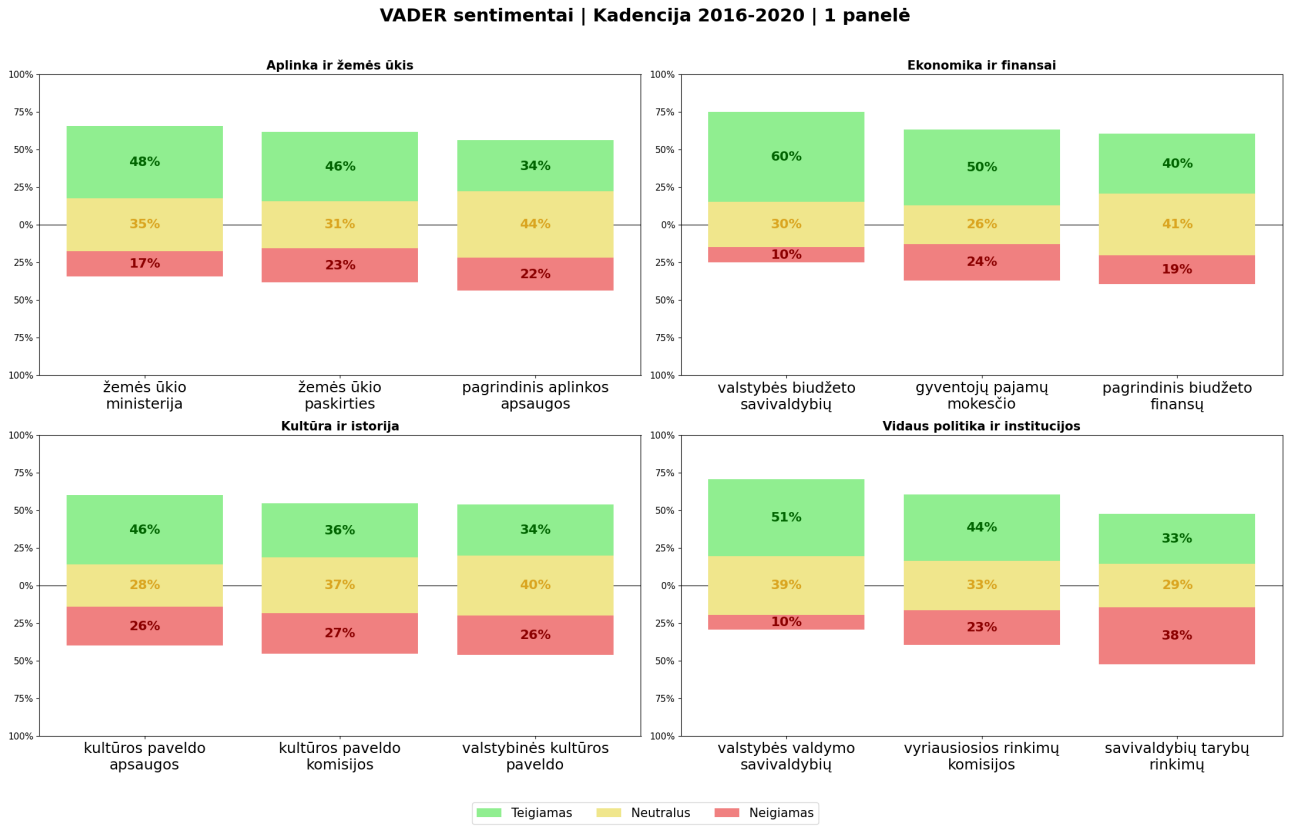
Originalus tekstas (LT): „Pažiūrėkite, kiek uždirba, pavyzdžiui, Vyriausiosios rinkimų komisijos nariai, kitų komisijų nariai, kuriems atlyginama už valandas, kurias jie dirbo, o ne už pareigas, kurias jie eina.“

Vertimas (EN): „Look at how much they earn, for example, members of the Central Electoral Commission, members of other commissions who are compensated for the hours they worked, not the positions they hold.“

6.3. VADER sentimentų analizė

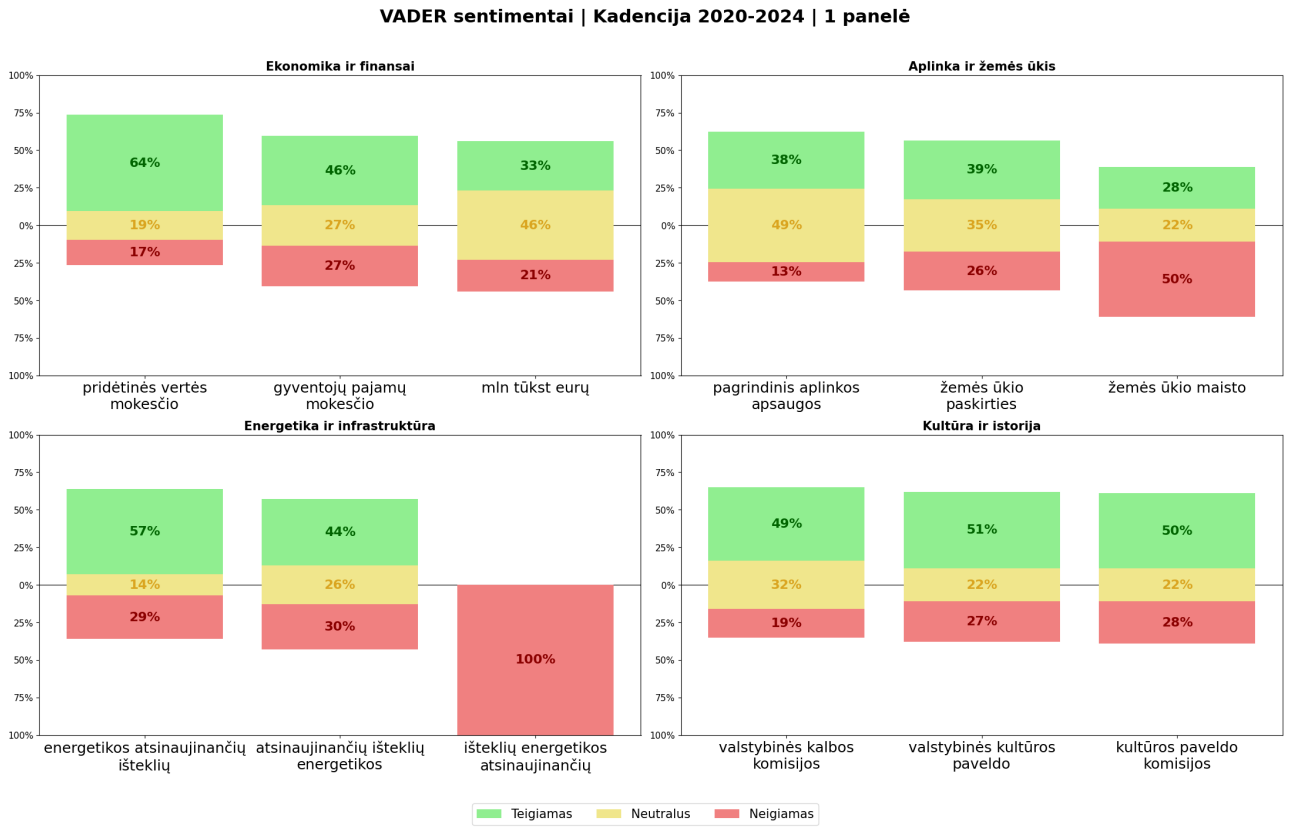
Po kontekstų vertimo buvo atliekama sentimentų analizė naudojant VADER metodą. Analizei naudojami vieno sakinio kontekstai. Kadangi ta pati frazė stenogramose gali pasikartoti kelis kartus

skirtinguose kontekstuose, sentimentų analizė atliekama kiekvienam pasirodymui atskirai. Kiekvienam kontekstui apskaičiuojamas sentimentų balas, pagal kurį tekstas priskiriamas teigiamai, neigiamai arba neutraliai kategorijai. Vėliau kiekvienai frazei apskaičiuojamas galutinis vidutinis sentimentų balas, pagal kurį nustatoma galutinė frazės sentimentų klasė.



9 pav. *Diskusijų temų svarbiausių frazių VADER sentimentų pasiskirstymas (2016–2020) (1)*

Grafike (žr. 9 pav.) matyti, kad aplinkos ir žemės ūkio temos frazė „pagrindinis aplinkos apsaugos“, ekonomikos ir finansų temos fraze „pagrindinis biudžeto finansų“, kultūros ir istorijos temos frazės „valstybinės kultūros paveldo“ ir „kultūros paveldo komisijos“ daugiausiai buvo minimos neutraliame kontekste. Pastebėta ir tai kad šioje kadencijoje tik vidaus politikos ir institucijų temos frazė „savivaldybių tarybų rinkimų“ daugiausiai buvo minėta neigiamu tonu. Likusios temų frazės (žr. 13 pav.) daugiausiai buvo minimos teigiamu tonu.



10 pav. *Diskusijų temų svarbiausių frazių VADER sentimentų pasiskirstymas (2020–2024) (1)*

Grafike (žr. 10 pav.) matyti, kad 2020–2024 metų kadencijoje vos dvi frazės daugiausiai buvo aptariamos neutraliu tonu: Ekonomikos ir finansų temos frazė „mln tūkst eurų“ bei kaip ir anksčiau aptartoje kadencijoje aplinkos ir žemės ūkio temos frazė „pagrindinis aplinkos apsaugos“. Neigiamai daugiausiai minėtos aplinkos ir žemės ūkio temos frazė „žemės ūkio maisto“ bei energetikos ir infrastruktūros frazė „išteklių energetikos atsinaujinančių“, kuri visuose tekstuose buvo paminėta vieną kartą. Likusios temų frazės (žr. 14 pav.) daugiausiai buvo minimos teigiamu tonu.

11 lentelė. *2016–2020 metų kadencijos svarbiausių frazių VADER sentimentai*

Tema	Frazė	VADER vidurkis	Sentimentas
Aplinka ir žemės ūkis	pagrindinis aplinkos apsaugos	0,051	teigiamas
	žemės ūkio ministerija	0,157	teigiamas
	žemės ūkio paskirties	0,109	teigiamas
Ekonomika ir mokesčiai	gyventojų pajamų mokesčio	0,115	teigiamas
	pagrindinis biudžeto finansų	0,106	teigiamas
	valstybės biudžeto savivaldybių	0,252	teigiamas
Energetika ir infrastruktūra	atsinaujinančių išteklių energetikos	0,060	teigiamas
	energetikos kontrolės komisijos	0,318	teigiamas
	kainų energetikos kontrolės	0,324	teigiamas
Gynyba ir užsienis	Europos Sąjungos valstybių	0,108	teigiamas
	nacionalinio saugumo gynybos	0,344	teigiamas
	valstybės saugumo departamento	0,312	teigiamas
Kultūra ir istorija	kultūros paveldo apsaugos	0,087	teigiamas
	kultūros paveldo komisijos	0,034	neutralus
	valstybinės kultūros paveldo	0,019	neutralus
Socialinė politika	privalomojo sveikatos draudimo	0,191	teigiamas
	socialinių reikalų darbo	0,153	teigiamas
	sveikatos draudimo fondo	0,220	teigiamas
Teisė ir teisėtvarka	kanceliarijos teisės departamento	0,286	teigiamas
	teisės departamento dalies	0,149	teigiamas
	teisės departamento nuomonė	0,142	teigiamas
Vidaus politika ir institucijos	savivaldybių tarybų rinkimų	-0,055	neigiamas
	valstybės valdymo savivaldybių	0,177	teigiamas
	vyriausiosios rinkimų komisijos	0,110	teigiamas

lentelėje (žr. 11 lentelėje) matyti, kad 2016–2020 metų kadencijoje aiškiai dominuoja teigiamas tonas visose pagrindinėse politikos srityse. Aplinkos, ekonomikos, energetikos, gynybos, socialinės politikos ir teisėtvarkos temose sentimentai yra stabiliai teigiami, o visų vienos frazės pasikartojimų vidurkiai svyruoja maždaug nuo 0,05 iki 0,34. Tai rodo nuoseklią, silpnai teigiama emocija pasižyminčią politinę kalbą. Kultūros ir istorijos temoje sentimentas daugiausiai neutralus. Vidaus politikos ir institucijų tematikoje matomas vienas silpno neigiamo sentimento atvejis, tačiau likusios dvi frazės išlieka teigiamos.

12 lentelė. *2020–2024 metų kadencijos svarbiausių frazių VADER sentimentai*

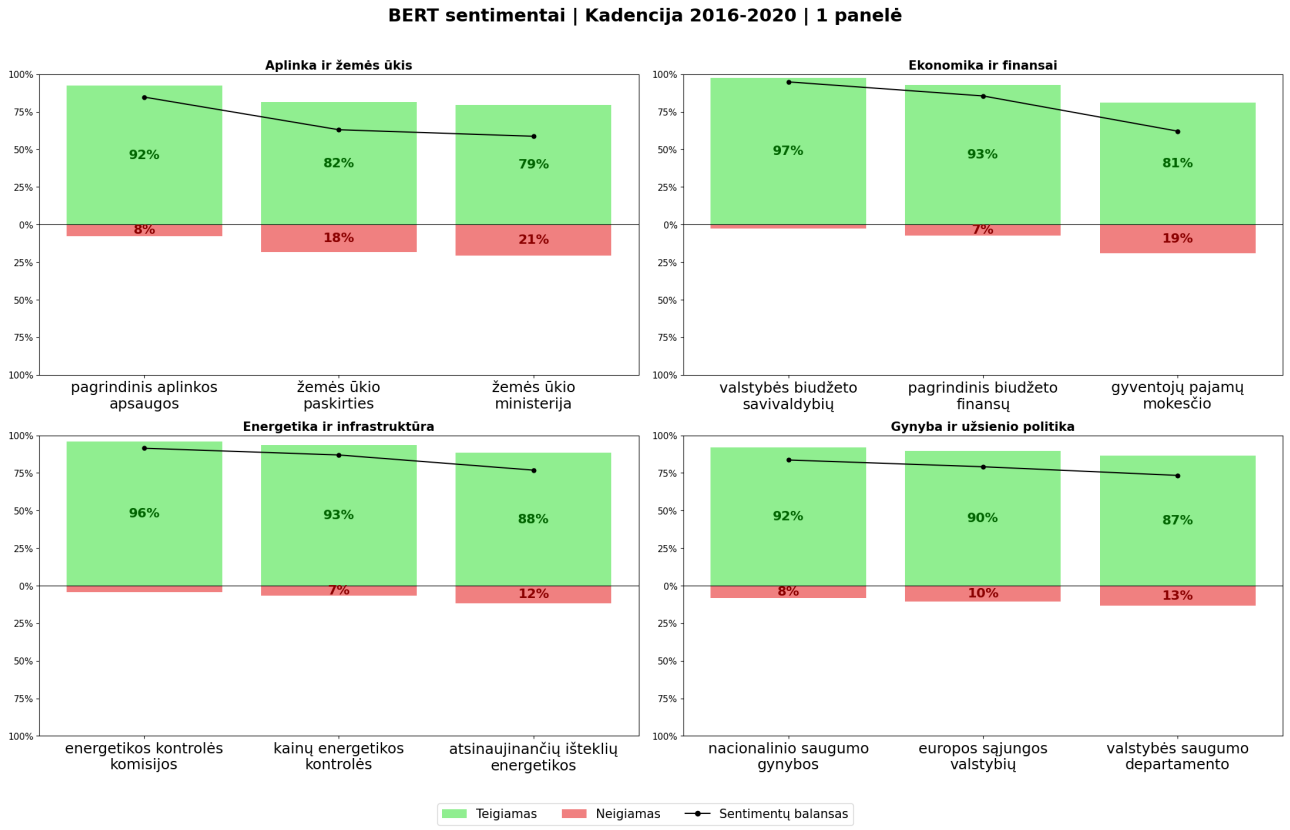
Tema	Frazė	VADER vidurkis	Sentimentas
Aplinka ir žemės ūkis	pagrindinis aplinkos apsaugos	0,119	teigiamas
	žemės ūkio maisto	-0,205	neigiamas
	žemės ūkio paskirties	0,064	teigiamas
Ekonomika ir mokesčiai	gyventojų pajamų mokesčio	0,082	teigiamas
	mln tūkst eurų	0,081	teigiamas
	pridėtinės vertės mokesčio	0,221	teigiamas
Energetika ir infrastruktūra	atsinaujinančių išteklių energetikos	0,067	teigiamas
	energetikos atsinaujinančių išteklių	0,173	teigiamas
	išteklių energetikos atsinaujinančių	-0,886	neigiamas
Gynyba ir užsienis	Europos Sąjungos valstybių	0,079	teigiamas
	krašto apsaugos sistemos	0,113	teigiamas
	nacionalinio saugumo gynybos	0,361	teigiamas
Kultūra ir istorija	kultūros paveldo komisijos	0,104	teigiamas
	valstybinės kalbos komisijos	0,165	teigiamas
	valstybinės kultūros paveldo	0,114	teigiamas
Socialinė politika	socialinių reikalų darbo	0,165	teigiamas
	sveikatos draudimo fondo	0,228	teigiamas
	sveikatos priežiūros įstaigų	0,182	teigiamas
Teisė ir teisėtvarka	kanceliarijos teisės departamento	0,333	teigiamas
	teisės departamento pristatyti	0,314	teigiamas
	viena teisės departamento	0,233	teigiamas
Vidaus politika ir institucijos	valstybės biudžeto savivaldybių	0,226	teigiamas
	valstybės valdymo savivaldybių	0,170	teigiamas
	vyriausiosios rinkimų komisijos	0,132	teigiamas

Lentelėje (žr. 12 lentelėje) matyti, kad 2020–2024 metų kadencijoje bendras sentimentų vaizdas taip pat išlieka daugiausia teigiamas, tačiau matomas didesnis svyravimas tarp temų. Ekonomikoje, gynyboje, socialinėje politikoje, teisėtvarkoje ir kultūroje sentimentai išlieka stabiliai teigiami, dažnai panašūs ar šiek tiek aukštesni nei ankstesnėje kadencijoje. Tačiau aplinkos ir žemės ūkio bei energetikos srityse atsiranda ryškesni neigiami sentimentai, įskaitant vieną stipriai neigiamą reikšmę energetikos temoje (-0,886). Kitaip nei ankstesnėje kadencijoje vidaus politikos srityje sentimentai stabiliai teigiami be neigiamų reikšmių.

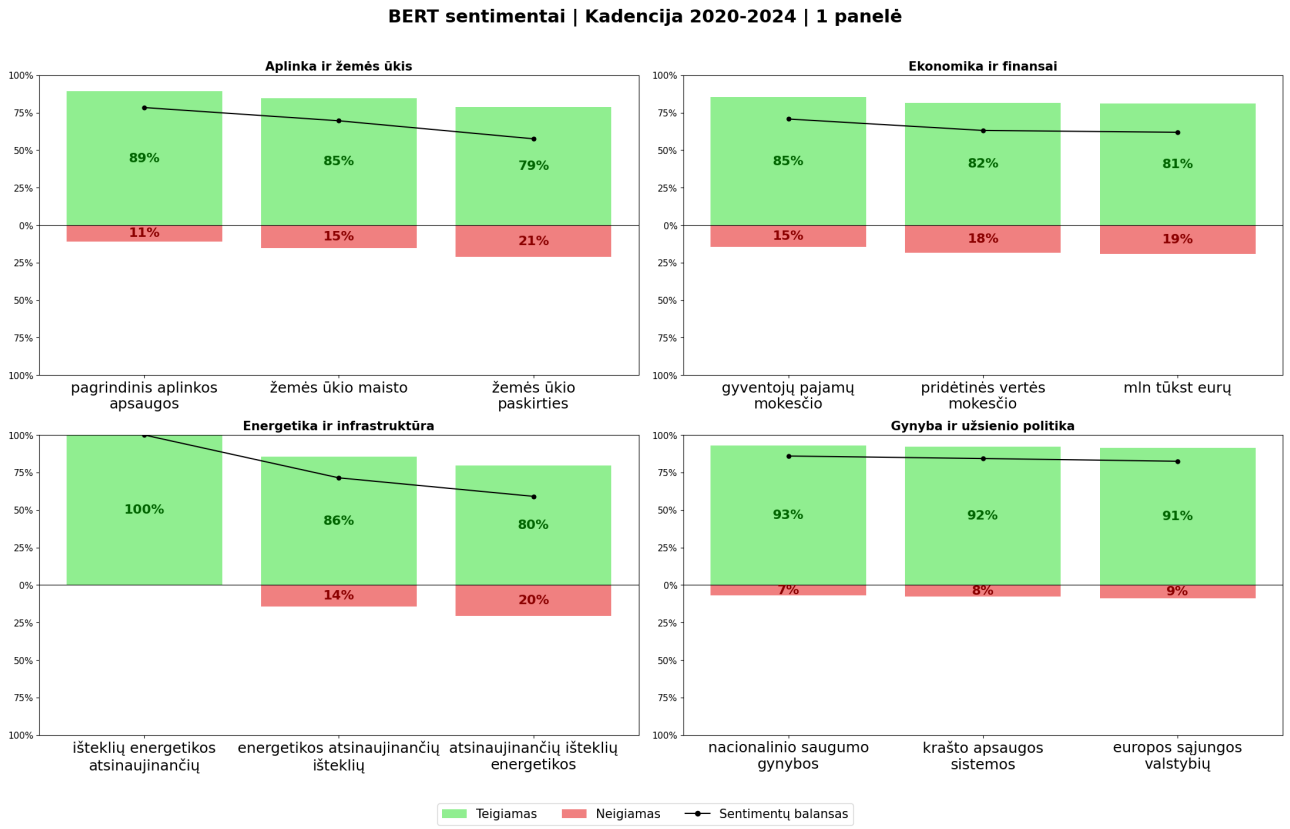
Abiejų analizuotų kadencijų grafikų panelės ir lentelės rodo, kad nors analizuoti tekstai pasižymi neutralia politine kalba, leksikonu pagrįstas VADER sentimentų analizės modelis politinei kalbai labiausiai linkęs priskirti teigiamą, kiek rečiau neutralų, o rečiausiai neigiamą sentimentus.

6.4. BERT sentimentų analizė

Toliau buvo atlikta sentimentų analizė naudojant BERT modelį. Analizei naudojami trijų sakinių kontekstai (Sakinys prieš, sakinys kuriame yra svarbiausia frazė ir sekantis sakinys). Taip pat, kaip ir naudojant VADER modelį, sentimentų analizė atliekama kiekvienam svarbiausios frazės pasirodymui atskirai. Kiekvienam kontekstui apskaičiuojama teigiamo ir neigiamo sentimentų tikimybė, pagal kurių skirtumą nustatomas bendras sentimentų balansas. Vėliau kiekvienai frazei priskiriamas galutinis sentimentas pagal daugumos balsų (angl. *majority vote*) principą.



11 pav. *Diskusijų temų svarbiausių frazių BERT sentimentų pasiskirstymas (2016–2020) (1)*



12 pav. *Diskusijų temų svarbiausių frazių BERT sentimentų pasiskirstymas (2020–2024) (1)*

Sentimentų pasiskirstymo grafikuose (žr. 11 ir 12 pav.) matyti, kad tiek 2016–2020 metų, tiek ir 2020–2024 metų kadencijoje, visų temų, visos frazės posėdžių diskusijose buvo minimos daugiausiai teigiamu emociniu tonu (nuo 79 % iki 100 % frazės pasikartojimų priskirtas teigiamas sentimentas). Juodi taškai apjungti linijomis parodo bendrą kiekvienos frazės sentimentų balansą, skaičiuojamą kaip teigiamų sentimentų dalies ir neigiamų sentimentų dalies skirtumą. Jis abiejų kadencijų grafikuose parodo žymią teigiamų sentimentų persvarą prieš neigiamus. Likę BERT sentimentų pasiskirstymo grafikai pateikiami priede (žr. 15 ir 16 pav.).

BERT modelio pateikiami temų svarbiausių frazių sentimentų analizės rezultatai pasirodė mažai informatyvūs, didžiajai daliai frazių pasikartojimų priskirti teigiami sentimentai. Dėl šios priežasties visoms 24 analizuotų frazių buvo priskirtas teigiamas galutinis sentimentas. Informatyvumo trūkumo problema kyla dėl to, kad tyrime naudotas modelis tekstams priskiria tik teigiamas arba neigiamas reikšmes.

6.5. Sentimentų analizės metodų palyginimas

Lyginant VADER ir BERT modelių rezultatus, matyti, kad VADER modelis suteikia daugiau informacijos apie sentimentų pasiskirstymą politiniuose tekstuose. Jo rezultatai yra aiškesni: matomi ne tik teigiami bei neigiami, bet ir neutralūs sentimentai, o pačios reikšmės atrodo mažiau klaidinančios. Tai leidžia geriau suprasti politinių tekstų pobūdį, kuris dažniausiai yra formalus, santūrus ir emociškai neutralus. Dėl šios priežasties VADER rezultatai atrodo lengviau interpretuojami ir artimesni realiam politinės kalbos pobūdžiui.

Tuo tarpu BERT modelio rezultatuose beveik visose temose dominuoja teigiamas sentimentas, todėl sentimentų pasiskirstymas tampa mažiau informatyvus. Dėl to neutralūs politiniai tekstai dažnai priskiriami teigiamam sentimentui, kas gali iškreipti bendrą rezultatų vaizdą. Nors BERT modelis geriau analizuoja kontekstą, šiame tyrime jo pateikiami rezultatai atrodo mažiau tikslūs politinių tekstų analizės požiūriu.

Dėl šių priežasčių, atliktai sentimentų analizei ir šiam tyrimui labiau tinkamu laikomas leksikonu pagrįstas sentimentų analizės modelis VADER.

Rezultatai ir išvados

Pritaikius kalbos modelius Lietuvos Respublikos Seimo stenogramų klasifikavimui į posėdžių temas pastebėta, kad tekstas tiksliau klasifikuojamas papildomai apmokius LitLatBERT modelį, nei jį naudojant semantiniam panašumui su apibrėžtomis temomis. Svarbiausių frazių išskyrimo dalyje nustatyta, kad TF–IDF metodas yra tinkamas paprastiems ir trumpiems teminiams terminams identifikuoti. YAKE metodas, priešingai, dažniau grąžino ilgesnes ir konkretesnes frazes, todėl kai kuriais atvejais geriau atskleidė temų turinį. Kita vertus, LitLatBERT modeliu pagrįstas KeyBERT metodas šiame tyrime pateikė prasčiau interpretuojamus rezultatus. BERT sentimentų analizės modelio rezultatuose daugumai frazių pasikartojimų buvo priskirtas teigiamas sentimentas, todėl pasiskirstymas tapo mažiau informatyvus ir ribojo galimybes aiškiai interpretuoti neutralią politinę kalbą. Tuo tarpu VADER analizė parodė, kad Seimo posėdžių diskusijose dominuoja silpnai teigiamas arba neutralus tonas, tačiau aiškiai buvo matomi neutralūs bei neigiami atvejai, todėl ji geriau atspindėjo formalią politinę kalbą ir suteikė informatyvesnį sentimentų pasiskirstymą.

Lietuvos Respublikos Seimo stenogramų analizė atskleidė, kad keičiantis kadencijoms posėdžių temų pasiskirstymas, vertinant pagal aptarimo dažnumą, nekito. Posėdžių metu daugiausia buvo diskutuojama apie teisę ir teisėtvarką, vidaus politiką ir institucijas bei socialinę politiką. Mažiausiai aptartos buvo energetikos ir infrastruktūros bei kultūros ir istorijos temos. Lyginant kadencijas, didžiausi skirtumai tarp esminių frazių rasti ekonomikos ir mokesčių, energetikos ir infrastruktūros temose. Ekonomikos ir mokesčių kategorijoje abiejose kadencijose išlieka gyventojų pajamų mokesčio tematika, tačiau 2020–2024 m. kadencijoje papildomai išryškėja pridėtinės vertės mokesčio ir piniginių sumų kontekstas. Energetikos ir infrastruktūros temoje abiem laikotarpiais svarbios atsinaujinančių išteklių frazės, tačiau 2016–2020 m. kadencijoje ryškesnis energetikos kontrolės ir kainų reguliavimo aspektas, o 2020–2024 m. rezultatuose labiau dominuoja atsinaujinančių išteklių energetikos junginiai. 2016—2020 metų kadencijoje Seimo posėdžių diskusijose dauguma temų buvo aptariamos teigiamu tonu, o neutralūs ar neigiami atvejai pasitaikė retai, pavyzdžiui, kultūros, istorijos ir vidaus politikos temose. Apskritai diskusijos buvo gana stabilios ir dažniausiai švelniai teigiamos. 2020—2024 metų kadencijoje taip pat išliko bendras teigiamas tonas, tačiau matėsi daugiau skirtumų tarp temų. Kai kuriose srityse, ypač energetikos ir žemės ūkio, atsirado daugiau neigiamų atvejų, o kitos frazės išliko neutralios arba švelniai teigiamos. Dėl to ši kadencija pasižymi įvairesniu ir mažiau vienodu diskusijų tonu.

Literatūra ir šaltiniai

[AB18] G. Abercrombie, R. T. Batista-Navarro. „'Aye' or 'No'? Speech-level Sentiment Analysis of Hansard UK Parliamentary Debate Transcripts“. Iš: *Proceedings of the Eleventh International Conference on Language Resources and Evaluation (LREC 2018)*. Miyazaki, Japan: European Language Resources Association, 2018, puslapiai 4164–4168.

[AB20] G. Abercrombie, R. Batista-Navarro. „Sentiment and position-taking analysis of parliamentary debates: a systematic literature review“. Iš: *J Comput Soc Sc* 3 (2020), puslapiai 245–270. https://doi.org/10.1007/s42001-019-00060-w. URL: https://doi.org/10.1007/s42001-019-00060-w.

[AI] F. AI. *XLM-RoBERTa Base*. URL: https://huggingface.co/FacebookAI/xlm-roberta-base (žiūrėta 2026-04-09).

[AMB+15] B. Agarwal, N. Mittal, P. Bansal, S. Garg. „Sentiment Analysis Using Common-Sense and Context Information“. Iš: *Computational Intelligence and Neuroscience* (2015), puslapis 715730. https://doi.org/10.1155/2015/715730. URL: https://doi.org/10.1155/2015/715730.

[BD18] S. Bhatia, D.P. *Topic-Specific Sentiment Analysis Can Help Identify Political Ideology*. žiūrėta 2026-04-08. 2018. URL: https://arxiv.org/abs/1810.12897.

[BMH+18] K. Bennani-Smires, C. Musat, A. Hossmann, M. Baeriswyl, M. Jaggi. „Simple Unsupervised Keyphrase Extraction using Sentence Embeddings“. Iš: *Proceedings of the 22nd Conference on Computational Natural Language Learning*. Brussels, Belgium: Association for Computational Linguistics, 2018, puslapiai 221–229. https://doi.org/10.18653/v1/K18-1022. URL: https://aclanthology.org/K18-1022/.

[BS25] B. Babayigit, H. Sattuf. „An Investigation of Keyword Extraction in Arabic and Turkish Employing BERT Models“. Iš: *Deep Learning with Emerging Engineering Applications*. Sudarė A. Akdagli. Online First, žiūrėta 2026-05-26. IntechOpen, 2025. https://doi.org/10.5772/intechopen.1012609. URL: https://www.intechopen.com/online-first/1233797.

[CMP+20] R. Campos, V. Mangaravite, A. Pasquali, A. Jorge, C. Nunes, A. Jatowt. „YAKE! Keyword extraction from single documents using multiple local features“. Iš: *Information Sciences* 509 (2020), puslapiai 257–289. URL: https://www.sciencedirect.com/science/article/pii/S0020025519308588?via%3Dihub#sec0006.

[DCL+19] J. Devlin, M. Chang, K. Lee, K. Toutanova. *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*. https://arxiv.org/abs/1810.04805. e-print arXiv:1810.04805, žiūrėta 2026-04-08. 2019.

[DVM+21] R. Devika, S. Vairavasundaram, S. J. Mahenthar, V. Varadarajan, K. Kotecha. „A Deep Learning Model Based on BERT and Sentence Transformer for Semantic Keyphrase Extraction on Big Social Data“. Iš: *IEEE Access* (2021). https://doi.org/10.1109/ACCESS.2021.3133651. URL: https://www.researchgate.net/publication/356857974_A_Deep_Learning_Model_Based_on_BERT_and_Sentence_Transformer_for_Semantic_Keyphrase_Extraction_on_Big_Social_Data.

[EMB] EMBEDDIA. *LitLat: Lithuanian Language Embeddings*. URL: https://huggingface.co/EMBEDDIA/litlat (žiūrėta 2026-04-09).

[Gee25] GeeksforGeeks. *Python | Sentiment Analysis using VADER*. 2025. URL: https://www.geeksforgeeks.org/python/python-sentiment-analysis-using-vader/ (žiūrėta 2026-04-09).

[Gro20] M. Grootendorst. *KeyBERT: Minimal keyword extraction with BERT*. https://github.com/MaartenGr/KeyBERT. Dokumentacija apie įrankį pateikta GitHub README.md faile. 2020.

[GS13] J. Grimmer, B. Stewart. „Text as Data: The Promise and Pitfalls of Automatic Content Analysis Methods for Political Texts“. Iš: *Political Analysis* 21.3 (2013), puslapiai 267–297. https://doi.org/10.1093/pan/mps028. URL: https://doi.org/10.1093/pan/mps028.

[HSF22] V. Hangya, H. S. Saadi, A. Fraser. „Improving low-resource languages in pre-trained multilingual language models“. Iš: *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*. 2022, puslapiai 11993–12006.

[You24] D. Youvan. *Understanding Sentiment Analysis with VADER: A Comprehensive Overview and Application*. https://doi.org/10.13140/RG.2.2.33567.98726. ResearchGate publication, žiūrėta 2026-04-08. 2024.

[JM23] D. Jurafsky, J. H. Martin. *Speech and Language Processing (3rd Edition, draft)*. PDF, https://web.stanford.edu/~jurafsky/slp3/. Stanford, CA, USA: Stanford University, 2023.

[KSM23] E. Kankevičiūtė, M. Songailaitė, J. Mandravickaitė. „Neapykantos kalbos atpažinimas lietuviškuose komentaruose panaudojant dirbtinį intelektą“. Iš: *Konferencijos "Lietuvos magistrantų informatikos ir IT tyrimai" darbai*. Vilnius, Lietuva: Vilniaus universiteto leidykla, 2023, puslapiai 27–34. https://doi.org/10.15388/LMITT.2023.3. URL: https://www.journals.vu.lt/open-series/en/article/view/32216.

[Kut25] J. Kutylowski. *DeepL*. 2025. URL: https://www.deepl.com/ (žiūrėta 2025-05-28).

[KZ25] C. Kertcher, M. Zwilling. „The meaning of sentiment analysis of UN speeches on the Russia-Ukraine war: a comparative study using VADER and BERT NLP techniques“. Iš: *Front. Polit. Sci.* 7 (2025), puslapis 1546822. https://doi.org/10.3389/fpos.2025.1546822. URL: https://doi.org/10.3389/fpos.2025.1546822.

[Lie] Lietuvos Respublikos Seimas. *Seimo posėdžių stenogramos*. žiūrėta 2026-04-10. URL: https://www.lrs.lt/pls/inter/w5_sale.kad_ses (žiūrėta 2026-04-10).

[mys18] mystery0116. *NLP — How does NLTK VADER calculate sentiment?* Accessed: 2026-04-09. 2018. URL: https://medium.com/@mystery0116/nlp-how-does-nltk-vader-calculate-sentiment-6c32d0f5046b.

[Muk18] Mukesh. *Silhouette Analysis in K-Means Clustering*. 2018. URL: https://medium.com/@cmukesh8688/silhouette-analysis-in-k-means-clustering-cefa9a7ad111 (žiūrėta 2026-05-20).

[NLP20] H. NLP. *opus-mt-tc-big-lt-en*. 2020. URL: https://huggingface.co/Helsinki-NLP/opus-mt-tc-big-lt-en (žiūrėta 2026-05-24).

[Ope25] OpenAI. *ChatGPT*. 2025. URL: https://www.openai.com/ (žiūrėta 2025-05-28).

[PT19] E. Papagiannopoulou, G. Tsoumakas. *A Review of Keyphrase Extraction*. https://arxiv.org/abs/1905.05044. e-print arXiv:1905.05044, žiūrėta 2026-04-10. 2019.

[RB25] J. Reichel, Ľ. Benko. „Preservation of sentiment in machine translation of low-resource languages: a case study on Slovak movie subtitles“. Iš: *Language Resources & Evaluation* 59 (2025), puslapiai 779–805. https://doi.org/10.1007/s10579-024-09736-w. URL: https://doi.org/10.1007/s10579-024-09736-w.

[RRU20] A. Rokas, S. Rackevičienė, A. Utka. „Automatic Extraction of Lithuanian Cybersecurity Terms Using Deep Learning Approaches“. Iš: *Human Language Technologies – The Baltic Perspective*. Sudarė A. Utka ir kiti. IOS Press, 2020, puslapiai 39–46. https://doi.org/10.3233/FAIA200600. URL: https://portalcris.vdu.lt/server/api/core/bitstreams/27758c8d-5e8d-4668-b25c-acbd37b26757/content.

[Sá] L. de Sá. *Text Clustering with K-Means*. URL: https://medium.com/@lucasdesa/text-clustering-with-k-means-a039d84a941b (žiūrėta 2026-04-09).

[SBM22] T. Schopf, D. Braun, F. Matthes. „Evaluating Unsupervised Text Classification: Zero-shot and Similarity-based Approaches“. Iš: *Proceedings of the 2022 6th International Conference on Natural Language Processing and Information Retrieval*. ACM, 2022, puslapiai 6–15. https://doi.org/10.1145/3582768.3582795. URL: http://dx.doi.org/10.1145/3582768.3582795.

[TLP+20] Z. Terechshenko, F. Linder, V. Padmakumar, F. Liu, J. Nagler, J. A. Tucker, R. Bonneau. *A Comparison of Methods in Political Science Text Classification: Transfer Learning Language Models for Politics*. https://ssrn.com/abstract=3724644. Available at SSRN, doi:10.2139/ssrn.3724644, žiūrėta 2026-04-08. 2020.

[VSP+17] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, I. Polosukhin. „Attention Is All You Need“. Iš: *Advances in Neural Information Processing Systems* 30 (2017). URL: https://arxiv.org/abs/1706.03762.

[Wan23] Y. Wang. „Topic Classification for Political Texts with Pretrained Language Models“. Iš: *Political Analysis* 31.4 (2023), puslapiai 662–668. https://doi.org/10.1017/pan.2023.3. URL: https://doi.org/10.1017/pan.2023.3.

A priedas: Dirbtinio intelekto naudojimas

Šiame darbe dirbtinio intelekto įrankis „OpenAI ChatGPT“ [Ope25] buvo naudojamas, programinio kodo klaidų taisymui, grafikų estetikos tvarkymui, lentelių formatavimui programavimo ir „LaTeX“ aplinkose, rašybos bei stilistinių klaidų taisymui, pagalbai metodų ir literatūros paieškoje. Papildomai tekstui bei terminams versti buvo naudojamas „DeepL“ [Kut25] įrankis.

B priedas: Klasterizuotų tekstų lentelės

13 lentelė. *2016–2020 m. kadencijos klasteriai, jų raktažodžiai ir žymos*

Klasteris	Raktažodžiai	Žyma
0	pertraukos, pusės, pertrauka, frakcijos, kito, pusės pertraukos, pertrauką, pertraukos kito, pusės pertrauka, vedimo, vedimo tvarkos, pertrauka kito, tvarkos, frakcijos pusės, pertraukai	vidaus politika ir institucijos
1	išmokų, paramos, socialinės paramos, išmokų vaikams, vaikams, piniginės socialinės, socialinės, piniginės, nepasiturintiems, nepasiturintiems gyventojams, paramos nepasiturintiems, gyventojams, darbo, socialinių, teisinės pagalbos	socialinė politika
2	mokslo, studijų, mokslo studijų, studijų mokslo, švietimo, švietimo mokslo, svarstymo, mokyklų, aukštųjų mokyklų, aukštųjų, studentų, universitetų, teisės, vyriausybės, aukštojo	socialinė politika
3	mokesčio, pajamų, biudžeto, mokesčių, finansų, gyventojų pajamų, pelno, pelno mokesčio, pvm, pajamų mokesčio, gyventojų, biudžeto finansų, pridėtinės vertės, pridėtinės, vertės mokesčio	ekonomika ir finansai
4	pensijų, valstybinių pensijų, valstybinių, šalpos, socialinio draudimo, socialinio, draudimo, kaupimo, pensijų kaupimo, pensijos, draudimo pensijų, darbo, šalpos pensijų, pareigūnų karių, karių valstybinių	socialinė politika
5	netekusiu, netekusiu galios, pripažinimo netekusiu, pripažinimo, galios, skirsnio pripažinimo, ketvirtojo skirsnio, ketvirtojo, skirsnio, teisės, tarnybos, apsaugos pripažinimo, farmacijos, niekas, žemės	vidaus politika ir institucijos
6	darbo, valstybės, socialinių, darbuotojų, apmokėjimo, darbo apmokėjimo, reikalų darbo, socialinių reikalų, reikalų, dagys, tarnybos, valstybės tarnybos, valstybės politikų, valstybės pareigūnų, pareigūnų	socialinė politika
7	administracinių nusižengimų, nusižengimų, administracinių, nusižengimų administracinių, teisės, teisės teisėtvarkos, teisėtvarkos, nusižengimų lydimasis, lydimasis, lydimasis administracinių, nusižengimų teisės, nusižengimų es, es, svarstymo, niekas	teisė ir teisėtvarka

Klasteris	Raktažodžiai	Žyma
8	svarstymo, narių, teisės, niekas, europos, ratifikavimo, vyriausybės, stadijoje, reikalų, svarstymas, valstybės, pateikimo, svarstė, du, ekonomikos	-
9	darbų, darbų programos, programos, darbų programą, programą, programos darbų, pateikimo svarstymas, svarstymas, pateikimo, papildyti darbų, papildyti, įtraukti, niekas, įtraukti darbų, klausti	vidaus politika ir institucijos
10	teisės departamento, departamento, teisės, visam, nuomonė, kanceliarijos teisės, departamento nuomonė, kanceliarijos, dalies, pobūdžio, departamento teisės, redakcinio pobūdžio, redakcinio, nuomonei, visam teisės	teisė ir teisėtvarka
11	teismo, apeliacinio teismo, apeliacinio, teisėjo, aukščiausiojo, aukščiausiojo teismo, teismo teisėjo, teisėjo pareigų, pareigų, teisėja, teismo teisėja, teisėjų, atleisti, bylų, teisėju	teisė ir teisėtvarka
12	rinkimų, rinkimų komisijos, vyriausiosios rinkimų, vyriausiosios, komisijos, tarybų rinkimų, savivaldybių tarybų, tarybų, savivaldybių, rinkimuose, partijos, rinkimų rinkimų, vrk, europos parlamentą, rinkimų europos	vidaus politika ir institucijos
13	kultūros, paveldo, kultūros paveldo, etninės kultūros, etninės, paveldo apsaugos, paveldo komisijos, nekilnojamojo kultūros, globos, valstybinės kultūros, kultūros globos, globos tarybos, kultūros centrų, nekilnojamojo, valstybinės	kultūra ir istorija
14	energetikos, elektros, elektros energetikos, dujų, gamtinių dujų, gamtinių, atsinaujinančių, ekonomikos, energetikos energetikos, energijos, išteklių, atsinaujinančių išteklių, energetikos komisijos, išteklių energetikos, komisijos	energetika ir infrastruktūra
15	teisių, teisių apsaugos, vaiko, pripažinimo netekusiais, netekusiais galios, netekusiais, vaiko teisių, vartotojų teisių, apsaugos, vartotojų, pripažinimo, galios, apsaugos pagrindų, teisės, sveikatai atlyginimo	teisė ir teisėtvarka
16	komisijos, komisija, peticijos, peticijų, protokolinio, atmesti, komisiją, etikos procedūrų, tyrimo, pateiktą, etikos, procedūrų, peticijų komisijos, peticiją, komisijų	teisė ir teisėtvarka

Klasteris	Raktažodžiai	Žyma
17	civilinio, proceso, baudžiamojo, baudžiamojo proceso, civilinio proceso, teisės, teisės teisėtvarkos, teisėtvarkos, civilinio civilinio, proceso civilinio, proceso baudžiamojo, baudžiamojo baudžiamojo, teisės departamento, es, departamento	teisė ir teisėtvarka
18	draudimo, socialinio draudimo, socialinio, valstybinio socialinio, valstybinio, motinystės, ligos, motinystės socialinio, ligos motinystės, nedarbo, darbo, nedarbo socialinio, draudimo valstybinio, socialinių, sodros	socialinė politika
19	sveikatos, sveikatos priežiūros, sveikatos draudimo, priežiūros, draudimo, priežiūros įstaigų, įstaigų, privalomojo sveikatos, sveikatos reikalų, privalomojo, sveikatos sistemos, visuomenės sveikatos, reikalų, sistemos, draudimo fondo	socialinė politika
20	eismo, saugaus eismo, eismo automobilių, saugaus, automobilių keliais, automobilių, keliais, transporto, keliais saugaus, teisės, vairuotojo, vairuoti, teisės teisėtvarkos, teisėtvarkos, administracinių	teisė ir teisėtvarka
21	tarnybinės etikos, tarnybinės, vyriausiosios tarnybinės, etikos komisijos, etikos, vyriausiosios, komisijos, vtek, skyrimo vyriausiosios, komisija, interesų, privačių interesų, komisijos nariu, nariu, valdymo savivaldybių	teisė ir teisėtvarka
22	aplinkos, aplinkos apsaugos, atliekų, apsaugos, atliekų tvarkymo, tvarkymo, pakuočių, deginimo, atliekas, teršimą, teisės, aplinkos teršimą, medžioklės, kontrolės, valstybinės kontrolės	aplinka ir žemės ūkis
23	tarnybos, saugumo, nacionalinio saugumo, gynybos, nacionalinio, karo, krašto, krašto apsaugos, saugumo gynybos, valstybės tarnybos, apsaugos, valstybės, apsaugos sistemos, karo tarnybos, sistemos organizavimo	gynyba ir užsienio politika
24	tabako, alkoholio, kontrolės, gaminių, alkoholio kontrolės, tabako gaminių, medžiagų, narkotinių psichotropinių, psichotropinių medžiagų, narkotinių, psichotropinių, susijusių gaminių, tabako tabako, gaminių kontrolės, gaminių jais	teisė ir teisėtvarka

Klasteris	Raktažodžiai	Žyma
25	valstybės, žemės, vyriausybės, europos, metais, darbo, pasakyti, atveju, žmonių, būtent, teisės, žmonės, turėtų, švietimo, kartą	-
26	išbraukti, seniūnų, frakcijos, sueigos, seniūnų sueigos, frakcijos išbraukti, sueigoje, seniūnų sueigoje, narių, įtraukti, patvirtinta, išbrauktas, sueigos seniūnų, lieka, seniūnų sueiga	vidaus politika ir institucijos
27	savivaldybių, valstybės, savivaldybės, valdymo, savivaldos, vietos savivaldos, valdymo savivaldybių, valstybės valdymo, vietos, audito, įmonių, turto, valstybės savivaldybių, teisės, tarybos	vidaus politika ir institucijos
28	informaciniai pranešimai, informaciniai, pranešimai, frakcijos, grupę, frakcija, tarpparlamentinių, tarpparlamentinių ryšių, grupės, žaliųjų sąjungos, valstiečių žaliųjų, sąjungos, žaliųjų, tvarka teisingumas, ryšių	vidaus politika ir institucijos

14 lentelė. *2020–2024 m. kadencijos klasteriai, jų raktažodžiai ir žymos*

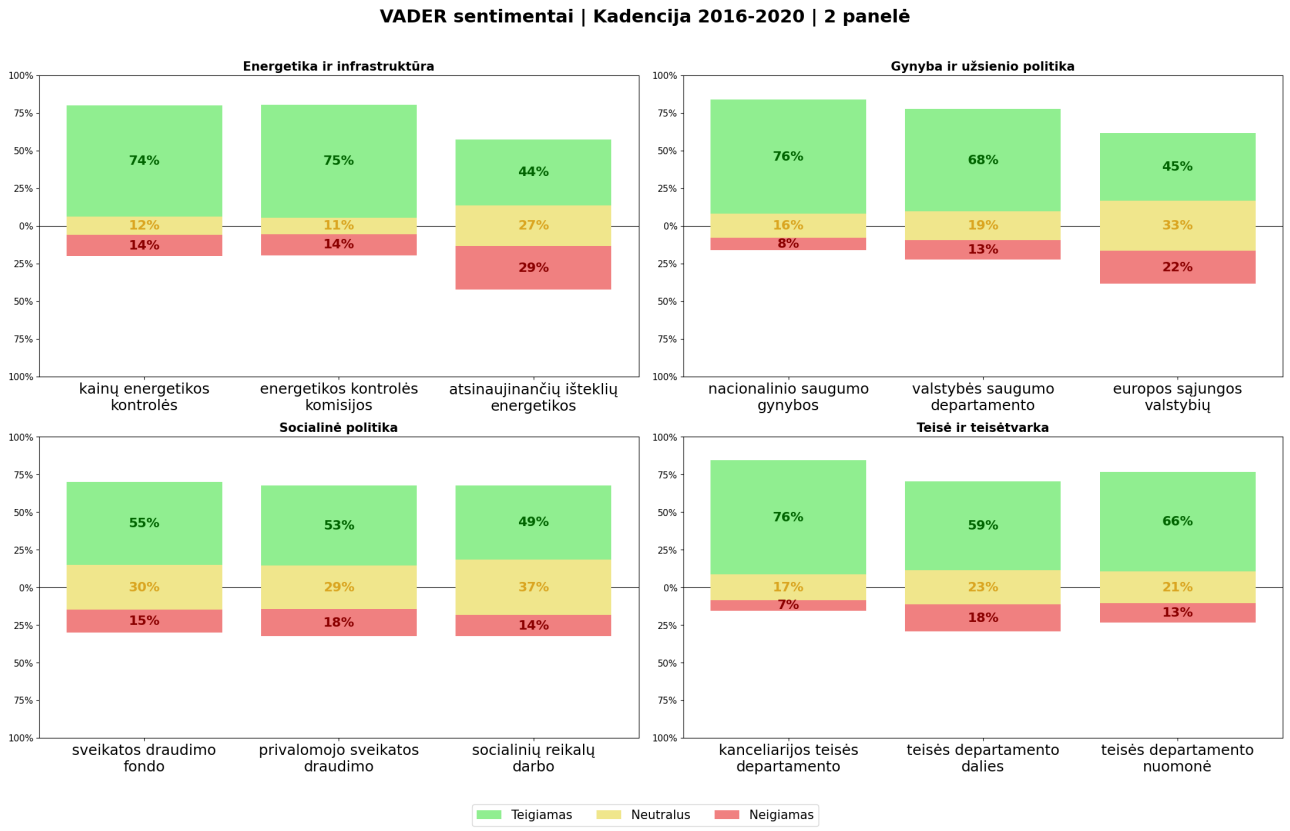
Klasteris	Raktažodžiai	Žyma
0	narių, sudėties, svarstymo, niekas, narių skaičiaus, komisijų, delegacijų, nebalsavo, nebalsavo niekas, skaičiaus, pateikimo, pateikimo svarstymo, nenori, niekas nenori, stadija	vidaus politika ir institucijos
1	sveikatos, sveikatos draudimo, sveikatos priežiūros, draudimo, priežiūros, sveikatos sistemos, priežiūros įstaigų, įstaigų, sistemos, sveikatos reikalų, privalomojo sveikatos, privalomojo, draudimo sveikatos, draudimo fondo, reikalų	socialinė politika
2	saugaus eismo, saugaus, automobilių keliais, eismo automobilių, eismo, keliais, automobilių, keliais saugaus, teisės teisėtvarkos, teisėtvarkos, teisės, transporto, svarstymo, kelių, pateikimo	teisė ir teisėtvarka
3	peticijų, peticijų komisijos, peticijos, komisijos, komisijos išvados, išvados, peticijos peticijų, peticijų komisija, peticiją, komisija, atmesti, peticijoje, komisijos išvadai, priėmė, komisijos nuostatų	teisė ir teisėtvarka

Klasteris	Raktažodžiai	Žyma
4	energetikos, elektros, elektros energetikos, atsinaujinančių, atsinaujinančių išteklių, išteklių energetikos, išteklių, energijos, elektros energijos, energetikos elektros, vėjo, energetikos energetikos, ekonomikos, saulės, dujų	energetika ir infrastruktūra
5	teisės departamento, departamento, teisės, departamento teisės, kanceliarijos teisės, kanceliarijos, suredaguotam, suredaguotam teisės, dalies, viena teisės, viena, visam, numatytu, niekas, pobūdžio	teisė ir teisėtvarka
6	netekusiu, netekusiu galios, pripažinimo netekusiu, pripažinimo, galios, pavadinimo, pavadinimo pripažinimo, skirsnio, svarstymo, transporto, skirsnio pavadinimo, administracinių nusižengimų, nusižengimų, administracinių, biudžeto sandaros	vidaus politika ir institucijos
7	civilinio, pateikimo, teisės teisėtvarkos, teisėtvarkos, pagrindinis, teisės, reikalų, vyriausybės išvados, išvados, vyriausybės, pateikimo pagrindinis, papildomas, teisių, biudžeto finansų, svarstymo	teisė ir teisėtvarka
8	mokesčio, vertės mokesčio, pridėtinės vertės, pridėtinės, vertės, pelno mokesčio, pelno, gyventojų pajamų, pajamų mokesčio, pajamų, gyventojų, pvm, mokesčio pridėtinės, biudžeto, finansų	ekonomika ir finansai
9	informaciniai pranešimai, informaciniai, pranešimai, frakcija, frakcijos, demokratų, partijos, grupės, partijos frakcija, sąjungos, balsų skaičiavimo, skaičiavimo, socialdemokratų partijos, seniūno, darbo partijos	vidaus politika ir institucijos
10	valstybės, valstybės tarnybos, tarnybos, valstybės savivaldybių, savivaldybių, darbo apmokėjimo, apmokėjimo, darbo, valdymo, tarnybos valstybės, įstaigų darbuotojų, savivaldybių turto, valdymo naudojimo, valstybės skolos, turto valdymo	vidaus politika ir institucijos
11	žemės, ūkio, žemės ūkio, kaimo, žemės naudojimo, naudojimo sąlygų, specialiųjų žemės, ūkio maisto, kaimo reikalų, naudojimo, specialiųjų, maisto, žemės žemės, sąlygų, maisto ūkio	aplinka ir žemės ūkis
12	mokslo studijų, studijų, mokslo, studijų mokslo, švietimo, švietimo mokslo, studijų pripažinimo, svarstymo, pripažinimo, galios, pripažinimo netekusiu, netekusiu, netekusiu galios, stadija, teisės departamento	socialinė politika

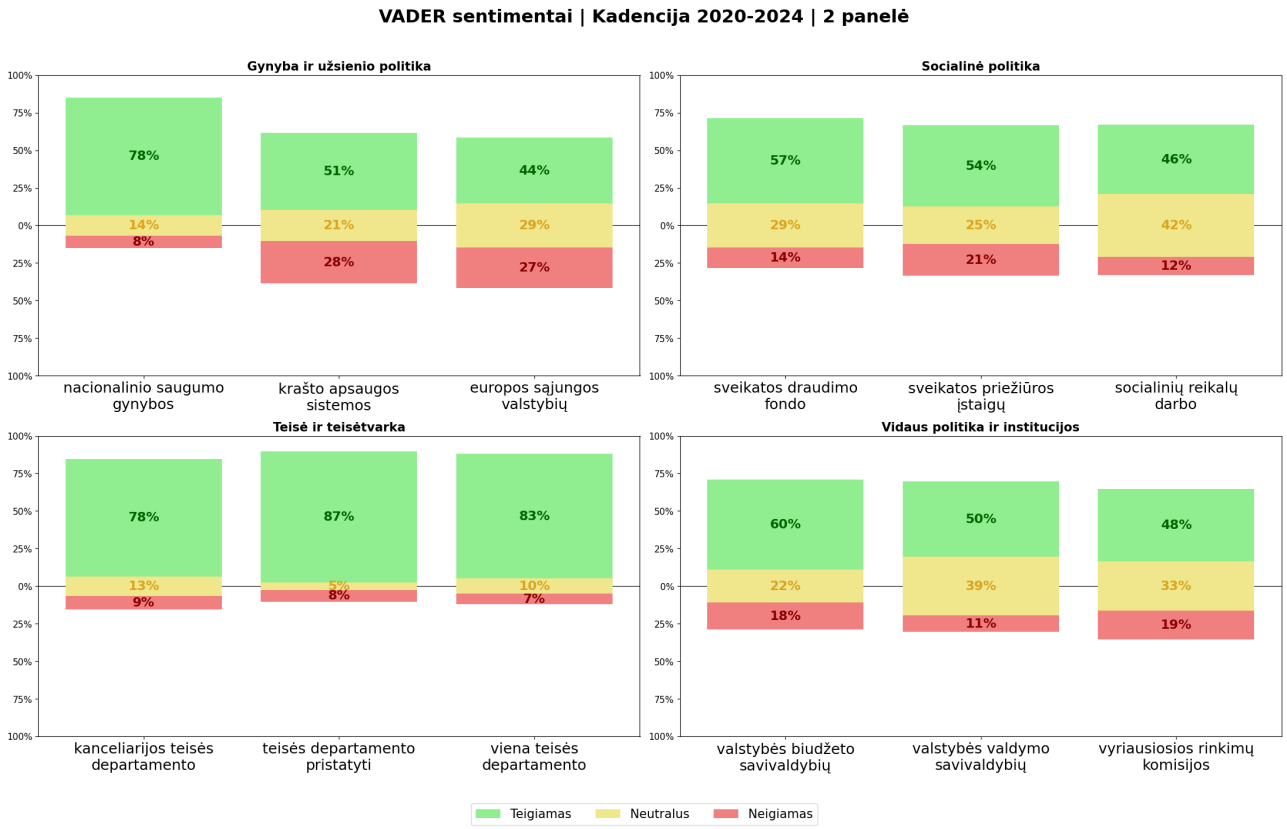
Klasteris	Raktažodžiai	Žyma
13	pensijų, šalpos, šalpos pensijų, pensijų kaupimo, kaupimo, valstybinių pensijų, valstybinių, anuitetų, pensijų anuitetų, anuitetų fondo, pensijos, fondo, pensijų šalpos, biudžeto, biudžeto rodiklių	socialinė politika
14	administracinių nusižengimų, nusižengimų, administracinių, nusižengimų administracinių, lydimasis administracinių, nusižengimų svarstymo, nusižengimų lydimasis, svarstymo, lydimasis, teisės, teisės teisėtvarkos, teisėtvarkos, niekas, nusižengimų dviejų, numatytu	teisė ir teisėtvarka
15	baudžiamojo, baudžiamojo proceso, proceso, proceso baudžiamojo, baudžiamojo baudžiamojo, lydimasis baudžiamojo, teisės teisėtvarkos, teisėtvarkos, es, teisės, svarstymo, niekas, baudžiamojo skyriaus, lydimasis, pateikimo	teisė ir teisėtvarka
16	švietimo, mokslo, švietimo mokslo, švietimo švietimo, ugdymo, pateikimo, sporto, mokyklos, mokyklų, pagrindinis švietimo, atmintinų, metais, mokymo, svarstymo, profesinio	socialinė politika
17	kontrolės, tabako, gaminių, ginklų, ginklų šaudmenų, šaudmenų kontrolės, šaudmenų, alkoholio, alkoholio kontrolės, tabako gaminių, gaminių kontrolės, tabako tabako, susijusių gaminių, gaminių jais, jais susijusių	teisė ir teisėtvarka
18	socialinio draudimo, draudimo, socialinio, valstybinio socialinio, valstybinio, motinystės, ligos motinystės, motinystės socialinio, ligos, nedarbo socialinio, nedarbo, draudimo pensijų, pensijų, draudimo fondo, fondo	socialinė politika
19	kultūros, paveldo, kultūros paveldo, etninės kultūros, etninės, globos, paveldo komisijos, kultūros centrų, valstybinės kultūros, valstybinės, kultūros globos, centrų, tarybos, bibliotekų, visuomenės informavimo	kultūra ir istorija
20	savivaldybių biudžetų, biudžetų, biudžeto savivaldybių, biudžeto, rodiklių, savivaldybių, valstybės biudžeto, visumos planuojamų, planuojamų rodiklių, konsoliduotos visumos, biudžetų konsoliduotos, konsoliduotos, visumos, planuojamų, valstybės	vidaus politika ir institucijos

Klasteris	Raktažodžiai	Žyma
21	viešųjų, pirkimų, viešųjų pirkimų, viešųjų įstaigų, pirkimų atliekamų, įstaigų, atliekamų, pirkimų viešųjų, vandentvarkos energetikos, atliekamų vandentvarkos, perkančiųjų, energetikos transporto, vandentvarkos, perkančiųjų subjektų, transporto pašto	ekonomika ir finansai
22	svarstymo, lydimasis, apsaugos, narių, niekas, balsavimas, stadija, balsavimo, frakcijos, numatytu, nematau, kelių, aplinkos, teisių, darbo	-
23	teismo, apeliacinio, apeliacinio teismo, teisėjo, aukščiausiojo teismo, aukščiausiojo, teismo teisėjo, teisėjo pareigų, teisėja, konstitucinio teismo, konstitucinio, teisėju, pareigų, teismų, teismo teisėju	teisė ir teisėtvarka
24	rinkimų, vyriausiosios, komisijos, rinkimų konstitucinio, konstitucinio, rinkimų komisijos, tarnybinės etikos, vyriausiosios rinkimų, tarnybinės, etikos komisijos, vyriausiosios tarnybinės, etikos, konstitucinio rinkimų, pirmalaikių, vrk	vidaus politika ir institucijos
25	darbo, valstybės, europos, žmonių, metais, būtent, vyriausybės, žmonės, eurų, atveju, pasakyti, turėtų, mln, teisės, metus	-
26	paramos, socialinės, socialinės paramos, piniginės socialinės, nepasiturintiems, nepasiturintiems gyventojams, paramos nepasiturintiems, piniginės, labdaros paramos, labdaros, neįgaliųjų socialinės, socialinės integracijos, integracijos, gyventojams, neįgaliųjų	socialinė politika
27	ratifikavimo, respublikos, susitarimo, europos, europos sąjungos, sąjungos, valstybių narių, apsaugos, vyriausybės, sąjungos valstybių, valstybių, baudžiamosiose bylose, baudžiamosiose, apsaugos sistemos, krašto apsaugos	gynyba ir užsienio politika
28	vietos savivaldos, savivaldos, vietos, savivaldos vietos, valdymo savivaldybių, savivaldybių, valstybės valdymo, pateikimo, valdymo, pagrindinis valstybės, valstybės, svarstymo, lydimasis, skirsnio, pagrindinis	vidaus politika ir institucijos

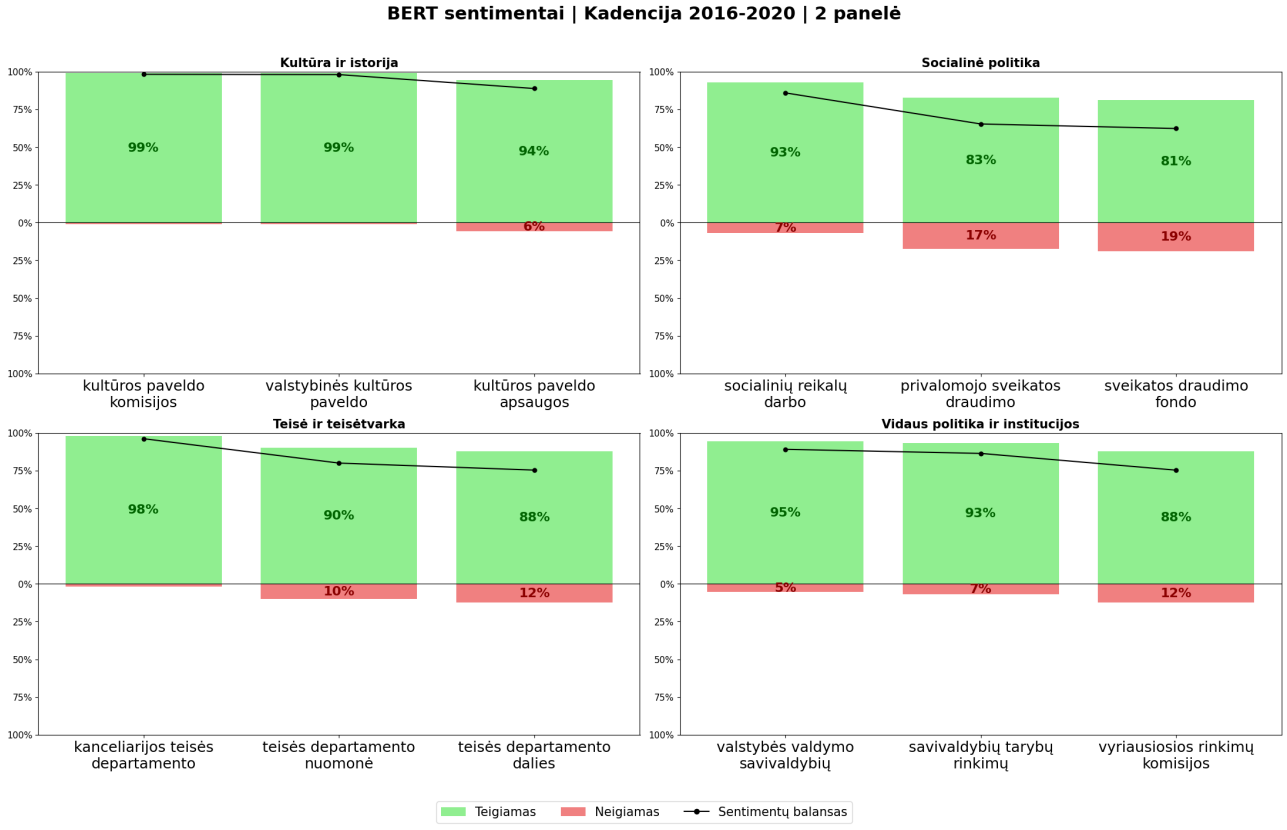
C priedas: Sentimentų analizės grafikai



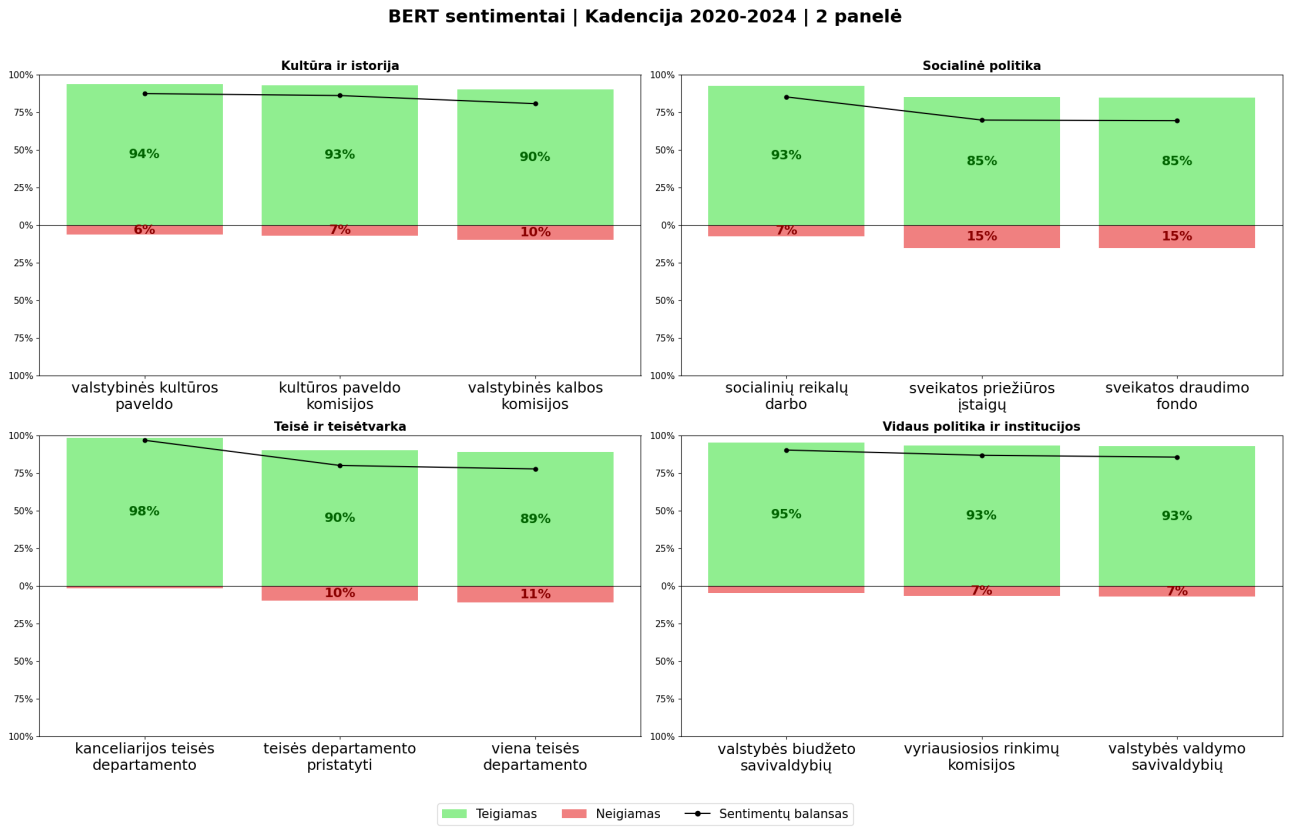
13 pav. *Diskusijų temų svarbiausių frazių VADER sentimentų pasiskirstymas (2016–2020) (2)*



14 pav. *Diskusijų temų svarbiausių frazių VADER sentimentų pasiskirstymas (2020–2024) (2)*



15 pav. *Diskusijų temų svarbiausių frazių BERT sentimentų pasiskirstymas (2016–2020) (2)*



16 pav. *Diskusijų temų svarbiausių frazių BERT sentimentų pasiskirstymas (2020–2024) (2)*

D priedas: Python kodas

```
import re
import pandas as pd
from wordcloud import WordCloud
import matplotlib.pyplot as plt
import numpy as np
import torch
import yake
from pathlib import Path
from docx import Document
from tokenizers import Tokenizer
from transformers import AutoModel
from huggingface_hub import hf_hub_download
from sklearn.metrics.pairwise import cosine_similarity
from tqdm.auto import tqdm
from IPython.display import display
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.cluster import KMeans
from keybert import KeyBERT
from sentence_transformers import SentenceTransformer, models
from pathlib import Path
```

```
import gc
from transformers import MarianMTModel, MarianTokenizer
from transformers import pipeline
from tqdm import tqdm
import nltk
from nltk.sentiment import SentimentIntensityAnalyzer
from matplotlib.patches import Patch
from matplotlib.lines import Line2D
import spacy
nlp_spacy = spacy.load("lt_core_news_sm")
from sklearn.metrics import silhouette_score
import matplotlib.pyplot as plt
from sklearn.metrics import pairwise_distances
from torch.utils.data import Dataset, DataLoader
from transformers import (
    AutoModelForSequenceClassification,
    get_scheduler)
from torch.cuda.amp import autocast, GradScaler
from sklearn.metrics import f1_score
from torch.optim import AdamW
from sklearn.model_selection import train_test_split
import time

ROOT_2016 = Path(r"C:\Users\Arnas\Downloads\stenogramos\2016 - 2020 metų kadencija")
ROOT_2020 = Path(r"C:\Users\Arnas\Downloads\stenogramos\2020 - 2024 metų kadencija")

OUTPUT_DIR = Path(r"C:\Users\Arnas\Downloads\stenogramos")
OUTPUT_DIR.mkdir(parents=True, exist_ok=True)

def read_docx_text(docx_path: Path) -> str:
    """
    Nuskaito tik .docx pastraipų tekstą.
    Lentelės nėra nuskaitomos.
    """
    doc = Document(docx_path)
    parts = []

    for p in doc.paragraphs:
        text = p.text.strip()
        if text:
            parts.append(text)

    return "\n".join(parts).strip()
```

```
def extract_posedzio_nr(filename: str):
    """
    Iš failo pavadinimo, pvz. 'Posėdis+Nr.29.docx', ištraukia 29.
    Veikia ir jei yra lietuviškos/nelietuviškos raidės ar smulkūs formatavimo
    ↪  skirtumai.
    """
    pattern = r"pos[eė]dis\+?nr\.?\s*(\d+)"
    m = re.search(pattern, filename, flags=re.IGNORECASE)
    return int(m.group(1)) if m else None


def extract_session_order(folder_name: str):
    """
    Iš aplanko pavadinimo ištraukia sesijos numerį ir tipą.
    Pvz.:
    '1 eilinė Seimo sesija (2016-11-14 - 2017-01-17)' -> (1, 'eilinė')
    '7 neeilinė Seimo sesija' -> (7, 'neeilinė')
    """
    pattern = r"(\d+)\s+(eilinė|neeilinė)"
    m = re.search(pattern, folder_name, flags=re.IGNORECASE)
    if m:
        return int(m.group(1)), m.group(2).lower()
    return None, None


def clean_session_name(folder_name: str) -> str:
    """
    Pašalina datas skliaustuose iš aplanko pavadinimo pabaigos, jei jos yra.
    """
    return re.sub(r"\s*\(.*?\)\s*$", "", folder_name).strip()


def process_kadencija(root_dir: Path, kadencija: str) -> pd.DataFrame:
    """
    Nuskaito vienos kadencijos visų sesijų .docx stenogramas į DataFrame.
    """
    records = []

    if not root_dir.exists():
        print(f"Klaida: aplankas nerastas -> {root_dir}")
        return pd.DataFrame()

    session_dirs = [p for p in root_dir.iterdir() if p.is_dir()]
```

```
all_docx_files = []
for session_dir in session_dirs:
    for docx_file in session_dir.glob("*.docx"):
        all_docx_files.append((session_dir, docx_file))

print(f"\nApdorojama kadencija: {kadencija}")
print(f"Rasta sesijų aplankų: {len(session_dirs)}")
print(f"Rasta .docx failų: {len(all_docx_files)}")

for session_dir, docx_file in tqdm(all_docx_files, desc=f"Nuskaitomos
↪  stenogramos {kadencija}"):
    session_name_raw = session_dir.name
    session_name = clean_session_name(session_name_raw)
    session_order, session_type = extract_session_order(session_name)

    posedzio_nr = extract_posedzio_nr(docx_file.name)

    try:
        text = read_docx_text(docx_file)
        error_msg = None
    except Exception as e:
        text = None
        error_msg = str(e)

    records.append({
        "kadencija": kadencija,
        "sesijos_aplankas": session_name_raw,
        "sesija": session_name,
        "sesijos_nr": session_order,
        "sesijos_tipas": session_type,
        "failo_pavadinimas": docx_file.name,
        "posedzio_nr": posedzio_nr,
        "failo_kelias": str(docx_file),
        "tekstas": text,
        "teksto_ilgis": len(text) if text else 0,
        "klaida": error_msg
    })

df = pd.DataFrame(records)

if df.empty:
    print(f"Kadencijai {kadencija} nerasta duomenų.")
    return df
```

```
    df = df.sort_values(
        by=["posedzio_nr", "sesijos_nr", "failo_pavadinimas"],
        na_position="last"
    ).reset_index(drop=True)

    print(f"Sėkmingai apdorota failų: {len(df)}")
    return df


def check_numbering(df: pd.DataFrame, kadencija: str):
    """
    Patikrina posėdžių numerių seką.
    """
    if df.empty:
        print(f"\n{kadencija}: duomenų nėra.")
        return

    valid_nums = df["posedzio_nr"].dropna().astype(int).sort_values().tolist()

    if valid_nums:
        expected = list(range(min(valid_nums), max(valid_nums) + 1))
        missing = sorted(set(expected) - set(valid_nums))
        duplicates = (df["posedzio_nr"][df["posedzio_nr"]
        .duplicated(keep=False)]
        .dropna()
        .astype(int)
        .tolist())

        print(f"\nNumeracijos patikra: {kadencija}")
        print(f"Mažiausias posėdžio nr.: {min(valid_nums)}")
        print(f"Didžiausias posėdžio nr.: {max(valid_nums)}")
        print(f"Trūkstami numeriai: {missing if missing else 'nėra'}")
        print(f"Besidubliuojantys numeriai: {sorted(set(duplicates)) if duplicates
        ↪  else 'nėra'}")
    else:
        print(f"\n{kadencija}: nepavyko rasti nė vieno posėdžio numerio.")


def save_outputs(df: pd.DataFrame, parquet_path: Path, csv_path: Path, label: str):
    """
    Išsaugo DataFrame į parquet ir csv.
    """
    if df.empty:
```

```
        print(f"{label}: nieko nesaugoma, nes DataFrame tuščias.")
        return

    df.to_parquet(parquet_path, index=False)
    df.to_csv(csv_path, index=False, encoding="utf-8-sig")

    print(f"\nIšsaugota: {label}")
    print(parquet_path)
    print(csv_path)

df_2016 = process_kadencija(ROOT_2016, "2016-2020")
df_2020 = process_kadencija(ROOT_2020, "2020-2024")

save_outputs(
    df_2016,
    OUTPUT_DIR / "seimo_stenogramos_2016_2020.parquet",
    OUTPUT_DIR / "seimo_stenogramos_2016_2020.csv",
    "2016-2020"
)

save_outputs(
    df_2020,
    OUTPUT_DIR / "seimo_stenogramos_2020_2024.parquet",
    OUTPUT_DIR / "seimo_stenogramos_2020_2024.csv",
    "2020-2024"
)

save_outputs(
    df_all,
    OUTPUT_DIR / "seimo_stenogramos_2016_2024.parquet",
    OUTPUT_DIR / "seimo_stenogramos_2016_2024.csv",
    "2016-2024"
)

df = pd.read_parquet("seimo_stenogramos_2016_2024.parquet")
print(df.columns)
print(df.head())

df['zodziu_sk'] = df['tekstas'].apply(lambda x: len(str(x).split()))

df['sakiniu_sk'] = df['tekstas'].apply(
    lambda x: len(re.findall(r'[.!?]+', str(x)))
)
```

```
# Žodžių kiekio mediana stenogramose
kadenciju_lentele = df.groupby('kadencija').agg(
    stenogramu_sk=('tekstas', 'count'),
    zodziu_sk=('zodziu_sk', 'sum'),
    vid_zodziu_stenogramoje=('zodziu_sk', 'mean'),
    med_zodziu_stenogramoje=('zodziu_sk', 'median'),
    sakiniu_sk=('sakiniu_sk', 'sum')
)

kadenciju_lentele['vid_sakinio_ilgis'] = (
    kadenciju_lentele['zodziu_sk'] / kadenciju_lentele['sakiniu_sk']
)

# Funkcija sakinių ilgiams suskaičiuoti
def sakiniu_ilgiai(text):
    sakiniai = re.split(r'[.!?]+', str(text))
    sakiniai = [s.strip() for s in sakiniai if s.strip()]
    return [len(s.split()) for s in sakiniai]

# Sakinio ilgio mediana pagal kadenciją
sakiniu_medianos = (
    df.groupby('kadencija')['tekstas']
    .apply(lambda tekstai: np.median([
        ilgis
        for tekstas in tekstai
        for ilgis in sakiniu_ilgiai(tekstas)
    ]) if any(sakiniu_ilgiai(t) for t in tekstai) else np.nan)
)

kadenciju_lentele['med_sakinio_ilgis'] = sakiniu_medianos

visi_sakiniu_ilgiai = [
    ilgis
    for tekstas in df['tekstas']
    for ilgis in sakiniu_ilgiai(tekstas)
]

bendra_eilute = pd.DataFrame({
    'stenogramu_sk': [len(df)],
    'zodziu_sk': [df['zodziu_sk'].sum()],
    'vid_zodziu_stenogramoje': [df['zodziu_sk'].mean()],
    'med_zodziu_stenogramoje': [df['zodziu_sk'].median()],
    'sakiniu_sk': [df['sakiniu_sk'].sum()],
    'vid_sakinio_ilgis': [df['zodziu_sk'].sum() / df['sakiniu_sk'].sum()],
```

```
    'med_sakinio_ilgis': [np.median(visi_sakiniu_ilgiai) if visi_sakiniu_ilgiai else
    ↪  np.nan]
}, index=['Bendrai'])

galutine_lentele = pd.concat([kadenciju_lentele, bendra_eilute])

galutine_lentele = galutine_lentele.round(2)

print("\nPradinė stenogramų analizė")
print("-----------------------------")
print(galutine_lentele)

kadencijos = df['kadencija'].unique()

for kadencija in kadencijos:

    tekstas = " ".join(
        df[df['kadencija'] == kadencija]['tekstas'].astype(str)
    )

    wc = WordCloud(
        width=1600,
        height=800,
        background_color='white',
        collocations=False
    ).generate(tekstas)

    plt.figure(figsize=(14, 7))
    plt.imshow(wc, interpolation='bilinear')
    plt.axis('off')
    plt.title(f'Žodžių debesis: {kadencija}', fontsize=16)
    plt.show()

def extract_speakers(text):
    if pd.isna(text):
        return []

    pattern = r'\b[A-ZĄČĘĖĮŠŲŪŽ]\.\s[A-ZĄČĘĖĮŠŲŪŽ]{2,}'
    return re.findall(pattern, str(text))

df['kalbetojai'] = df['tekstas'].apply(extract_speakers)

rezultatai = []
```

```
for kadencija in df['kadencija'].unique():

    subset = df[df['kadencija'] == kadencija]

    visi_kalbetojai = []

    for sarasas in subset['kalbetojai']:
        visi_kalbetojai.extend(sarasas)

    counter = Counter(visi_kalbetojai)

    unikalus_kiekis = len(counter)

    top5 = counter.most_common(5)

    top_rez = []

    for vardas, kiekis in top5:
        top_rez.append(f"{vardas} ({kiekis})")

    while len(top_rez) < 5:
        top_rez.append('')

    rezultatai.append({
        'kadencija': kadencija,
        'kalbėtojų skaičius': unikalus_kiekis,
        'TOP1': top_rez[0],
        'TOP2': top_rez[1],
        'TOP3': top_rez[2],
        'TOP4': top_rez[3],
        'TOP5': top_rez[4]
    })

    kalbetoju_lentele = pd.DataFrame(rezultatai)

print("\nKalbėtojų analizė pagal kadencijas")
print("-----------------------------------")
print(kalbetoju_lentele)

time_pattern = r'(\d{1,2}\.\d{2}\s*val\.)'

columns_to_keep = [col for col in df.columns
                   if col not in ["tekstas", "klaida", "failo_pavadinimas",
                   ↪  "failo_kelias"]]
```

```
rows = []

for idx, row in df.iterrows():
    text = str(row["tekstas"])
    parts = re.split(time_pattern, text)

    for i in range(1, len(parts), 2):
        laikas = parts[i].strip()
        segmentas = parts[i + 1].strip() if i + 1 < len(parts) else ""

        zodziu_kiekis = len(segmentas.split())

        if segmentas and zodziu_kiekis >= 30:
            new_row = {}

            for col in columns_to_keep:
                new_row[col] = row[col]

            new_row["laikas"] = laikas
            new_row["skyrelis"] = segmentas
            new_row["teksto_ilgis"] = len(segmentas)
            new_row["zodziu_kiekis"] = zodziu_kiekis

            rows.append(new_row)

df_segments = pd.DataFrame(rows)

print(df_segments.head())
print("Pradinis eilučių skaičius:", len(df))
print("Naujas skyrelių skaičius:", len(df_segments))

lt_stop = set(open("stopwords-lt.txt", encoding="utf-8").read().splitlines())
print(lt_stop)

# Pašalina nereikšmingus žodžius iš teksto naudojant regex
def regex_clean(text):

    text = re.sub(r'\bNr\.\s*[A-Z0-9-]+\b', ' ', text, flags=re.IGNORECASE)

    patterns = [
        r"\b\d+\s*straipsn\w*\b",
        r"\b\d+\s*str\.\b",
        r"\b\d+\s*dalis\b",
```

```
r"\b\d+\s*d\.\b",
r"\bnr\b",
r"\bm\b",
r"\bd\b",
r"\bstenograma\b",
r"\b(lietuvos respublikos seim(as|o)|seim\w*)\b",
r"\b(pavasario|rudens|žiemos|vasaros)\b",
r"\b(ryt|vakar)\w*\b",
r"\bposėdžio\b",
r"\bsesij\w*\b",
r"\blab(a|as) (diena|rytas|vakaras)\b",
r"\b(dien(a|os)|ryt(as|o)|vakar(as|o))\b",
r"\b(saus|vasar|kov|balandž|geguž|birželi|liep|rugpjūč|rugsė|spal|lapkr|gruo
↪  d)\w*\b",
r"\blietuvos respublikos\b",
r"\bpirmininkė\w*\b",
r"\bpavaduotoj\w*\b",
r"\bkviečiu\b",
r"\b(registr|užsireg)\w*\b",
r"\b(prašau|prašome|prašom)\b",
r"\bval\b",
r"\bsėst\w*\b",
r"\b(koleg\w*|gerbiam\w*)\b",
r"\bužsireg\w*\b",
 r"\bačiū\b",
r"\b(dėkoju|dėkui|dėkojame)\b",
r"\bšiandien\w*\b",
r"\b(pirmadien(io|is|į)|antradien(io|is)|trečiadien(io|is)|ketvirtadien(io|
↪  is)|penktadien(io|is)|šeštadien(io|is)|sekmadien(io|is))\b",
r"\b(tiek|tam|tiem|tokiu|tokią|tokį|tokie|tokių|tas|tų|tuo|tokio|tokios|to|
↪  tos|tą|tie|todėl|tada|ten|tuos)\b",
r"\b(šio|šių|šią|šį|šios|šioje|šiam|šito|šitą|šiuo|šiek)\b",
r"\b(kitą|kitu|kitų|kiti|kuris|kurie|kurio|kuriuo|kuriuos|kuriam|kurių|kuri
↪  os|kuria|kuri|kurį|kurią|ko|ką|kokio)\b",
r"\b(jo|jos|juo|ja|jie|jos|jį|ją|jų|jūsų|jam|jai)\b",
r"\b(viso|visi|visiems|visų|vieno|vieną)\b",
r"\bįstatym\w*\b",
r"\bprojekt\w*\b",
r"\b(būti|būtų|bus|buvo|nebuvo|nebūtų|nebus|yra|nebėra|nėra)\b",
r"\bpriimt\w*\b",
r"\bpritart\w*\b",
r"\bturim\w*\b",
r"\bturi\b",
r"\bčia\b",
```

```
r"\blabai\b",
r"\bgongas\b",
r"\bjau\b",
r"\blietuv\w*\b",
r"\balsavo\b",
r"\b(nariai|narys)\b",
r"\bsusilaikė\b",
r"\bpritarė\b",
r"\bministr\w*\b",
r"\bbalsavo\b",
r"\bkomitet\w*\b",
r"\bbalsuoj\w*\b",
r"\bbalsuot\w*\b",
r"\bmikrofon\w*\b",
r"\bposėdį\b",
r"\bpastab\w*\b",
r"\bpasiūlym\w*\b",
r"\bpritart\w*\b",
r"\bnepritart\w*\b",
r"\bsiūl\w*\b",
r"\bbalsavimas pradėtas\b",
r"\bsutar\w*\b",
r"\bruošiasi\b",
r"\bklausi\w*\b",
r"\bgalim\w*\b",
r"\bpriimame\b",
r"\bpriėmim\w*\b",
r"\bpastraipsniui\b",
r"\bbendru\b",
r"\bpritariam\w*\b",
r"\bmotyv\w*\b",
r"\bneužsiraš\w*\b",
r"\bužsirašius\w*\b",
r"\bniekas neužsiraš\w*\b",
r"\bnorinčių kalbėti\b",
r"\bkalbėt\w*\b",
r"\bpaklaus\w*\b",
r"\b(gauta|negauta|nėra gauta)\b",
r"\bkiekvienam\b",
r"\bbendru\b",
r"\bnutarim\w*\b",
r"\bsudarym\w*\b",
r"\bpakeitim\w*\b",
r"\bpapildym\w*\b",
```

```
r"\bpatvirtinim\w*\b",
r"\bpaskelbim\w*\b",
r"\bpriedo\b",
r"\bkodeks\w*\b",
r"\bdarbotvark\w*\b",
r"\brezervin\w*\b",
r"\bataskait\w*\b",
r"\brinkinio\b",
r"\b(metų|metu|metai)\b",
r"\batrodo\b",
r"\bgalima\b",
r"\breikia\b",
r"\bkartu\b",
r"\besmė\b",
r"\baišku\b",
r"\bmatyt\b",
r"\bdalyk\w*\b",
r"\bdabar\b",
r"\bgerai\b",
r"\bpavyzdžiui\b",
r"\bkitaip\b",
r"\bsvarstyti\b",
r"\bžinote\b",
r"\blaiko\b",
r"\bkalbame\b",
r"\bpranešėj\w*\b",
r"\bviską\b",
r"\b(nori|noriu|norėčiau)\b"
r"\bnėra\b",
r"\bstatut\w*\b",
r"\bpremjer\w*\b",
r"\bpriesaik\w*\b",
r"\bprisiek\w*\b",
r"\bmanote\b",
r"\bapsispresk\w*\b",
r"\bnepritar\w*\b",
r"\bpritar\w*\b",
r"\bbalsuo\w*\b",
r"\bgalėt\w*\b",
r"\btikrai\b",
r"\bmanau\b",
r"\bpatais\w*\b",
r"\biš tikrųjų\b",
r"\bsalė\w*\b",
```

```
        r"\bminėjau\b",
        r"\bdaug\w*\b",
        r"\bgali\b"
        ]

    for pattern in patterns:
        text = re.sub(pattern, " ", text, flags=re.IGNORECASE)

    text = re.sub(r"\s+", " ", text).strip()
    return text

# Pašalina skaičius ir visus simbolius, kurie nėra raidės, iš teksto
def basic_clean(text):
  text = re.sub(r"\d+", " ", text)
  text = re.sub(r"[^a-ząčęėįšųūž\s]", " ", text)
  return text

# Pašalina kalbėtojų kalbėtojų vardus iš teksto
def remove_speakers(text):

    text = re.sub(r"\b[A-ZĄČĘĖĮŠŲŪŽ]\.\s*(?:[A-ZĄČĘĖĮŠŲŪŽa-ząčęėįšųūž]+(?:-[A-ZĄČĘĖ
    ↪ ĮŠŲŪŽa-ząčęėįšųūž]+)*|[A-ZĄČĘĖĮŠŲŪŽ]+)\s*:?",
        " ", text)

    return text

# Sutvarko tarpus
def normalize_spaces(text):
  return re.sub(r"\s+", " ", text).strip()

# Pašalina nereikšmingus žodžius iš teksto
def stopwords_valymas(text, stopwords):
  return " ".join([w for w in text.split() if w not in stopwords])

# Apjungia visas valymo funkcijas
def clean(text, stopwords):
  text = str(text)
  text = re.sub(r"\(.*?\)", " ", text, flags=re.DOTALL) #valo tekstas skliaustuose
  text = remove_speakers(text)
  text = re.sub(r"\b[IVXLCDM]+\b", " ", text) # valo romėniškus skaičius
  text = text.lower()
  text = regex_clean(text)
  text = basic_clean(text)
  text = stopwords_valymas(text, stopwords)
```

```
  text = normalize_spaces(text)
  return text

df_2016_2020_segments = df_segments[df_segments["kadencija"] == "2016-2020"].copy()
df_2020_2024_segments = df_segments[df_segments["kadencija"] == "2020-2024"].copy()

df_2016_2020_segments["clean_text"] = df_2016_2020_segments["skyrelis"].apply(lambda
↪  x: clean(x, lt_stop))
df_2020_2024_segments["clean_text"] = df_2020_2024_segments["skyrelis"].apply(lambda
↪  x: clean(x, lt_stop))

sample_2016 = df_2016_2020_segments.sample(3, random_state=24)

print("\n===== 2016-2020 KADENCIJA =====\n")
for _, row in sample_2016.iterrows():
    print("--------- ORIGINALUS --------")
    print(row["skyrelis"][:5000])
    print("\n--------- VALYTAS ---------")
    print(row["clean_text"][:5000])
    print("\n====================\n")

sample_2016 = df_2020_2024_segments.sample(3, random_state=24)

print("\n===== 2020-2024 KADENCIJA =====\n")
for _, row in sample_2016.iterrows():
    print("--------- ORIGINALUS --------")
    print(row["skyrelis"][:5000])
    print("\n--------- VALYTAS ---------")
    print(row["clean_text"][:5000])
    print("\n====================\n")

def generate_wordcloud(df_kad, title=None):
    tekstas = " ".join(df_kad['clean_text'].astype(str))
    wc = WordCloud(width=1600, height=800, background_color='white',
    ↪  collocations=False).generate(tekstas)
    plt.figure(figsize=(14,7))
    plt.imshow(wc, interpolation='bilinear')
    plt.axis('off')
    if title is None:
        title = df_kad['kadencija'].iloc[0]
    plt.title(f'Žodžių debesis: {title}', fontsize=16)
    plt.show()

generate_wordcloud(df_2016_2020_segments, title="2016-2020")
```

```
generate_wordcloud(df_2020_2024_segments, title="2020-2024")

MODEL_NAME = "EMBEDDIA/litlat-bert"

tokenizer_path = hf_hub_download(repo_id=MODEL_NAME, filename="tokenizer.json")
tokenizer = Tokenizer.from_file(tokenizer_path)

model = AutoModel.from_pretrained(MODEL_NAME)

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model.to(device)
model.eval()

labels = {
    "gynyba_ir_uzsienis": (
        "krašto apsauga kariuomenė nato nacionalinis saugumas gynyba karas "
        "ukraina rusija sienų apsauga užsienio politika diplomatija sankcijos "
        "tarptautiniai santykiai geopolitika misijos"
    ),

    "ekonomika_ir_mokesciai": (
        "ekonomika biudžetas finansai mokesčiai pvm pajamos išlaidos deficitas "
        "infliacija investicijos ekonomikos augimas fiskalinė politika fondai"
    ),

    "socialine_politika": (
        "socialinė apsauga sveikata švietimas ligoninės gydytojai mokyklos "
        "pensijos pašalpos šeimos politika vaiko teisės gerovė neįgalieji"
    ),

    "teise_ir_teisetvarka": (
        "teisė teisėtvarka teismai policija prokuratūra konstitucinis teismas "
        "baudžiamoji atsakomybė administraciniai nusižengimai įstatymai"
    ),

    "vidaus_politika_ir_institucijos": (
        "vidaus politika seimas vyriausybė institucijos rinkimai partijos vrk "
        "komisijos įstatymų leidyba valdymas politinė sistema"
    ),

    "aplinka_ir_zemes_ukis": (
        "aplinkos apsauga klimato kaita ekologija miškai vanduo atliekos "
        "žemės ūkis ūkininkai kaimas žuvininkystė gamtos ištekliai"
    ),
```

```
    "energetika_ir_infrastruktura": (
        "energetika elektros energija dujos šildymas kainos transportas "
        "infrastruktūra viešieji pirkimai atsinaujinanti energija"
    ),

    "kultura_ir_istorija": (
        "kultūra menas paveldas muziejai istorija istorinė atmintis genocidas "
        "rezistencija švietimo kultūra mokslas universitetai"
    )
}

label_names = list(labels.keys())
label_texts = list(labels.values())

def encode_batch(texts, max_length=512):
    input_ids_list = []
    attention_masks = []

    for text in texts:
        enc = tokenizer.encode(str(text))
        ids = enc.ids[:max_length]
        attn = [1] * len(ids)
        if len(ids) < max_length:
            pad_len = max_length - len(ids)
            ids = ids + [1] * pad_len
            attn = attn + [0] * pad_len
        input_ids_list.append(ids)
        attention_masks.append(attn)

    input_ids = torch.tensor(input_ids_list, dtype=torch.long).to(device)
    attention_mask = torch.tensor(attention_masks, dtype=torch.long).to(device)
    return input_ids, attention_mask

def mean_pooling(model_output, attention_mask):
    token_embeddings = model_output.last_hidden_state
    input_mask_expanded =
    ↪  attention_mask.unsqueeze(-1).expand(token_embeddings.size()).float()
    summed = torch.sum(token_embeddings * input_mask_expanded, dim=1)
    counted = torch.clamp(input_mask_expanded.sum(dim=1), min=1e-9)
    return summed / counted

def get_embeddings(texts, batch_size=8, max_length=512):
    all_embeddings = []
```

```
    for i in tqdm(range(0, len(texts), batch_size), desc="Skaičiuojami
    ↪ embeddingai"):
        batch_texts = texts[i:i + batch_size]
        input_ids, attention_mask = encode_batch(batch_texts, max_length=max_length)
        with torch.no_grad():
            outputs = model(input_ids=input_ids, attention_mask=attention_mask)
        batch_embeddings = mean_pooling(outputs, attention_mask)
        batch_embeddings = batch_embeddings.cpu().numpy()
        all_embeddings.append(batch_embeddings)
    return np.vstack(all_embeddings)

label_embeddings = get_embeddings(label_texts, batch_size=4)

def classification(texts, label_names, label_embeddings, batch_size=8):
    text_embeddings = get_embeddings(texts, batch_size=batch_size)
    similarity_matrix = cosine_similarity(text_embeddings, label_embeddings)
    best_idx = similarity_matrix.argmax(axis=1)
    predicted_labels = [label_names[i] for i in best_idx]
    predicted_scores = similarity_matrix.max(axis=1)
    return predicted_labels, predicted_scores, similarity_matrix

df_work = pd.concat([df_2016_2020_segments, df_2020_2024_segments],
↪ ignore_index=True)
text_column = "clean_text"

predicted_labels, predicted_scores, similarity_matrix = classification(
    texts=df_work[text_column].tolist(),
    label_names=label_names,
    label_embeddings=label_embeddings,
    batch_size=8
)

df_work["tema"] = predicted_labels
df_work["temos_patikimumas"] = predicted_scores

for i, tema in enumerate(label_names):
    df_work[f"sim_{tema}"] = similarity_matrix[:, i]

df_temos = df_work.copy()

print("\n=== Pirmi klasifikavimo rezultatai ===")
display(df_temos[["kadencija", "laikas", "skyrelis", text_column, "tema",
↪ "temos_patikimumas"]].head(10))
```

```
print("\n=== Temų bendras pasiskirstymas ===")
tema_counts = df_temos["tema"].value_counts()
display(tema_counts.to_frame(name="kiekis"))

print("\n=== Temų pasiskirstymas pagal kadenciją ===")
tema_kad = pd.crosstab(df_temos["kadencija"], df_temos["tema"])
display(tema_kad)

tema_kad.plot(kind="bar", figsize=(12, 6))

plt.title("Temų pasiskirstymas pagal kadencijas")
plt.xlabel("Kadencija")
plt.ylabel("Įrašų skaičius")
plt.xticks(rotation=0)
plt.legend(
    title="Tema",
    labels=[
        "Aplinka ir žemės ūkis",
        "Ekonomika ir finansai",
        "Energetika ir infrastruktūra",
        "Gynyba ir užsienio politika",
        "Kultūra ir istorija",
        "Socialinė politika",
        "Teisė ir teisėtvarka",
        "Vidaus politika ir institucijos"])
plt.tight_layout()
plt.show()

def optimal_k(X, k_range=range(2, 30), label=None):
    sil_scores = []

    for k in k_range:
        kmeans = KMeans(n_clusters=k, random_state=8, n_init=10)
        labels = kmeans.fit_predict(X)

        score = silhouette_score(X, labels, metric='cosine')
        sil_scores.append(score)

    plt.plot(list(k_range), sil_scores, marker='o', label=label)
    plt.title("Silhouette metodo įvertis pagal klasterių skaičių")
    plt.xlabel("Klasterių skaičius (k)")
    plt.ylabel("Silhouette įvertis")
    plt.legend()
```

```
    best_k = list(k_range)[np.argmax(sil_scores)]
    print("Optimalus klasterių sk.:", best_k)

    return best_k

def cluster_text(df_kad, n_clusters=None):

    vectorizer = TfidfVectorizer(
        max_df=0.80,
        min_df=5,
        max_features=20000,
        ngram_range=(1, 2)
    )

    X = vectorizer.fit_transform(df_kad['clean_text'])

    if n_clusters is None:
      n_clusters = optimal_k(X, label=df_kad['kadencija'].iloc[0])

    kmeans = KMeans(
        n_clusters=n_clusters,
        random_state=8,
        n_init=10,
        init='k-means++',
        max_iter=300
    )

    kmeans.fit(X)

    df_kad = df_kad.copy()
    df_kad['cluster'] = kmeans.labels_

    terms = vectorizer.get_feature_names_out()
    centroids = kmeans.cluster_centers_

    print(f"\n--- {df_kad['kadencija'].iloc[0]} klasteriai ---")

    for i in range(n_clusters):
        top_terms = centroids[i].argsort()[-15:][::-1]
        print(f"{i} klasteris: {', '.join(terms[j] for j in top_terms)}")

    return df_kad, kmeans, vectorizer, X

# Artimiausių tekstų ištraukimas
```

```
def closest_texts(df, X, kmeans, cluster_id, n_return=5):
    df = df.reset_index(drop=True)

    mask = (df['cluster'].values == cluster_id)
    X_cluster = X[mask]

    if X_cluster.shape[0] == 0:
        return []

    centroid = kmeans.cluster_centers_[cluster_id].reshape(1, -1)

    distances = pairwise_distances(X_cluster, centroid, metric='cosine').ravel()

    idx_sorted = np.argsort(distances)

    return df.loc[mask, 'clean_text'].iloc[idx_sorted[:n_return]].tolist()

# 2016-2020 klasterizavimas
df_2016_2020_segments, kmeans_2016, vect_2016, X_2016 =
↪  cluster_text(df_2016_2020_segments)

# 2020-2024 klasterizavimas
df_2020_2024_segments, kmeans_2020, vect_2020, X_2020 =
↪  cluster_text(df_2020_2024_segments)

# Tekstų skaičius kiekviename klasteryje
print(df_2016_2020_segments['cluster'].value_counts().sort_index())
print(df_2020_2024_segments['cluster'].value_counts().sort_index())

# gynyba ir užsienio politika 2016
texts_2016_c23 = closest_texts(df_2016_2020_segments, X_2016, kmeans_2016, 23, 140)
# gynyba ir užsienio politika 2020
texts_2020_27 = closest_texts(df_2020_2024_segments, X_2020, kmeans_2020, 27, 140)

# ekonomika ir mokesčiai 2016
texts_2016_c3 = closest_texts(df_2016_2020_segments, X_2016, kmeans_2016, 3, 94)
# ekonomika ir mokesčiai  2020
texts_2020_c8 = closest_texts(df_2020_2024_segments, X_2020, kmeans_2020, 8, 93)
texts_2020_c21 = closest_texts(df_2020_2024_segments, X_2020, kmeans_2020, 21, 93)

# socialinė politika 2016
texts_2016_c1 = closest_texts(df_2016_2020_segments, X_2016, kmeans_2016, 1, 24)
texts_2016_c2 = closest_texts(df_2016_2020_segments, X_2016, kmeans_2016, 2, 24)
texts_2016_c4 = closest_texts(df_2016_2020_segments, X_2016, kmeans_2016, 4, 23)
```

```
texts_2016_c6 = closest_texts(df_2016_2020_segments, X_2016, kmeans_2016, 6, 23)
texts_2016_c18 = closest_texts(df_2016_2020_segments, X_2016, kmeans_2016, 18, 23)
texts_2016_c19 = closest_texts(df_2016_2020_segments, X_2016, kmeans_2016, 19, 23)
# socialinė politika 2020
texts_2020_c1 = closest_texts(df_2020_2024_segments, X_2020, kmeans_2020, 1, 24)
texts_2020_c12 = closest_texts(df_2020_2024_segments, X_2020, kmeans_2020, 12, 24)
texts_2020_c13 = closest_texts(df_2020_2024_segments, X_2020, kmeans_2020, 13, 23)
texts_2020_c16 = closest_texts(df_2020_2024_segments, X_2020, kmeans_2020, 16, 23)
texts_2020_c18 = closest_texts(df_2020_2024_segments, X_2020, kmeans_2020, 18, 23)
texts_2020_c26 = closest_texts(df_2020_2024_segments, X_2020, kmeans_2020, 26, 23)

# teisė ir teisėtvarka 2016
texts_2016_c7 = closest_texts(df_2016_2020_segments, X_2016, kmeans_2016, 7, 17)
texts_2016_c10 = closest_texts(df_2016_2020_segments, X_2016, kmeans_2016, 10, 17)
texts_2016_c11 = closest_texts(df_2016_2020_segments, X_2016, kmeans_2016, 11, 17)
texts_2016_c15 = closest_texts(df_2016_2020_segments, X_2016, kmeans_2016, 15, 17)
texts_2016_c16 = closest_texts(df_2016_2020_segments, X_2016, kmeans_2016, 16, 16)
texts_2016_c17 = closest_texts(df_2016_2020_segments, X_2016, kmeans_2016, 17, 16)
texts_2016_c20 = closest_texts(df_2016_2020_segments, X_2016, kmeans_2016, 20, 16)
texts_2016_c21 = closest_texts(df_2016_2020_segments, X_2016, kmeans_2016, 21, 16)
texts_2016_c24 = closest_texts(df_2016_2020_segments, X_2016, kmeans_2016, 24, 16)
# teisė ir teisėtvarka 2020
texts_2020_c2 = closest_texts(df_2020_2024_segments, X_2020, kmeans_2020, 2, 17)
texts_2020_c3 = closest_texts(df_2020_2024_segments, X_2020, kmeans_2020, 3, 17)
texts_2020_c5 = closest_texts(df_2020_2024_segments, X_2020, kmeans_2020, 5, 17)
texts_2020_c7 = closest_texts(df_2020_2024_segments, X_2020, kmeans_2020, 7, 17)
texts_2020_c14 = closest_texts(df_2020_2024_segments, X_2020, kmeans_2020, 14, 16)
texts_2020_c15 = closest_texts(df_2020_2024_segments, X_2020, kmeans_2020, 15, 16)
texts_2020_c17 = closest_texts(df_2020_2024_segments, X_2020, kmeans_2020, 17, 16)
texts_2020_c23 = closest_texts(df_2020_2024_segments, X_2020, kmeans_2020, 23, 16)

# vidus politika ir institucijos 2016
texts_2016_c0 = closest_texts(df_2016_2020_segments, X_2016, kmeans_2016, 0, 20)
texts_2016_c5 = closest_texts(df_2016_2020_segments, X_2016, kmeans_2016, 5, 20)
texts_2016_c9 = closest_texts(df_2016_2020_segments, X_2016, kmeans_2016, 9, 20)
texts_2016_c12 = closest_texts(df_2016_2020_segments, X_2016, kmeans_2016, 12, 20)
texts_2016_c26 = closest_texts(df_2016_2020_segments, X_2016, kmeans_2016, 26, 20)
texts_2016_c27 = closest_texts(df_2016_2020_segments, X_2016, kmeans_2016, 27, 20)
texts_2016_c28 = closest_texts(df_2016_2020_segments, X_2016, kmeans_2016, 28, 20)
# vidus politika ir institucijos 2020
texts_2020_c0 = closest_texts(df_2020_2024_segments, X_2020, kmeans_2020, 0, 20)
texts_2020_c6 = closest_texts(df_2020_2024_segments, X_2020, kmeans_2020, 6, 20)
texts_2020_c9 = closest_texts(df_2020_2024_segments, X_2020, kmeans_2020, 9, 20)
texts_2020_c10 = closest_texts(df_2020_2024_segments, X_2020, kmeans_2020, 10, 20)
```

```
texts_2020_c20 = closest_texts(df_2020_2024_segments, X_2020, kmeans_2020, 20, 20)
texts_2020_c24 = closest_texts(df_2020_2024_segments, X_2020, kmeans_2020, 24, 20)
texts_2020_c28 = closest_texts(df_2020_2024_segments, X_2020, kmeans_2020, 28, 20)

# aplinka ir žemės ūkis 2016
texts_2016_c22 = closest_texts(df_2016_2020_segments, X_2016, kmeans_2016, 22, 140)
# aplinka ir žemės ūkis 2020
texts_2020_c11 = closest_texts(df_2020_2024_segments, X_2020, kmeans_2020, 11, 140)

# energetika ir infrastruktūra 2016
texts_2016_c14 = closest_texts(df_2016_2020_segments, X_2016, kmeans_2016, 14, 140)
# energetika ir infrastruktūra 2020
texts_2020_c4 = closest_texts(df_2020_2024_segments, X_2020, kmeans_2020, 4, 140)

# kultūra ir istorija 2016
texts_2016_c13 = closest_texts(df_2016_2020_segments, X_2016, kmeans_2016, 13, 106)
# kultūra ir istorija 2020
texts_2020_c19 = closest_texts(df_2020_2024_segments, X_2020, kmeans_2020, 19, 150)

# Testavimo aibės sukūrimas
test_df_2016 = df_2016_2020_segments[
    df_2016_2020_segments['cluster'].isin([8, 25])
]

test_df_2020 = df_2020_2024_segments[
    df_2020_2024_segments['cluster'].isin([22, 25])
]

test_df = pd.concat([test_df_2016, test_df_2020], ignore_index=True)

print(test_df.shape)

test_df.to_parquet("test_df.parquet", index=False)

# Duomenų žymėjimas
def labeling(cluster_texts_dict):

    data = []

    for label, texts in cluster_texts_dict.items():
        for t in texts:
            data.append({"text": t, "label": label})

    return pd.DataFrame(data)
```

```
cluster_texts = {
    "gynyba_ir_uzsienis": texts_2016_c23 + texts_2020_27,
    "ekonomika_ir_mokesciai": texts_2016_c3 + texts_2020_c8 + texts_2020_c21,
    "socialine_politika": texts_2016_c1 + texts_2016_c2 + texts_2016_c4 +
    ↪  texts_2016_c6 + texts_2016_c18 + texts_2016_c19 + texts_2020_c1 +
    ↪  texts_2020_c12 + texts_2020_c13 + texts_2020_c16 + texts_2020_c18 +
    ↪  texts_2020_c26,
    "teise_ir_teisetvarka": texts_2016_c7 + texts_2016_c10 + texts_2016_c11 +
    ↪  texts_2016_c15 + texts_2016_c16 + texts_2016_c17 + texts_2016_c20 +
    ↪  texts_2016_c21 + texts_2016_c24 + texts_2020_c2 + texts_2020_c3 +
    ↪  texts_2020_c5 + texts_2020_c7 + texts_2020_c14 + texts_2020_c15 +
    ↪  texts_2020_c17 + texts_2020_c23,
    "vidaus_politika_ir_institucijos": texts_2016_c0 + texts_2016_c5 + texts_2016_c9
    ↪  + texts_2016_c12 + texts_2016_c26 + texts_2016_c27 + texts_2016_c28 +
    ↪  texts_2020_c0 + texts_2020_c6 + texts_2020_c9 + texts_2020_c10 +
    ↪  texts_2020_c20 + texts_2020_c24 + texts_2020_c28,
    "aplinka_ir_zemes_ukis": texts_2016_c22 + texts_2020_c11,
    "energetika_ir_infrastruktura": texts_2016_c14 + texts_2020_c4,
    "kultura_ir_istorija": texts_2016_c13 + texts_2020_c19
}

tuning_df = labeling(cluster_texts).sample(frac=1,
↪  random_state=8).reset_index(drop=True)
tuning_df.to_parquet("tuning_dataset.parquet", index=False)

total = sum(len(texts) for texts in cluster_texts.values())
print(total)

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

train_df, val_df = train_test_split(tuning_df, test_size=0.1,
↪  stratify=tuning_df["label"], random_state=8)

labels = sorted(tuning_df["label"].unique().tolist())
label2id = {l: i for i, l in enumerate(labels)}
id2label = {i: l for l, i in label2id.items()}

train_df["label_id"] = train_df["label"].map(label2id)
val_df["label_id"] = val_df["label"].map(label2id)

MODEL_NAME = "EMBEDDIA/litlat-bert"

tokenizer_path = hf_hub_download(repo_id=MODEL_NAME, filename="tokenizer.json")
```

```
tokenizer = Tokenizer.from_file(tokenizer_path)

model = AutoModelForSequenceClassification.from_pretrained(
    MODEL_NAME,
    num_labels=len(labels),
    id2label=id2label,
    label2id=label2id
)

model.gradient_checkpointing_enable()
model.to(device)

class TextDataset(Dataset):
    def __init__(self, df):
        self.texts = df["text"].tolist()
        self.labels = df["label_id"].tolist()

        self.pad_id = tokenizer.token_to_id("<pad>")
        self.cls_id = tokenizer.token_to_id("<s>")
        self.sep_id = tokenizer.token_to_id("</s>")

    def __len__(self):
        return len(self.labels)

    def __getitem__(self, idx):
        encoding = tokenizer.encode(self.texts[idx])

        input_ids = [self.cls_id] + encoding.ids + [self.sep_id]
        attention_mask = [1] * len(input_ids)

        max_len = 256

        if len(input_ids) > max_len:
            input_ids = input_ids[:max_len]
            attention_mask = attention_mask[:max_len]
        else:
            pad_len = max_len - len(input_ids)
            input_ids += [self.pad_id] * pad_len
            attention_mask += [0] * pad_len

        return {
            "input_ids": torch.tensor(input_ids),
            "attention_mask": torch.tensor(attention_mask),
            "labels": torch.tensor(self.labels[idx])
```

```
        }

train_dataset = TextDataset(train_df)
val_dataset = TextDataset(val_df)

train_loader = DataLoader(
    train_dataset,
    batch_size=8,
    shuffle=True,
    num_workers=2,
    pin_memory=True
)

val_loader = DataLoader(
    val_dataset,
    batch_size=8,
    num_workers=2,
    pin_memory=True
)

optimizer = AdamW(model.parameters(), lr=2e-5, weight_decay=0.01)

epochs = 3
num_training_steps = epochs * len(train_loader)
num_warmup_steps = int(0.1 * num_training_steps)

lr_scheduler = get_scheduler(
    "linear",
    optimizer=optimizer,
    num_warmup_steps=num_warmup_steps,
    num_training_steps=num_training_steps
)

scaler = GradScaler()

start_epoch = 0

for epoch in range(start_epoch, epochs):
    model.train()
    total_loss = 0

    for batch in tqdm(train_loader, desc=f"Epoch {epoch+1}"):
        batch = {k: v.to(device) for k, v in batch.items()}
```

```
        optimizer.zero_grad()

        with autocast():
            outputs = model(**batch)
            loss = outputs.loss

        scaler.scale(loss).backward()

        torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)

        scaler.step(optimizer)
        scaler.update()
        lr_scheduler.step()

        total_loss += loss.item()

    avg_train_loss = total_loss / len(train_loader)

    print(f"\nEpoch {epoch+1}")
    print(f"Train loss: {avg_train_loss:.4f}")

    model.eval()

    val_loss = 0
    all_preds = []
    all_labels = []

    with torch.no_grad():
        for batch in val_loader:
            batch = {k: v.to(device) for k, v in batch.items()}

            with autocast():
                outputs = model(**batch)
                loss = outputs.loss
                logits = outputs.logits

            val_loss += loss.item()

            preds = torch.argmax(logits, dim=1)

            all_preds.extend(preds.cpu().numpy())
            all_labels.extend(batch["labels"].cpu().numpy())

    avg_val_loss = val_loss / len(val_loader)
```

```
    acc = np.mean(np.array(all_preds) == np.array(all_labels))
    f1 = f1_score(all_labels, all_preds, average="weighted")

    print(f"Val loss: {avg_val_loss:.4f}")
    print(f"Val accuracy: {acc:.4f}")
    print(f"Val F1: {f1:.4f}")

    checkpoint = {
        "epoch": epoch + 1,
        "model_state_dict": model.state_dict(),
        "optimizer_state_dict": optimizer.state_dict(),
        "scheduler_state_dict": lr_scheduler.state_dict(),
        "scaler_state_dict": scaler.state_dict(),
        "train_loss": avg_train_loss,
        "val_loss": avg_val_loss,
        "val_acc": acc,
        "val_f1": f1,
        "timestamp": time.strftime("%Y-%m-%d %H:%M:%S")
    }

    torch.save(checkpoint, f"checkpoint_epoch_{epoch+1}.pt")

    print(f"Checkpoint saved: epoch_{epoch+1}")

class InferenceDataset(Dataset):
    def __init__(self, df):
        self.texts = df["clean_text"].tolist()

        self.pad_id = tokenizer.token_to_id("<pad>")
        self.cls_id = tokenizer.token_to_id("<s>")
        self.sep_id = tokenizer.token_to_id("</s>")

    def __len__(self):
        return len(self.texts)

    def __getitem__(self, idx):
        encoding = tokenizer.encode(self.texts[idx])

        input_ids = [self.cls_id] + encoding.ids + [self.sep_id]
        attention_mask = [1] * len(input_ids)

        max_len = 256

        if len(input_ids) > max_len:
```

```
            input_ids = input_ids[:max_len]
            attention_mask = attention_mask[:max_len]
        else:
            pad_len = max_len - len(input_ids)
            input_ids += [self.pad_id] * pad_len
            attention_mask += [0] * pad_len

        return {
            "input_ids": torch.tensor(input_ids),
            "attention_mask": torch.tensor(attention_mask)
        }

def predict(model, loader):
    model.eval()
    preds_all = []

    with torch.no_grad():
        for batch in loader:
            batch = {k: v.to(device) for k, v in batch.items()}

            with autocast():
                logits = model(**batch).logits

            preds = torch.argmax(logits, dim=1)
            preds_all.extend(preds.cpu().numpy())

    return preds_all

test_dataset = InferenceDataset(test_df)
test_loader = DataLoader(test_dataset, batch_size=8)

test_preds = predict(model, test_loader)
test_df["pred_label"] = [id2label[i] for i in test_preds]

test_df[["clean_text", "pred_label"]].sample(10, random_state=42)

label_mapping = {
    "aplinka_ir_zemes_ukis": "Aplinkos ir žemės ūkis",
    "ekonomika_ir_mokesciai": "Ekonomika ir mokesčiai",
    "energetika_ir_infrastruktura": "Energetika ir infrastruktūra",
    "gynyba_ir_uzsienis": "Gynyba ir užsienis",
    "kultura_ir_istorija": "Kultūra ir istorija",
    "socialine_politika": "Socialinė politika",
    "teise_ir_teisetvarka": "Teisė ir teisėtvarka",
```

```
    "vidaus_politika_ir_institucijos": "Vidaus politika ir institucijos"
}

label_counts = test_df["pred_label"].value_counts()

label_counts.index = [
    label_mapping[label]
    for label in label_counts.index
]

plt.figure(figsize=(10, 8))

ax = label_counts.sort_values().plot(
    kind="barh",
    color="#4C72B0"
)

plt.title("Mišrių klasterių prognozuotų klasių pasiskirstymas")
plt.xlabel("Priskirtų tekstų skaičius")
plt.ylabel("Prognozuota klasė")

for container in ax.containers:
    ax.bar_label(container, padding=3)

plt.grid(axis='x', linestyle='--', alpha=0.3)

plt.tight_layout()
plt.show()

# Visų tekstų klasifikavimas
inf_dataset_1 = InferenceDataset(df_2016_2020_segments)
inf_loader_1 = DataLoader(inf_dataset_1, batch_size=8)

inf_dataset_2 = InferenceDataset(df_2020_2024_segments)
inf_loader_2 = DataLoader(inf_dataset_2, batch_size=8)

preds_2016_2020 = predict(model, inf_loader_1)
preds_2020_2024 = predict(model, inf_loader_2)

df_2016_2020_segments["pred_label"] = [id2label[i] for i in preds_2016_2020]
df_2020_2024_segments["pred_label"] = [id2label[i] for i in preds_2020_2024]

sampled_texts = (
    df_2016_2020_segments
```

```
    .groupby("pred_label")["skyrelis"]
    .apply(lambda x: x.sample(min(4, len(x)), random_state=8))
)

for label in sampled_texts.index.get_level_values(0).unique():
    print("\n" + "="*80)
    print(label)
    print("="*80)

    texts = sampled_texts.loc[label]

    for i, text in enumerate(texts, 1):
        print(f"\n{i}. {text}\n")

counts_2016_2020 = df_2016_2020_segments["pred_label"].value_counts().sort_index()
counts_2020_2024 = df_2020_2024_segments["pred_label"].value_counts().sort_index()

comparison_df = pd.DataFrame({
    "2016-2020": counts_2016_2020,
    "2020-2024": counts_2020_2024
}).fillna(0)

print(comparison_df)

new_label = [
    "Aplinkos ir žemės ūkis",
    "Ekonomika ir finansai",
    "Energetika ir infrastruktūra",
    "Gynyba ir užsienio politika",
    "Kultūra ir istorija",
    "Socialinė politika",
    "Teisė ir teisetvarka",
    "Vidaus politika ir institucijos"
]

ax = comparison_df.plot(
    kind="bar",
    figsize=(12, 7),
    color=["#80668c", "#709470"]
)

plt.title("Prognozuotų klasių pasiskirstymas", fontsize=16)
plt.xlabel("Prognozuota klasė", fontsize=13)
plt.ylabel("Tekstų skaičius", fontsize=13)
```

```
ax.set_xticklabels(new_label, rotation=30, ha='right')

plt.legend(title="Laikotarpis")

for container in ax.containers:
    ax.bar_label(
        container,
        fmt='%d',
        padding=3,
        fontsize=10
    )

plt.tight_layout()
plt.show()

# Svarbiausių frazių išrinkimas naudojant TF-IDF metodą
df1 = pd.read_parquet("pred_2016_2020.parquet")
df2 = pd.read_parquet("pred_2020_2024.parquet")

text_col = "clean_text"
top_n = 3

def extract_tfidf_phrases(df, df_name, kadencija):
    results = []

    labels = df["pred_label"].dropna().unique()

    # Progress bar per temas
    for label in tqdm(labels, desc=f"Processing {df_name}"):

        texts = (
            df.loc[df["pred_label"] == label, text_col]
            .dropna()
            .astype(str)
        )

        vectorizer = TfidfVectorizer(
            ngram_range=(2, 3),
            sublinear_tf=False
        )

        tfidf_matrix = vectorizer.fit_transform(texts)
```

```
        feature_names = vectorizer.get_feature_names_out()

        mean_tfidf = tfidf_matrix.mean(axis=0).A1

        top_indices = mean_tfidf.argsort()[::-1][:top_n]

        for rank, idx in enumerate(top_indices, start=1):
            results.append({
                "dataset": df_name,
                "kadencija": kadencija,
                "tema": label,
                "rank": rank,
                "fraze": feature_names[idx],
                "tfidf_score": round(mean_tfidf[idx], 5)
            })

    return pd.DataFrame(results)

df1_tfidf_phrases = extract_tfidf_phrases(
    df1,
    "df1",
    "2016-2020"
)

df2_tfidf_phrases = extract_tfidf_phrases(
    df2,
    "df2",
    "2020-2024"
)

df1_tfidf_phrases.to_parquet(
    "2016-2020_tfidf_phrases.parquet",
    index=False
)

df2_tfidf_phrases.to_parquet(
    "2020-2024_tfidf_phrases.parquet",
    index=False
)

print("Parquet failai išsaugoti.")


# Svarbiausių frazių išrinkimas naudojant YAKE metodą
```

```
df1 = pd.read_parquet("pred_2016_2020.parquet")
df2 = pd.read_parquet("pred_2020_2024.parquet")

lt_stop = set(open("stopwords-lt.txt", encoding="utf-8").read().splitlines())
print(lt_stop)

REGEX_PATTERNS = [
    r"\b\d+\s*straipsn\w*\b",
    r"\b\d+\s*str\.\b",
    r"\b\d+\s*dalis\b",
    r"\b\d+\s*d\.\b",
    r"\bnr\b",
    r"\bm\b",
    r"\bd\b",
    r"\bstenograma\b",
    r"\b(lietuvos respublikos seim(as|o)|seim\w*)\b",
    r"\b(pavasario|rudens|žiemos|vasaros)\b",
    r"\b(ryt|vakar)\w*\b",
    r"\bposėdžio\b",
    r"\bsesij\w*\b",
    r"\blab(a|as) (diena|rytas|vakaras)\b",
    r"\b(dien(a|os)|ryt(as|o)|vakar(as|o))\b",
    r"\b(saus|vasar|kov|balandž|geguž|biržel|liep|rugpjūč|rugsė|spal|lapkr|gruod)\w
    ↪  *\b",
    r"\blietuvos respublikos\b",
    r"\bpirmininк\w*\b",
    r"\bpavaduotoj\w*\b",
    r"\bkviečiu\b",
    r"\b(registr|užsireg)\w*\b",
    r"\b(prašau|prašome|prašom)\b",
    r"\bval\b",
    r"\bsėst\w*\b",
    r"\b(koleg\w*|gerbiam\w*)\b",
    r"\bužsireg\w*\b",
    r"\bačiū\b",
    r"\b(dėkoju|dėkui|dėkojame)\b",
    r"\bšiandien\w*\b",
    r"\b(pirmadien(io|is|į)|antradien(io|is)|trečiadien(io|is)|ketvirtadien(io|is)|
    ↪  penktadien(io|is)|šeštadien(io|is)|sekmadien(io|is))\b",
    r"\b(tiek|tam|tiem|tokiu|tokią|tokį|tokie|tokių|tas|tų|tuo|tokio|tokios|to|tos|
    ↪  tą|tie|todėl|tada|ten|tuos)\b",
    r"\b(šio|šių|šią|šį|šios|šioje|šiam|šito|šitą|šiuo|šiek)\b",
```

```
r"\b(kitą|kitu|kitų|kiti|kuris|kurie|kurio|kuriuo|kuriuos|kuriam|kurių|kurios|k⌋
↪  uria|kuri|kurį|kurią|ko|ką|kokio)\b",
r"\b(jo|jos|juo|ja|jie|jos|jį|ją|jų|jūsų|jam|jai)\b",
r"\b(viso|visi|visiems|visų|vieno|vieną)\b",
r"\bįstatym\w*\b",
r"\bprojekt\w*\b",
r"\b(būti|būtų|bus|buvo|nebuvo|nebūtų|nebus|yra|nebėra|nėra)\b",
r"\bpriimt\w*\b",
r"\bpritart\w*\b",
r"\bturim\w*\b",
r"\bturi\b",
r"\bčia\b",
r"\blabai\b",
r"\bgongas\b",
r"\bjau\b",
r"\blietuv\w*\b",
r"\balsavo\b",
r"\b(nariai|narys)\b",
r"\bsusilaikė\b",
r"\bpritarė\b",
r"\bministr\w*\b",
r"\bkomitet\w*\b",
r"\bbalsuoj\w*\b",
r"\bbalsuot\w*\b",
r"\bmikrofon\w*\b",
r"\bposėdį\b",
r"\btvirtin\w*\b",
r"\bpastab\w*\b",
r"\bpasiūlym\w*\b",
r"\bnepritart\w*\b",
r"\bsiūl\w*\b",
r"\bbalsavimas pradėtas\b",
r"\bsutar\w*\b",
r"\bruošiasi\b",
r"\bklausi\w*\b",
r"\bgalim\w*\b",
r"\bpriimame\b",
r"\bstraipsn\w*\b",
r"\bpriėmim\w*\b",
r"\bpastraipsniui\b",
r"\bbendru\b",
r"\bpritariam\w*\b",
r"\bmotyv\w*\b",
r"\bneužsiraš\w*\b",
```

```
r"\bužsirašius\w*\b",
r"\bniekas neužsiraš\w*\b",
r"\bnorinčių kalbėti\b",
r"\bkalbėt\w*\b",
r"\bpaklaus\w*\b",
r"\b(gauta|negauta|nėra gauta)\b",
r"\bkiekvienam\b",
r"\bnutarim\w*\b",
r"\bsudarym\w*\b",
r"\bpakeitim\w*\b",
r"\bpapildym\w*\b",
r"\bpatvirtinim\w*\b",
r"\bpaskelbim\w*\b",
r"\bpriedo\b",
r"\bkodeks\w*\b",
r"\bdarbotvark\w*\b",
r"\brezervin\w*\b",
r"\bataskait\w*\b",
r"\brinkinio\b",
r"\b(metų|metu|metai)\b",
r"\batrodo\b",
r"\bgalima\b",
r"\breikia\b",
r"\bkartu\b",
r"\besmė\b",
r"\baišku\b",
r"\bmatyt\b",
r"\bdalyk\w*\b",
r"\bdabar\b",
r"\bgerai\b",
r"\bpavyzdžiui\b",
r"\bkitaip\b",
r"\bsvarstyti\b",
r"\bžinote\b",
r"\blaiko\b",
r"\bkalbame\b",
r"\bpranešėj\w*\b",
r"\bviską\b",
r"\b(nori|noriu|norėčiau)\b",
r"\bvienbals\w*\b",
r"\bsusilaik\w*\b",
r"\bdien\w*\b",
r"\bvaland\w*\b",
r"\bpatiksl\w*\b",
```

```
    r"\btikslin\w*\b",
    r"\bposėd\w*\b",
    r"\bvardu\b",
    r"\bstatut\w*\b",
    r"\bpremjer\w*\b",
    r"\bpriesaik\w*\b",
    r"\bprisiek\w*\b",
    r"\bsveiki\b",
    r"\bmanote\b",
    r"\bapsispresk\w*\b",
    r"\bbalsuo\w*\b",
    r"\bgalėt\w*\b",
    r"\btikrai\b",
    r"\bmanau\b",
    r"\bpatais\w*\b",
    r"\biš tikrųjų\b",
    r"\bsalė\w*\b",
    r"\bminėjau\b",
    r"\bdaug\w*\b",
    r"\bgali\b"
]

COMBINED_REGEX = re.compile("|".join(REGEX_PATTERNS), flags=re.IGNORECASE)

NR_REGEX = re.compile(r"\bNr\.\s*[A-Z0-9-]+\b", flags=re.IGNORECASE)
BRACKETS_REGEX = re.compile(r"\(.*?\)", flags=re.DOTALL)
ROMAN_REGEX = re.compile(r"\b[IVXLCDM]+\b", flags=re.IGNORECASE)
DIGITS_REGEX = re.compile(r"\d+")
BASIC_ALLOWED_REGEX = re.compile(
    r"[^a-ząčęėįšųūž\.\,\!\?\:\;\-\(\)\s]",
    flags=re.IGNORECASE
)
SPACES_REGEX = re.compile(r"\s+")

SPEAKER_REGEX = re.compile(
    r"\b[A-ZĄČĘĖĮŠŲŪŽ]\.\s*[A-ZĄČĘĖĮŠŲŪŽa-ząčęėįšųūž\-]+\.?:?"
)

NAME_PAIR =
↪  r"[A-ZĄČĘĖĮŠŲŪŽ][a-ząčęėįšųūž]+(?:\s+[A-ZĄČĘĖĮŠŲŪŽ][a-ząčęėįšųūž]+){1,2}"
NAME_LIST_REGEX = re.compile(rf"(?:{NAME_PAIR}\s+){{4,}}")


def remove_speakers_and_name_lists(text):
```

```
    text = SPEAKER_REGEX.sub(" ", text)
    text = NAME_LIST_REGEX.sub(" ", text)
    return text


def regex_clean(text):
    text = NR_REGEX.sub(" ", text)
    text = COMBINED_REGEX.sub(" ", text)
    return SPACES_REGEX.sub(" ", text).strip()


def basic_clean(text):
    text = DIGITS_REGEX.sub(" ", text)
    text = BASIC_ALLOWED_REGEX.sub(" ", text)
    return text


def clean_punctuation_artifacts(text):
    text = re.sub(r"\s+[--]\s*[.,!?;:]\s+", " ", text)
    text = re.sub(r"[.,!?;:]\s+[--]\s*[.,!?;:]", " ", text)
    text = re.sub(r"\.\s+-\s+", ". ", text)
    text = re.sub(r"\s+([.,!?;:])", r"\1", text)
    text = re.sub(r"\s*-\s*", " - ", text)
    text = re.sub(r"([.,!?;:]){2,}", r"\1", text)
    return SPACES_REGEX.sub(" ", text).strip()


def stopwords_valymas(text, stopwords):
    return " ".join(
        w for w in text.split()
        if w not in stopwords
    )


def clean(text, stopwords):
    text = str(text)

    text = BRACKETS_REGEX.sub(" ", text)
    text = remove_speakers_and_name_lists(text)
    text = ROMAN_REGEX.sub(" ", text)
    text = text.lower()
    text = regex_clean(text)
    text = basic_clean(text)
    text = clean_punctuation_artifacts(text)
```

```
    text = stopwords_valymas(text, stopwords)
    text = SPACES_REGEX.sub(" ", text).strip()

    return text

df_2016_2020_segments = df1[df1["kadencija"] == "2016-2020"].copy()
df_2020_2024_segments = df2[df2["kadencija"] == "2020-2024"].copy()

df_2016_2020_segments["clean_text"] = df_2016_2020_segments["skyrelis"].apply(lambda
↪  x: clean(x, lt_stop))
df_2020_2024_segments["clean_text"] = df_2020_2024_segments["skyrelis"].apply(lambda
↪  x: clean(x, lt_stop))

text_col = "clean_text"
top_n = 10

def has_repeated_word(phrase):
    words = phrase.lower().split()
    return len(words) != len(set(words))

def extract_yake_phrases(df, df_name, kadencija):
    results = []

    labels = df["pred_label"].dropna().unique()

    for label in tqdm(labels, desc=f"Processing {df_name}"):

        texts = (
            df.loc[df["pred_label"] == label, text_col]
            .dropna()
            .astype(str)
        )

        # Sujungia visus temos tekstus
        combined_text = " ".join(texts)

        kw_extractor = yake.KeywordExtractor(
            lan="lt",
            n=3,
            top=top_n * 3,
            dedupLim=0.5,
            stopwords=None
        )
```

```
        keywords = kw_extractor.extract_keywords(combined_text)

        filtered_keywords = []

        for phrase, score in keywords:
            if not has_repeated_word(phrase):
                filtered_keywords.append((phrase, score))

            if len(filtered_keywords) == top_n:
                break

        for rank, (phrase, score) in enumerate(filtered_keywords, start=1):
            results.append({
                "dataset": df_name,
                "kadencija": kadencija,
                "tema": label,
                "rank": rank,
                "fraze": phrase,
                "yake_score": score
            })

    return pd.DataFrame(results)

df1_yake_phrases = extract_yake_phrases(
    df1,
    "df1",
    "2016-2020"
)

df2_yake_phrases = extract_yake_phrases(
    df2,
    "df2",
    "2020-2024"
)

df1_yake_phrases.to_parquet(
    "2016-2020_yake_phrases_sakiniai3.parquet",
    index=False
)

df2_yake_phrases.to_parquet(
    "2020-2024_yake_phrases_sakiniai3.parquet",
    index=False
)
```

```
print("YAKE parquet failai išsaugoti.")

# Svarbiausių frazių išrinkimas naudojant KeyBERT metodą

df1 = pd.read_parquet("pred_2016_2020.parquet")
df2 = pd.read_parquet("pred_2020_2024.parquet")

REGEX_PATTERNS = [
    # Teisėkūros / dokumentų technika
    r"\b\d+\s*straipsn\w*\b",
    r"\b\d+\s*str\.\b",
    r"\b\d+\s*dal\w*\b",
    r"\b\d+\s*d\.\b",
    r"\bnr\.?\b",
    r"\bstenogram\w*\b",
    r"\bįstatym\w*\b",
    r"\bprojekt\w*\b",
    r"\bnutarim\w*\b",
    r"\bkodeks\w*\b",
    r"\bdarbotvark\w*\b",
    r"\bpakeitim\w*\b",
    r"\bpapildym\w*\b",
    r"\bpatvirtinim\w*\b",
    r"\bpaskelbim\w*\b",
    r"\bpried\w*\b",
    r"\brinkin\w*\b",
    r"\bdepartament\w*\b",
    r"\bministerij\w*\b",
    r"\bnuostat\w*\b",
    r"\bprocedūr\w*\b",
    r"\bpunkt\w*\b",
    r"\bstraipsn\w*\b",
    r"\bpatais\w*\b",
    r"\bstatut\w*\b",

    # Seimo / posėdžio procedūros
    r"\b(lietuvos respublikos seim(as|o)|seim\w*)\b",
    r"\blietuvos respublikos\b",
    r"\bposėd\w*\b",
    r"\bsesij\w*\b",
    r"\bpirminink\w*\b",
    r"\bpavaduotoj\w*\b",
    r"\bkomitet\w*\b",
```

```
r"\bministr\w*\b",
r"\bpranešėj\w*\b",
r"\btribūn\w*\b",
r"\bmikrofon\w*\b",
r"\bsal\w*\b",
r"\bgong\w*\b",

# Procedūriniai veiksmai
r"\bkvie\w*\b",
r"\bpraš\w*\b",
r"\bregistr\w*\b",
r"\bužsireg\w*\b",
r"\bsėst\w*\b",
r"\bpriimt\w*\b",
r"\bpriim\w*\b",
r"\bpriėm\w*\b",
r"\bpriėmim\w*\b",
r"\bprit\w*\b",
r"\bpritar\w*\b",
r"\bneprit\w*\b",
r"\bnepritar\w*\b",
r"\bbals\w*\b",
r"\bsusilaik\w*\b",
r"\bsiūl\w*\b",
r"\bsutar\w*\b",
r"\bklaus\w*\b",
r"\bpaklaus\w*\b",
r"\bkalbėt\w*\b",
r"\bužsiraš\w*\b",
r"\bneužsiraš\w*\b",
r"\bpastraipsniui\b",
r"\bbendru\b",
r"\bmotyv\w*\b",
r"\bpasiūlym\w*\b",
r"\bpastab\w*\b",
r"\bgaut\w*\b",
r"\bnegaut\w*\b",
r"\bsvarst\w*\b",
r"\bpristat\w*\b",
r"\bdiskusij\w*\b",
r"\bdiskutu\w*\b",
r"\bpasisakym\w*\b",
r"\bprotokol\w*\b",
r"\bįraš\w*\b",
```

```
r"\bskub\w*\b",
r"\btaikom\w*\b",
r"\bnetaikom\w*\b",
r"\bnumatyt\w*\b",
r"\bsuderin\w*\b",
r"\bpakoreg\w*\b",
r"\bįsigalioj\w*\b",
r"\bstadij\w*\b",
r"\btvark\w*\b",
r"\breferendum\w*\b",
r"\bpertrauk\w*\b",

# Kreipiniai / mandagumo frazės
r"\bkoleg\w*\b",
r"\bgerbiam\w*\b",
r"\bmiel\w*\b",
r"\bačiū\b",
r"\bdėk\w*\b",
r"\blab(a|as)\s+(diena|rytas|vakaras)\b",
r"\batsipraš\w*\b",

# Laikas / datos
r"\bval\b",
r"\bm\b",
r"\bd\b",
r"\bmet\w*\b",
r"\bdien\w*\b",
r"\bryt\w*\b",
r"\bvakar\w*\b",
r"\bšiandien\w*\b",
r"\b(pavasar|ruden|žiem|vasar)\w*\b",
r"\b(saus|vasar|kov|balandž|geguž|birželi|liep|rugpjūč|rugsė|spal|lapkr|gruod)\w
↪  *\b",
r"\b(pirmadien|antradien|trečiadien|ketvirtadien|penktadien|šeštadien|sekmadien
↪  )\w*\b",

# Bendri stop žodžiai / įvardžiai
r"\b(tiek|tam|tiem|tokiu|tokią|tokį|tokie|tokių|tas|tų|tuo|tokio|tokios|to|tos|
↪  tą|tie|todėl|tada|ten|tuos)\b",
r"\b(šio|šių|šią|šį|šios|šioje|šiam|šito|šitą|šiuo|šiek|šitame|šiame)\b",
r"\b(kitą|kitu|kitų|kiti|kuris|kurie|kurio|kuriuo|kuriuos|kuriam|kurių|kurios|k
↪  uria|kuri|kurį|kurią|ko|ką|kokio)\b",
r"\b(jo|jos|juo|ja|jie|jį|ją|jų|jūsų|jam|jai|aš|mes|jūs|man|mums)\b",
r"\b(viso|visi|visiems|visų|vieno|vieną)\b",
```

```
r"\b(dėl|už|prieš|ir|nes|kad|kaip|kur|tai|į|iš|nuo|iki|per|po|apie|taip|bet|ar|
↪  o|bei)\b",

# Pagalbiniai / menkai informatyvūs
r"\b(būti|būtų|bus|buvo|nebuvo|nebūtų|nebus|yra|nebėra|nėra)\b",
r"\bmanau\b",
r"\bmanote\b",
r"\btikrai\b",
r"\btikr\w*\b",
r"\blabai\b",
r"\bjau\b",
r"\bčia\b",
r"\bdabar\b",
r"\bgerai\b",
r"\batrod\w*\b",
r"\brod\w*\b",
r"\baišku\b",
r"\bmatyt\b",
r"\bpavyzdžiui\b",
r"\bkitaip\b",
r"\bkartu\b",
r"\bminėj\w*\b",
r"\bapsisprend\w*\b",
r"\bpasak\w*\b",
r"\bgalvoj\w*\b",
r"\batsakyk\w*\b",
r"\bprikalb\w*\b",
r"\bspekuliuoj\w*\b",

# Balsavimo technika
r"\bnari\w*\b",
r"\bbiuleten\w*\b",
r"\bgaliojanč\w*\b",
r"\bnegaliojanč\w*\b",
r"\brast\w*\b",
r"\bišduot\w*\b",
r"\bišrinkim\w*\b",

# Instituciniai / procedūriniai likučiai
r"\bvaldymo\b",
r"\bvaldanči\w*\b",

# Iš rezultatų matytas retorinis triukšmas
r"\bekspromtu\b",
```

```
r"\bneišgąsd\w*\b",
r"\bnesuprat\w*\b",
r"\bstovėj\w*\b",
r"\bmatysim\w*\b",
r"\bpasipuikuot\w*\b",
r"\biškalb\w*\b",
r"\bproaktyv\w*\b",
r"\bkonstruktyv\w*\b",
r"\bbendradarbiaut\w*\b",
r"\bnetapt\w*\b",
r"\bnebalsuot\w*\b",
r"\bteko\b",
r"\btekt\w*\b",
r"\btrupuči\w*\b",
r"\bnesumažin\w*\b",
r"\bpadidint\w*\b",
r"\bnieko\b",
r"\bnenubyr\w*\b",
r"\bpralaimėj\w*\b",
r"\bpraeit\w*\b",
r"\bnuogąstav\w*\b",
r"\bnemažai\b",
r"\bbūsim\w*\b",
r"\bkritikuos\w*\b",
r"\bpripaž\w*\b",
r"\bnepatirt\w*\b",
r"\bpasiged\w*\b",
r"\bsužinoj\w*\b",
r"\bnesielg\w*\b",
r"\bužduo\w*\b",
r"\bpaskatin\w*\b",
r"\bnesprę\w*\b",
r"\bnūdien\w*\b",
r"\bmatys\w*\b",

r"\bdžiaug\w*\b",
r"\bpastebėj\w*\b",
r"\bakcentuoj\w*\b",
r"\bkreipi\w*\b",
r"\bprisim\w*\b",
r"\bsusimąst\w*\b",
r"\bpasisak\w*\b",
r"\bdiskutu\w*\b",
r"\bargument\w*\b",
```

```
r"\bsaulia\w*\b",
r"\bkarbausk\w*\b",

r"\bkažkok\w*\b",
r"\bįsižeid\w*\b",
r"\bneabejo\w*\b",
r"\bpasiged\w*\b",
r"\bneatrod\w*\b",
r"\bnenuspėjam\w*\b",
r"\babejing\w*\b",
r"\bsugebė\w*\b",
r"\bpasitais\w*\b",
r"\bnuomon\w*\b",
r"\btikiuos\w*\b",
r"\bnežin\w*\b",
r"\bpaaišk\w*\b",
r"\bsuprant\w*\b",

r"\batlik\w*\b",
r"\bureguli\w*\b",
r"\breglament\w*\b",
r"\bužbaig\w*\b",
r"\bsureag\w*\b",
r"\bsurast\w*\b",
r"\bsuteik\w*\b",
r"\bpadary\w*\b",
r"\bpradė\w*\b",
r"\bkeit\w*\b",
r"\bvykd\w*\b",
r"\bįgyvend\w*\b",
r"\batsispind\w*\b",
r"\batsirem\w*\b",
r"\bperplanu\w*\b",
r"\bįvertin\w*\b",
r"\bpatikslin\w*\b",
r"\bpalaik\w*\b",
r"\breagu\w*\b",

r"\bskaiči\w*\b",
r"\bkontrolin\w*\b",
r"\bparametr\w*\b",
r"\bprielaid\w*\b",
r"\bformalum\w*\b",
```

```
    r"\bpatyrim\w*\b",
    r"\bstadij\w*\b",
    r"\bpirmin\w*\b",
    r"\btrimeč\w*\b",
    r"\basignavim\w*\b",

    r"\bvardu\b",
    r"\blydim\w*\b",
    r"\bpaket\w*\b",
    r"\bpateik\w*\b",
    r"\bsvarstym\w*\b",
    r"\bpriėmim\w*\b",
    r"\bpritart\w*\b",
    r"\bnepritart\w*\b",
    r"\bapsispręs\w*\b",
    r"\bapsisprend\w*\b",
    r"\bnukelt\w*\b",
    r"\bįtraukt\w*\b",
    r"\bperkelt\w*\b",
    r"\bpatvirtint\w*\b",
    r"\bskelb\w*\b",
    r"\bbendruomen\w*\b",
    r"\bišvad\w*\b",
    r"\blygmen\w*\b",
    r"\bprojekt\w*\b",
    r"\blydin\w*\b",
    r"\bpaket\w*\b",
    r"\bsiūlym\w*\b",
    r"\bpasiūl\w*\b",
    r"\bkomisij\w*\b",
    r"\bkomitet\w*\b",
    r"\bdepartament\w*\b",
    r"\bministerij\w*\b",
    r"\bįstaig\w*\b",
    r"\btarnyb\w*\b",
]

COMBINED_REGEX = re.compile("|".join(REGEX_PATTERNS), flags=re.IGNORECASE)

NR_REGEX = re.compile(r"\bNr\.\s*[A-Z0-9-]+\b", flags=re.IGNORECASE)
BRACKETS_REGEX = re.compile(r"\(.*?\)", flags=re.DOTALL)
ROMAN_REGEX = re.compile(r"\b[IVXLCDM]+\b", flags=re.IGNORECASE)
DIGITS_REGEX = re.compile(r"\d+")
```

```
BASIC_ALLOWED_REGEX = re.compile(r"[^a-ząčęėįšųūž\.\,\!\?\:\;\-\(\)\s]",
↪  flags=re.IGNORECASE)
SPACES_REGEX = re.compile(r"\s+")

SPEAKER_REGEX = re.compile(
    r"\b[A-ZĄČĘĖĮŠŲŪŽ]\.\s*[A-ZĄČĘĖĮŠŲŪŽa-ząčęėįšųūž\-]+\.?:?"
)

NAME_PAIR =
↪  r"[A-ZĄČĘĖĮŠŲŪŽ][a-ząčęėįšųūž]+(?:\s+[A-ZĄČĘĖĮŠŲŪŽ][a-ząčęėįšųūž]+){1,2}"
NAME_LIST_REGEX = re.compile(rf"(?:{NAME_PAIR}\s+){{4,}}")


def remove_speakers_and_name_lists(text):
    text = SPEAKER_REGEX.sub(" ", text)
    text = NAME_LIST_REGEX.sub(" ", text)
    return text


def regex_clean(text):
    text = NR_REGEX.sub(" ", text)
    text = COMBINED_REGEX.sub(" ", text)
    return SPACES_REGEX.sub(" ", text).strip()


def basic_clean(text):
    text = DIGITS_REGEX.sub(" ", text)
    text = BASIC_ALLOWED_REGEX.sub(" ", text)
    return text


def clean_punctuation_artifacts(text):
    text = re.sub(r"\s+[--]\s*[.,!?;:]\s+", " ", text)
    text = re.sub(r"[.,!?;:]\s+[--]\s*[.,!?;:]", " ", text)
    text = re.sub(r"\.\s+-\s+", ". ", text)
    text = re.sub(r"\s+([.,!?;:])", r"\1", text)
    text = re.sub(r"\s*-\s*", " - ", text)
    text = re.sub(r"([.,!?;:]){2,}", r"\1", text)
    return SPACES_REGEX.sub(" ", text).strip()


def stopwords_valymas(text, stopwords):
    return " ".join(
        w for w in text.split()
```

```
        if w not in stopwords
    )


def clean(text, stopwords):
    text = str(text)
    text = BRACKETS_REGEX.sub(" ", text)
    text = remove_speakers_and_name_lists(text)
    text = ROMAN_REGEX.sub(" ", text)
    text = text.lower()
    text = regex_clean(text)
    text = basic_clean(text)
    text = clean_punctuation_artifacts(text)
    text = stopwords_valymas(text, stopwords)
    text = SPACES_REGEX.sub(" ", text).strip()
    # Pašalina per trumpus tekstus
    if len(text.split()) <= 25:
        return np.nan

    return text

tqdm.pandas()

df1["clean_text"] = df1["skyrelis"].progress_apply(
    lambda x: clean(x, stopwords)
)

df2["clean_text"] = df2["skyrelis"].progress_apply(
    lambda x: clean(x, stopwords)
)

df1 = df1.dropna(subset=["clean_text"]).copy()
df2 = df2.dropna(subset=["clean_text"]).copy()

text_col = "clean_text"
extract_top_n = 20
final_top_n = 3

model_name = "EMBEDDIA/litlat-bert"

word_embedding_model = models.Transformer(
    model_name,
    max_seq_length=512
)
```

```
pooling_model = models.Pooling(
    word_embedding_model.get_word_embedding_dimension(),
    pooling_mode_mean_tokens=True
)

sentence_model = SentenceTransformer(
    modules=[word_embedding_model, pooling_model],
    device="cuda"
)

kw_model = KeyBERT(model=sentence_model)
tokenizer = sentence_model._first_module().tokenizer

def split_into_sentences(text):
    text = str(text).strip()

    sentences = re.split(
        r"(?<=[.!?])\s+",
        text
    )

    return [
        s.strip()
        for s in sentences
        if s.strip()
    ]


def count_tokens(text):
    return (
        len(tokenizer.tokenize(str(text)))
        + tokenizer.num_special_tokens_to_add(pair=False)
    )


def split_into_chunks(text, max_tokens=480):
    sentences = split_into_sentences(text)

    chunks = []
    current_sentences = []

    for sent in sentences:
        sent_tokens = count_tokens(sent)
```

```
        if sent_tokens > max_tokens:
            if current_sentences:
                chunks.append(" ".join(current_sentences))
                current_sentences = []
            continue

        candidate = " ".join(current_sentences + [sent])
        candidate_tokens = count_tokens(candidate)

        if candidate_tokens <= max_tokens:
            current_sentences.append(sent)
        else:
            if current_sentences:
                chunks.append(" ".join(current_sentences))
            current_sentences = [sent]

    if current_sentences:
        chunks.append(" ".join(current_sentences))

    return chunks

def clean_phrase_for_lemma(phrase):
    phrase = str(phrase).lower().strip()

    phrase = re.sub(
        r"[^\wąčęėįšųūž\s]",
        " ",
        phrase
    )

    phrase = re.sub(
        r"\s+",
        " ",
        phrase
    ).strip()

    return phrase


def analyze_phrase_spacy(phrase):
    phrase = clean_phrase_for_lemma(phrase)

    if not phrase:
```

```
    return "", (), False

doc = nlp_spacy(phrase)

lemmas = []
pos_tags = []

for token in doc:
    if token.is_space or token.is_punct:
        continue

    lemma = token.lemma_.lower().strip()
    pos = token.pos_

    if lemma:
        lemmas.append(lemma)
    else:
        lemmas.append(token.text.lower())

    pos_tags.append(pos)

lemma_key = " ".join(lemmas)
pos_pattern = tuple(pos_tags)

has_noun = any(
    p in {"NOUN", "PROPN"}
    for p in pos_tags
)

verb_count = sum(
    p in {"VERB", "AUX"}
    for p in pos_tags
)

length_ok = len(pos_tags) <= 4

is_valid_pos = (
    has_noun
    and verb_count == 0
    and length_ok
)

return lemma_key, pos_pattern, is_valid_pos
```

```
def extract_keybert_phrases(df, df_name, kadencija):
    results = []

    labels = df["pred_label"].dropna().unique()

    for label in tqdm(labels, desc=f"Processing {df_name}"):

        texts = (
            df.loc[df["pred_label"] == label, text_col]
            .dropna()
            .astype(str)
            .tolist()
        )

        if not texts:
            continue

        all_chunks = []

        for text in tqdm(
            texts,
            desc=f"Chunking | {df_name} | {label}",
            leave=False
        ):
            all_chunks.extend(
                split_into_chunks(
                    text,
                    max_tokens=480
                )
            )

        all_chunks = [
            chunk
            for chunk in all_chunks
            if chunk.strip()
        ]

        if not all_chunks:
            continue

        chunk_embeddings = sentence_model.encode(
            all_chunks,
            convert_to_numpy=True,
```

```
    normalize_embeddings=True,
    show_progress_bar=True,
    batch_size=32
)

topic_center = chunk_embeddings.mean(axis=0)
topic_center = topic_center / np.linalg.norm(topic_center)

candidate_phrases = []

for chunk in tqdm(
    all_chunks,
    desc=f"KeyBERT | {df_name} | {label}",
    leave=False
):
    keywords = kw_model.extract_keywords(
        chunk,
        keyphrase_ngram_range=(2, 3),
        stop_words=list(stopwords),
        top_n=extract_top_n,
        use_mmr=True,
        diversity=0.1
    )

    if not keywords:
        continue

    for phrase, keybert_score in keywords:
        candidate_phrases.append({
            "fraze": phrase,
            "keybert_score": keybert_score
        })

if not candidate_phrases:
    continue

temp_df = pd.DataFrame(candidate_phrases)

phrase_df = (
    temp_df
    .groupby("fraze", as_index=False)
    .agg(
        keybert_score=("keybert_score", "mean"),
        pasikartojimai=("keybert_score", "count")
```

```
    )
)

phrases = phrase_df["fraze"].tolist()

phrase_embeddings = sentence_model.encode(
    phrases,
    convert_to_numpy=True,
    normalize_embeddings=True,
    show_progress_bar=True,
    batch_size=32
)

phrase_df["topic_center_score"] = phrase_embeddings @ topic_center

phrase_df = (
    phrase_df
    .sort_values(
        by="topic_center_score",
        ascending=False
    )
    .head(250)
    .copy()
)

tqdm.pandas(desc=f"spaCy POS | {df_name} | {label}")

phrase_analysis = (
    phrase_df["fraze"]
    .progress_apply(analyze_phrase_spacy)
)

phrase_df["lemma_key"] = phrase_analysis.apply(lambda x: x[0])
phrase_df["pos_pattern"] = phrase_analysis.apply(lambda x: x[1])
phrase_df["valid_pos"] = phrase_analysis.apply(lambda x: x[2])

phrase_df = phrase_df[
    (phrase_df["lemma_key"].str.strip() != "")
    &
    (phrase_df["valid_pos"])
]

phrase_df = (
    phrase_df
```

```
            .drop_duplicates(
                subset=["lemma_key"],
                keep="first"
            )
            .sort_values(
                by=["topic_center_score", "keybert_score", "pasikartojimai"],
                ascending=[False, False, False]
            )
            .head(final_top_n)
            .copy()
        )

        for rank, row in enumerate(phrase_df.itertuples(index=False), start=1):
            results.append({
                "dataset": df_name,
                "kadencija": kadencija,
                "tema": label,
                "rank": rank,
                "fraze": row.fraze,
                "lemma_key": row.lemma_key,
                "pos_pattern": row.pos_pattern,
                "keybert_score": round(row.keybert_score, 5),
                "topic_center_score": round(row.topic_center_score, 5),
                "pasikartojimai": row.pasikartojimai
            })

    return pd.DataFrame(results)

df1_keybert_phrases = extract_keybert_phrases(
    df1,
    "df1",
    "2016-2020"
)

df1_keybert_phrases.to_parquet(
    "2016-2020_keybert_phrases_top3.parquet",
    index=False
)

df2_keybert_phrases = extract_keybert_phrases(
    df2,
    "df2",
    "2020-2024"
)
```

```
df2_keybert_phrases.to_parquet(
    "2020-2024_keybert_phrases_top3.parquet",
    index=False
)


# KONTEKSTAI

TOP_N_PHRASES = 2
TEXT_COL    = "tekstas"
CADENCE_COL = "kadencija"
MIN_WORDS   = 3

# Duomenys
df_sten = pd.read_parquet("seimo_stenogramos_2016_2024.parquet")

df_frazes = pd.concat([
    pd.read_parquet("/content/tfidf_frazes_2016_2020.parquet"),
    pd.read_parquet("/content/tfidf_frazes_2020_2024.parquet"),
], ignore_index=True)

print(f"Stenogramos:   {len(df_sten)} eilučių")
print(f"Frazės (viso): {len(df_frazes)} eilučių")
print(f"Kadencijos:    {sorted(df_frazes['kadencija'].unique())}")
print(f"Temos:         {sorted(df_frazes['tema'].unique())}")
print()

# Iš failo atrenkame po tris frazes temai
df_top = (
    df_frazes
    .sort_values("vieta")
    .groupby(["kadencija", "tema"], group_keys=False)
    .head(TOP_N_PHRASES)
    .reset_index(drop=True)
)

print(f"Pasirinktos frazės (TOP {TOP_N_PHRASES} kiekvienoje temoje):")
for _, r in df_top.iterrows():
    print(f"  [{r['kadencija']}] {r['tema']:40s}  vieta={r['vieta']}
    ↪  '{r['fraze']}'")
print()

keyphrases = {}
```

```
for _, row in df_top.iterrows():
    keyphrases \
        .setdefault(row["kadencija"], {}) \
        .setdefault(row["tema"], []) \
        .append(row["fraze"])

# Atliekamas minimalus teksto valymas
PARTY_PATTERNS = [
    r'Tėvynės\s+sąjungos?(?:\s*[\--]\s*Lietuvos\s+krikščion\w+\s+demokrat\w+)?',
    r'Lietuvos\s+socialdemokrat\w+',
    r'Lietuvos\s+valstiečių\s+ir\s+žaliųjų\s+sąjungos?',
    r'Valstiečių\s+ir\s+žaliųjų\s+sąjungos?',
    r'Darbo\s+partij\w+',
    r'Liberalų\s+sąjūdžio?',
    r'Laisvės\s+partij\w+',
    r'Liberalų\s+ir\s+centro\s+sąjungos?',
    r'Lietuvos\s+lenkų\s+rinkimų\s+akcij\w+',
    r'Krikščionių\s+demokratų\s+sąjungos?',
    r'Lietuvos\s+žaliųjų\s+partij\w+',
    r'Lietuvos\s+centro\s+partij\w+',
    r'Demokratų\s+frakcij\w+',
]

def clean_text(text: str) -> str:
    if not text or not isinstance(text, str):
        return text

    ABBREV_MAP = {
        r'\bNr\.':      'Nr<DOT>',
        r'\bstr\.':     'str<DOT>',
        r'\bskyr\.':    'skyr<DOT>',
        r'\bprof\.':    'prof<DOT>',
        r'\bdoc\.':     'doc<DOT>',
        r'\bdr\.':      'dr<DOT>',
        r'\bpan\.':     'pan<DOT>',
        r'\bpvz\.':     'pvz<DOT>',
        r'\bt\.y\.':    'ty<DOT>',
        r'\bt\.t\.':    'tt<DOT>',
        r'\bir\s+kt\.': 'ir kt<DOT>',
        r'\bkt\.':      'kt<DOT>',
        r'\bvnt\.':     'vnt<DOT>',
        r'\bproc\.':    'proc<DOT>',
        r'\bmln\.':     'mln<DOT>',
        r'\bmlrd\.':    'mlrd<DOT>',
```

```
        r'\btūkst\.':   'tūkst<DOT>',
        r'\bm\.\s*m\.': 'm<DOT> m<DOT>',
        r'(?<=\d)\s*m\.': ' m<DOT>',   # metai po skaičiaus: "2020 m."
        r'(?<=\d)\s*d\.': ' d<DOT>',   # diena po skaičiaus: "15 d."
        r'(?<=\d)\s*p\.': ' p<DOT>',   # puslapis
    }
    for pat, repl in ABBREV_MAP.items():
        text = re.sub(pat, repl, text)

    text = re.sub(r'\([^)]*\)', ' ', text)

    text = re.sub(
        r'\b[A-ZĄČĘĖĮŠŲŪŽ]\.\s*[A-ZĄČĘĖĮŠŲŪŽ][A-ZĄČĘĖĮŠŲŪŽa-ząčęėįšųūž]{1,}\w*\b',
        ' ', text
    )

    text = re.sub(r'(?<!\w)[A-ZĄČĘĖĮŠŲŪŽ]\.(?!\w)', ' ', text)

    for pat in PARTY_PATTERNS:
        text = re.sub(pat, ' ', text, flags=re.IGNORECASE)

    text = re.sub(r'\b[A-ZĄČĘĖĮŠŲŪŽ]{2,}\b', ' ', text)
    text = re.sub(r'\b(?:X{1,3}|IX|IV|VI{0,3}|II{0,2}|I{1,3})\b', ' ', text)
    text = re.sub(r'\b\d+(?:[.,]\d+)?\b', ' ', text)
    text = re.sub(r'(\w)\s+\.', r'\1.', text)
    text = re.sub(r'\.(\s*\.)+', '.', text)
    text = re.sub(r'^\s*\.\s*', '', text)
    text = re.sub(r'(?<!\w)[,;:\--](?!\w)', ' ', text)
    text = re.sub(r'\s+', ' ', text).strip()

    return text

# Skaidome į sakinius
def split_sentences(text: str) -> list:
    if pd.isna(text):
        return []
    text = str(text).replace("\n", " ")
    text = re.sub(r"\s+", " ", text).strip()
    parts = re.split(r'(?<=[.!?])\s+(?=[A-ZĄČĘĖĮŠŲŪŽ„\"\u201E])', text)
    # <DOT> → '.' grąžiname čia, kiekviename sakinyje atskirai
    return [p.replace('<DOT>', '.').strip() for p in parts if p.strip()]

def word_count(sentence: str) -> int:
    return len(re.findall(r'\b\w{2,}\b', sentence))
```

```
def phrase_in_sentence(phrase: str, sentence: str) -> bool:
    return re.search(re.escape(phrase), sentence, flags=re.IGNORECASE) is not None


# Konteksto ištraukimo funkcija BERT modeliui
def get_bert_context(sentences: list, idx: int) -> str:
    """
    Grąžina kontekstą iš lygiai 3 sakinių:
      - tikslinis sakinys (sentences[idx])
      - po vieną sakinį prieš ir po jo (jei yra ir turi MIN_WORDS žodžių)

    Jei tiesioginis kaimynas per trumpas arba jo nėra,
    ieškoma toliau (iki 2 pozicijų) kad surinktume 3 pilnus sakinius.
    """
    n = len(sentences)

    def valid(s: str) -> bool:
        return word_count(s) >= MIN_WORDS

    prev = ""
    for offset in range(1, 3):
        if idx - offset < 0:
            break
        if valid(sentences[idx - offset]):
            prev = sentences[idx - offset]
            break

    nxt = ""
    for offset in range(1, 3):
        if idx + offset >= n:
            break
        if valid(sentences[idx + offset]):
            nxt = sentences[idx + offset]
            break

    parts = [s for s in [prev, sentences[idx], nxt] if s]
    return " ".join(parts).strip()

# Konteksto ištraukimas
rows = []
total = len(df_sten)

for idx, row in df_sten.iterrows():
```

```
    cadence  = row[CADENCE_COL]
    raw_text = row[TEXT_COL]

    if cadence not in keyphrases:
        continue

    cleaned   = clean_text(raw_text)
    sentences = split_sentences(cleaned)

    for topic, phrases in keyphrases[cadence].items():
        for phrase in phrases:
            for i, sent in enumerate(sentences):
                if phrase_in_sentence(phrase, sent):

                    rows.append({
                        "cadence":          cadence,
                        "topic":            topic,
                        "phrase":           phrase,
                        "sentence_index":   i,
                        "vader_context_lt": sent,
                        "bert_context_lt":  get_bert_context(sentences, i),
                    })

    if (idx + 1) % 500 == 0:
        print(f"  Apdorota: {idx + 1}/{total}, kontekstai: {len(rows)}")

print(f"\nViso kontekstų: {len(rows)}")

contexts_df = pd.DataFrame(rows)
contexts_df.insert(0, "context_id", range(1, len(contexts_df) + 1))

# Patikriname kokybę
bert_same_as_vader = (contexts_df["bert_context_lt"] ==
↪  contexts_df["vader_context_lt"]).sum()
sent_counts = contexts_df["bert_context_lt"].str.count(r'[.!?]')

print(f"\nKOKYBĖS ATASKAITA:")
print(f"  bert == vader (tik 1 sakinys):    {bert_same_as_vader}
↪  ({bert_same_as_vader/len(contexts_df)*100:.1f}%)")
print(f"  bert su 2+ skyrybos ženklais:     {(sent_counts >= 2).sum()}
↪  ({(sent_counts >= 2).sum()/len(contexts_df)*100:.1f}%)")
print(f"  Vidutinis bert ilgis (simboliai):
↪  {contexts_df['bert_context_lt'].str.len().mean():.0f}")
```

```
# Išsaugome kontekstus vertimui
out = Path("kontekstai_vertimui.parquet")
contexts_df.to_parquet(out, index=False)
print(f"\nIšsaugota: {out}  ({len(contexts_df)} eilučių)")
print(f"Stulpeliai: {list(contexts_df.columns)}")
print(contexts_df.head(3).to_string())


# VERTIMAS

# Nustatome modelio pavadinimą ir failų kelius
MODEL_NAME = "Helsinki-NLP/opus-mt-tc-big-lt-en"

INPUT_FILE  = Path("/content/kontekstai_vertimui.parquet")
OUTPUT_FILE = Path("/content/kontekstai_isversti.parquet")

BATCH_SIZE = 32
MAX_LEN    = 512

#Tikrinamas GPU prieinamumas
print("CUDA:", torch.cuda.is_available())

if torch.cuda.is_available():
    print("GPU:", torch.cuda.get_device_name(0))
else:
    print("GPU nerastas")

device = "cuda" if torch.cuda.is_available() else "cpu"

# Išvaloma atmintis prieš įkeliant modelį
gc.collect()

if torch.cuda.is_available():
    torch.cuda.empty_cache()

# Duomenys
df = pd.read_parquet(INPUT_FILE)

print(f"\nĮkelta eilučių: {len(df)}")

all_lt_texts = pd.unique(
    pd.concat([
        df["vader_context_lt"],
        df["bert_context_lt"]
```

```
    ])
).tolist()

all_lt_texts = [
    t for t in all_lt_texts
    if isinstance(t, str) and t.strip()
]

print(f"Unikalių tekstų vertimui: {len(all_lt_texts)}")

# Modelis
print(f"\nĮkeliamas modelis: {MODEL_NAME}")

tokenizer = MarianTokenizer.from_pretrained(MODEL_NAME)
model     = MarianMTModel.from_pretrained(MODEL_NAME)

model.to(device)
model.eval()

print("Modelis įkeltas")

# Vertimo funkcija
def translate_batch(texts):

    inputs = tokenizer(
        texts,
        return_tensors="pt",
        padding=True,
        truncation=True,
        max_length=MAX_LEN,
    )

    inputs = {k: v.to(device) for k, v in inputs.items()}

    with torch.no_grad():
        outputs = model.generate(**inputs)

    decoded = tokenizer.batch_decode(outputs, skip_special_tokens=True)

    del inputs, outputs

    if torch.cuda.is_available():
        torch.cuda.empty_cache()
```

```
    return decoded

# Vertimas
def translate_all(texts):

    translation_map = {}
    total = (len(texts) + BATCH_SIZE - 1) // BATCH_SIZE

    for i in range(total):

        batch = texts[i * BATCH_SIZE : (i + 1) * BATCH_SIZE]

        print(f"Paketas {i+1}/{total} ({len(batch)} tekstų)", flush=True)

        try:
            translated = translate_batch(batch)
            for lt, en in zip(batch, translated):
                translation_map[lt] = en

        except Exception as e:
            print(f"KLAIDA: {e}")
            for lt in batch:
                translation_map[lt] = None

    return translation_map

print("\nVerčiama\n")

translation_map = translate_all(all_lt_texts)

df["vader_context_en"] = df["vader_context_lt"].map(translation_map)
df["bert_context_en"]  = df["bert_context_lt"].map(translation_map)


# Rezultatai
ok      = sum(1 for v in translation_map.values() if v is not None)
missing = df["vader_context_en"].isna().sum()

print(f"\nIšversta: {ok}/{len(all_lt_texts)}")

if missing:
    print(f"Neišversta: {missing}")
```

```
# Pavyzdžiai
print("\nVERTIMŲ PAVYZDŽIAI:\n")

for _, row in df.dropna(subset=["vader_context_en"]).sample(3,
↪  random_state=42).iterrows():
    print("=" * 80)
    print("\n[LT]")
    print(row["vader_context_lt"])
    print("\n[EN]")
    print(row["vader_context_en"])


df.to_parquet(OUTPUT_FILE, index=False)
print(f"\nIšsaugota: {OUTPUT_FILE}")


# SENTIMENTŲ ANALIZĖ

# Duomenys ir failų keliai
INPUT_FILE              = Path("kontekstai_isversti.parquet")
OUTPUT_INDIVIDUAL_FILE  = Path("sentimentai_kiekvienam_pasirodymui.parquet")
OUTPUT_SUMMARY_FILE     = Path("sentimentai_pagal_frazes.parquet")

df = pd.read_parquet(INPUT_FILE)

print(f"Įkelta: {len(df)} eilučių")
print(f"Stulpeliai: {list(df.columns)}")
print()

# VADER sentimentai
nltk.download("vader_lexicon", quiet=True)

vader = SentimentIntensityAnalyzer()

def vader_score(text) -> float:
    if pd.isna(text) or str(text).strip() == "":
        return np.nan
    return vader.polarity_scores(str(text))["compound"]

def sentiment_label(score) -> str:
    if pd.isna(score):
        return np.nan
    elif score >= 0.05:
        return "positive"
```

```
    elif score <= -0.05:
        return "negative"
    else:
        return "neutral"

print("Skaičiuojamas VADER sentimentas...")

df["vader_score"] = df["vader_context_en"].apply(vader_score)
df["vader_label"] = df["vader_score"].apply(sentiment_label)

print(f"  Paskirstymas:\n{df['vader_label'].value_counts().to_string()}")
print()


# BERT sentimentai
device = 0 if torch.cuda.is_available() else -1

print("CUDA:", torch.cuda.is_available())

if torch.cuda.is_available():
    print("GPU:", torch.cuda.get_device_name(0))

print("Įkeliamas BERT modelis...")

bert_model = pipeline(
    "sentiment-analysis",
    model="phanerozoic/BERT-Sentiment-Classifier",
    device=device
)

texts = df["bert_context_en"].fillna("").astype(str).tolist()

batch_size = 128

bert_labels = []
bert_scores = []

print("Skaičiuojamas BERT sentimentas...")

for i in tqdm(range(0, len(texts), batch_size), desc="GPU sentiment"):

    batch = texts[i:i + batch_size]

    outputs = bert_model(
```

```
        batch,
        truncation=True,
        max_length=512
    )

    for o in outputs:

        label = o["label"]

        if label == "LABEL_1":
            label = "positive"
        elif label == "LABEL_0":
            label = "negative"

        bert_labels.append(label.lower())
        bert_scores.append(o["score"])

df["bert_label"] = bert_labels
df["bert_confidence"] = bert_scores

print(f"  Paskirstymas:\n{df['bert_label'].value_counts().to_string()}")
print()

# Rezultatai kiekvienam pasirodymui
df.to_parquet(OUTPUT_INDIVIDUAL_FILE, index=False)

print(f"Išsaugota: {OUTPUT_INDIVIDUAL_FILE}  ({len(df)} eilučių)")



# Suvestinė pagal frazes
def bert_final_label(row) -> str:

    shares = {
        "positive": row["bert_positive_share"],
        "negative": row["bert_negative_share"],
    }

    return max(shares, key=shares.get)

phrase_summary = (
    df
    .groupby(["cadence", "topic", "phrase"])
    .agg(
        occurrences          = ("context_id", "count"),
```

```
        vader_mean           = ("vader_score", "mean"),
        vader_median         = ("vader_score", "median"),

        vader_positive_share = ("vader_label", lambda x: (x == "positive").mean()),
        vader_negative_share = ("vader_label", lambda x: (x == "negative").mean()),
        vader_neutral_share  = ("vader_label", lambda x: (x == "neutral").mean()),

        bert_positive_share  = ("bert_label", lambda x: (x == "positive").mean()),
        bert_negative_share  = ("bert_label", lambda x: (x == "negative").mean()),

        bert_avg_confidence  = ("bert_confidence", "mean"),
    )
    .reset_index()
)

phrase_summary["vader_final_label"] = (
    phrase_summary["vader_mean"]
    .apply(sentiment_label)
)

# BERT sentimentas pagal daugumą
phrase_summary["bert_final_label"] = (
    phrase_summary.apply(bert_final_label, axis=1)
)

phrase_summary.to_parquet(OUTPUT_SUMMARY_FILE, index=False)

print(f"Išsaugota: {OUTPUT_SUMMARY_FILE}  ({len(phrase_summary)} frazių)")


# 6. Atvaizduojame rezultatus
pd.set_option("display.max_columns", None)
pd.set_option("display.width", 160)
pd.set_option("display.float_format", "{:.3f}".format)

print()
print("=" * 80)
print("SENTIMENTŲ SUVESTINĖ PAGAL FRAZĘ")
print("=" * 80)

for cadence in sorted(phrase_summary["cadence"].unique()):

    print(f"\n  Kadencija: {cadence}  ")
```

```
    subset = phrase_summary[
        phrase_summary["cadence"] == cadence
    ].copy()

    display_cols = [
        "topic",
        "phrase",
        "occurrences",

        "vader_mean",
        "vader_final_label",

        "bert_positive_share",
        "bert_negative_share",

        "bert_final_label",
    ]

    print(subset[display_cols].to_string(index=False))

print()
print("=" * 80)
print("TRUMPA SANTRAUKA")
print("=" * 80)

print(f"\nIš viso analizuota frazių: {len(phrase_summary)}")
print(f"Iš viso kontekstų (pasirodymų): {phrase_summary['occurrences'].sum()}")

for cadence in sorted(phrase_summary["cadence"].unique()):

    sub = phrase_summary[
        phrase_summary["cadence"] == cadence
    ]

    print(f"\n{cadence}:")

    print(
        f"  VADER: "
        f"{(sub['vader_final_label']=='positive').sum()} teigiamos, "
        f"{(sub['vader_final_label']=='negative').sum()} neigiamos, "
        f"{(sub['vader_final_label']=='neutral').sum()} neutralios"
    )
```

```
    print(
        f"  BERT:  "
        f"{(sub['bert_final_label']=='positive').sum()} teigiamos, "
        f"{(sub['bert_final_label']=='negative').sum()} neigiamos"
    )



# GRAFIKAI

# Nuskaitome failą
df = pd.read_parquet("sentimentai_pagal_frazes.parquet")

# Nustatome ka iš duomenų naudosime
TOP_N_TOPICS = 8
topics = sorted(df["topic"].unique())[:TOP_N_TOPICS]
cadences = sorted(df["cadence"].unique())

# Nustatome temų pavadinimus
TOPIC_LABELS = {
    "gynyba_ir_uzsienis": "Gynyba ir užsienio politika",
    "ekonomika_ir_mokesciai": "Ekonomika ir finansai",
    "socialine_politika": "Socialinė politika",
    "teise_ir_teisetvarka": "Teisė ir teisėtvarka",
    "vidaus_politika_ir_institucijos": "Vidaus politika ir institucijos",
    "aplinka_ir_zemes_ukis": "Aplinka ir žemės ūkis",
    "energetika_ir_infrastruktura": "Energetika ir infrastruktūra",
    "kultura_ir_istorija": "Kultūra ir istorija",
}

# Formatuojame y ašies procentus
def percent_no_minus(val, _):
    return f"{abs(val):.0f}%"


# Procentų korekcija, kad apvalinant neviršytų 100%
def fix_percentages(p_arr, n_arr, ng_arr):

    fixed_pos = []
    fixed_neu = []
    fixed_neg = []

    for p, n, ng in zip(p_arr, n_arr, ng_arr):
```

```
        vals = np.array([p, n, ng])

        # apvalinimas
        rounded = np.round(vals)

        # skirtumas iki 100
        diff = 100 - rounded.sum()

        # korekcija didžiausiam segmentui
        rounded[np.argmax(vals)] += diff

        fixed_pos.append(rounded[0])
        fixed_neu.append(rounded[1])
        fixed_neg.append(rounded[2])

    return (
        np.array(fixed_pos),
        np.array(fixed_neu),
        np.array(fixed_neg)
    )

# Formatuojame grafikų panelę

def plot_panel(df, cadence, label_col, title):

    data = df[df["cadence"] == cadence]

    fig, axes = plt.subplots(2, 4, figsize=(22, 10))
    axes = axes.flatten()

    for i, topic in enumerate(topics):

        ax = axes[i]

        subset = data[data["topic"] == topic]

        if subset.empty:
            ax.set_title(
                TOPIC_LABELS.get(topic, topic),
                fontsize=10
            )
            ax.axis("off")
            continue
```

```
# VADER grafikai
if label_col == "vader_label":

    subset = subset.sort_values("vader_mean", ascending=False)

    phrases = subset["phrase"].values[:10]

    raw_pos = subset["vader_positive_share"].values[:10] * 100
    raw_neu = subset["vader_neutral_share"].values[:10] * 100
    raw_neg = subset["vader_negative_share"].values[:10] * 100

    pos, neu, neg_abs = fix_percentages(
        raw_pos,
        raw_neu,
        raw_neg
    )

    half_neu = neu / 2

    x = np.arange(len(phrases))
    min_display = 5

    ax.bar(x,  half_neu,                    color="khaki")
    ax.bar(x, -half_neu,                    color="khaki")

    ax.bar(
        x,
        pos,
        bottom=half_neu,
        color="lightgreen"
    )

    ax.bar(
        x,
        -neg_abs,
        bottom=-half_neu,
        color="lightcoral"
    )

    for xi, (p, n, ng) in enumerate(zip(pos, neu, neg_abs)):

        if p >= min_display:
            ax.text(
```

```
                xi,
                half_neu[xi] + p / 2,
                f"{p:.0f}%",
                ha="center",
                va="center",
                fontsize=6,
                fontweight="bold",
                color="darkgreen"
            )

        if n >= min_display:
            ax.text(
                xi,
                0,
                f"{n:.0f}%",
                ha="center",
                va="center",
                fontsize=6,
                fontweight="bold",
                color="goldenrod"
            )

        if ng >= min_display:
            ax.text(
                xi,
                -half_neu[xi] - ng / 2,
                f"{ng:.0f}%",
                ha="center",
                va="center",
                fontsize=6,
                fontweight="bold",
                color="darkred"
            )

# BERT grafikai
else:

    subset = subset.sort_values(
        "bert_positive_share",
        ascending=False
    )

    phrases = subset["phrase"].values[:10]
```

```
pos = subset["bert_positive_share"].values[:10] * 100
neg_abs = subset["bert_negative_share"].values[:10] * 100

net = (
    subset["bert_positive_share"].values[:10]
    - subset["bert_negative_share"].values[:10]
) * 100

x = np.arange(len(phrases))
min_display = 5

ax.bar(x, pos, color="lightgreen")

ax.bar(x, -neg_abs, color="lightcoral")

for xi, (p, ng) in enumerate(zip(pos, neg_abs)):

    if p >= min_display:
        ax.text(
            xi,
            p / 2,
            f"{p:.0f}%",
            ha="center",
            va="center",
            fontsize=6,
            fontweight="bold",
            color="darkgreen"
        )

    if ng >= min_display:
        ax.text(
            xi,
            -ng / 2,
            f"{ng:.0f}%",
            ha="center",
            va="center",
            fontsize=6,
            fontweight="bold",
            color="darkred"
        )

ax.plot(
    x,
    net,
```

```
            color="black",
            linewidth=1.2,
            marker="o",
            markersize=4,
            zorder=5
        )

    # Formatuojame ašis ir etiketes
    ax.yaxis.set_major_formatter(
        plt.FuncFormatter(percent_no_minus)
    )

    ax.set_title(
        TOPIC_LABELS.get(topic, topic),
        fontsize=10
    )

    ax.set_xticks(x)

    ax.set_xticklabels(
        phrases,
        rotation=45,
        ha="right",
        fontsize=7
    )

    if label_col == "vader_label":
        ax.axhline(
            0,
            color="black",
            linewidth=0.8,
            zorder=0
        )
    else:
        ax.axhline(
            0,
            color="black",
            linewidth=0.8
        )

    ax.set_ylim(-100, 100)

# Sudarome legendą
if label_col == "vader_label":
```

```
        legend_elements = [
            Patch(facecolor="lightgreen", label="Teigiamas"),
            Patch(facecolor="khaki", label="Neutralus (centre)"),
            Patch(facecolor="lightcoral", label="Neigiamas"),
        ]

    else:

        legend_elements = [
            Patch(facecolor="lightgreen", label="Teigiamas"),
            Patch(facecolor="lightcoral", label="Neigiamas"),
            Line2D(
                [0], [0],
                color="black",
                marker="o",
                markersize=5,
                label="Sentimentų balansas"
            ),
        ]

    fig.legend(
        handles=legend_elements,
        loc="lower center",
        ncol=4,
        fontsize=10,
        frameon=True,
        bbox_to_anchor=(0.5, -0.02)
    )

    fig.suptitle(title, fontsize=18)

    plt.tight_layout(rect=[0, 0.04, 1, 1])

    plt.show()

# VADER panelės
for cadence in cadences:

    plot_panel(
        df=df,
        cadence=cadence,
        label_col="vader_label",
        title=f"VADER sentimentai | Kadencija {cadence}"
```

```
    )

# BERT panelės
for cadence in cadences:

    plot_panel(
        df=df,
        cadence=cadence,
        label_col="bert_label",
        title=f"BERT sentimentai | Kadencija {cadence}"
    )
```